

AZ500 Lab10: Key Vault (Implementing Secure Data by setting up Always Encrypted)

Introduction

This lab presents a scenario where one is tasked with building a proof-of-concept secure data solution in Microsoft Azure by combining **Azure Key Vault** with **Azure SQL Database Always Encrypted**. The goal is to protect sensitive data—such as personal or regulated fields—by ensuring that encryption keys are created, stored, and managed centrally in Key Vault, while data remains encrypted both at rest and in transit to the database. The proof of concept includes deploying a prepared environment (VM with development tools and an Azure SQL Database), creating and storing keys and secrets in Key Vault, registering an application in Microsoft Entra ID for secure access, and encrypting selected database columns using Always Encrypted keys retrieved from the vault. All resources are deployed in the East US region (subject to instructor confirmation), keeping the focus on modern cloud-native data protection practices.

Lab scenario

You have been asked to create a proof of concept application that makes use of the Azure SQL Database support for Always Encrypted functionality. All of the secrets and keys used in this scenario should be stored in Key Vault. The application should be registered in Microsoft Entra ID in order to enhance its security posture. To accomplish these objectives, the proof of concept should include:

- Creating an Azure Key Vault and storing keys and secrets in the vault.
- Create a SQL Database and encrypting content of columns in database tables by using Always Encrypted.

Note: For all the resources in this lab, we are using the **East US** region.

To keep the focus on the security aspects of Azure, related to building this proof of concept, you will start from an automated ARM template deployment, setting up a Virtual Machine with Visual Studio 2019 and SQL Server Management Studio 19.

Lab objectives

In this lab, I will complete the following exercises:

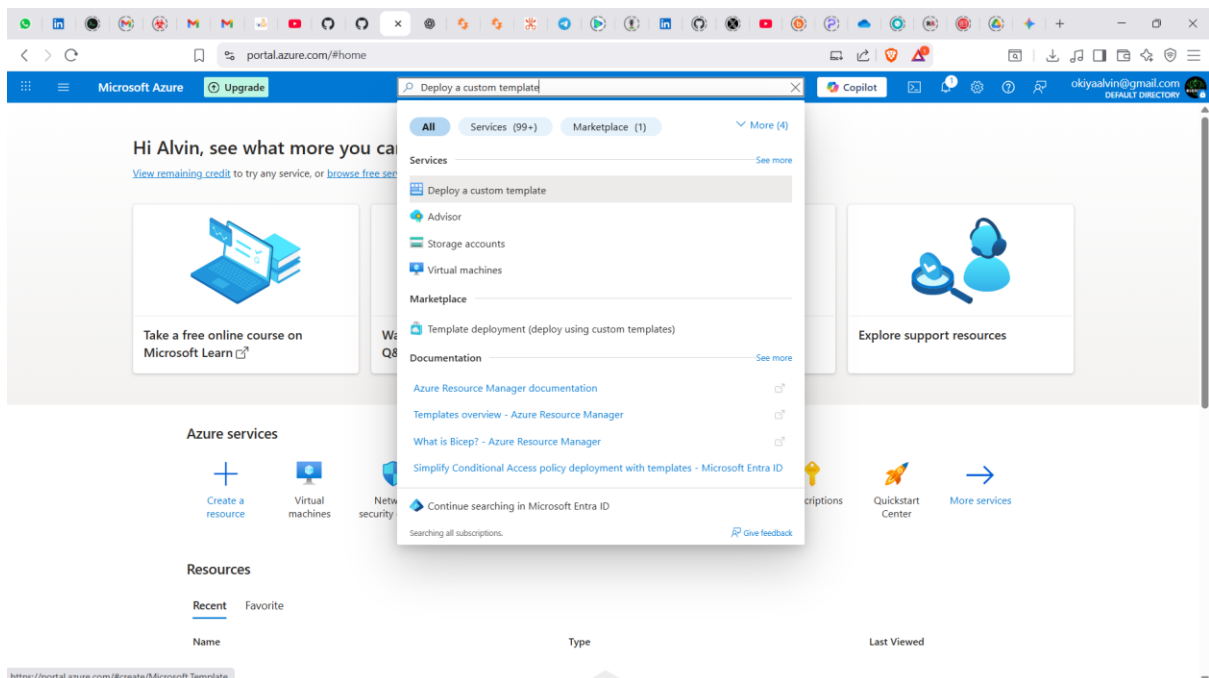
- Exercise 1: Deploy the base infrastructure from an ARM template
- Exercise 2: Configure the Key Vault resource with a key and a secret
- Exercise 3: Configure an Azure SQL database and a data-driven application

- Exercise 4: Demonstrate the use of Azure Key Vault in encrypting the Azure SQL database

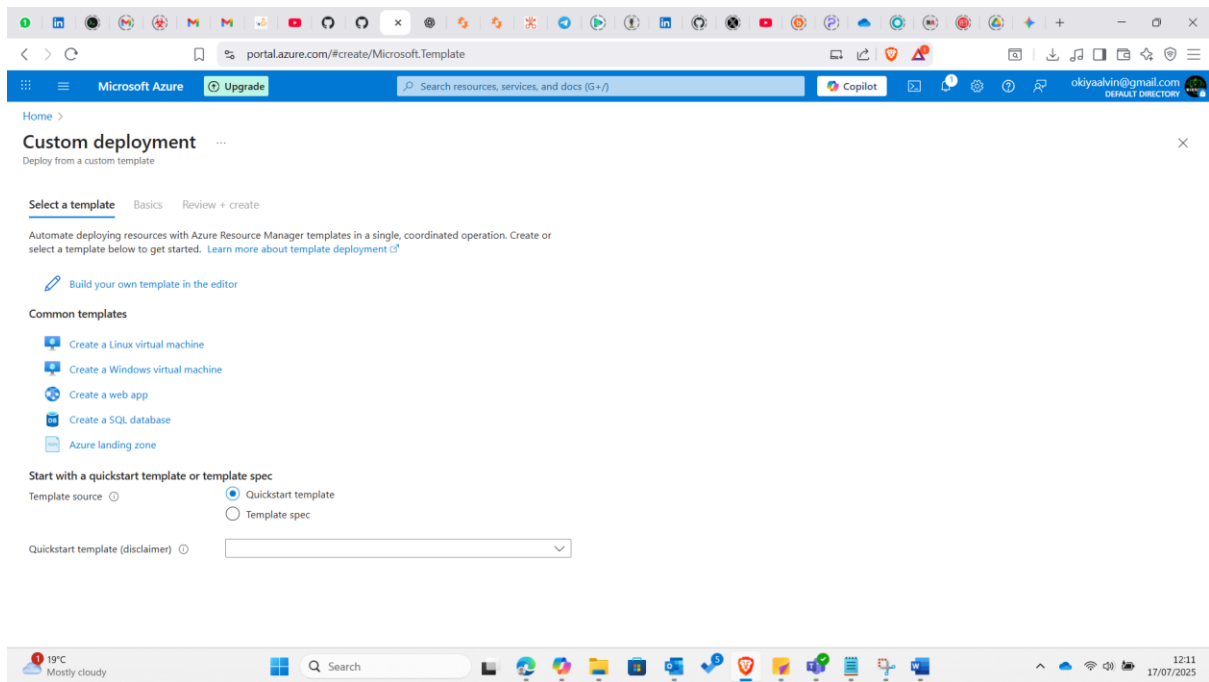
Exercise 1: Deploy the Base Infrastructure from an ARM Template

Task 1: Deploy an Azure VM and an Azure SQL Database using the provided ARM template.

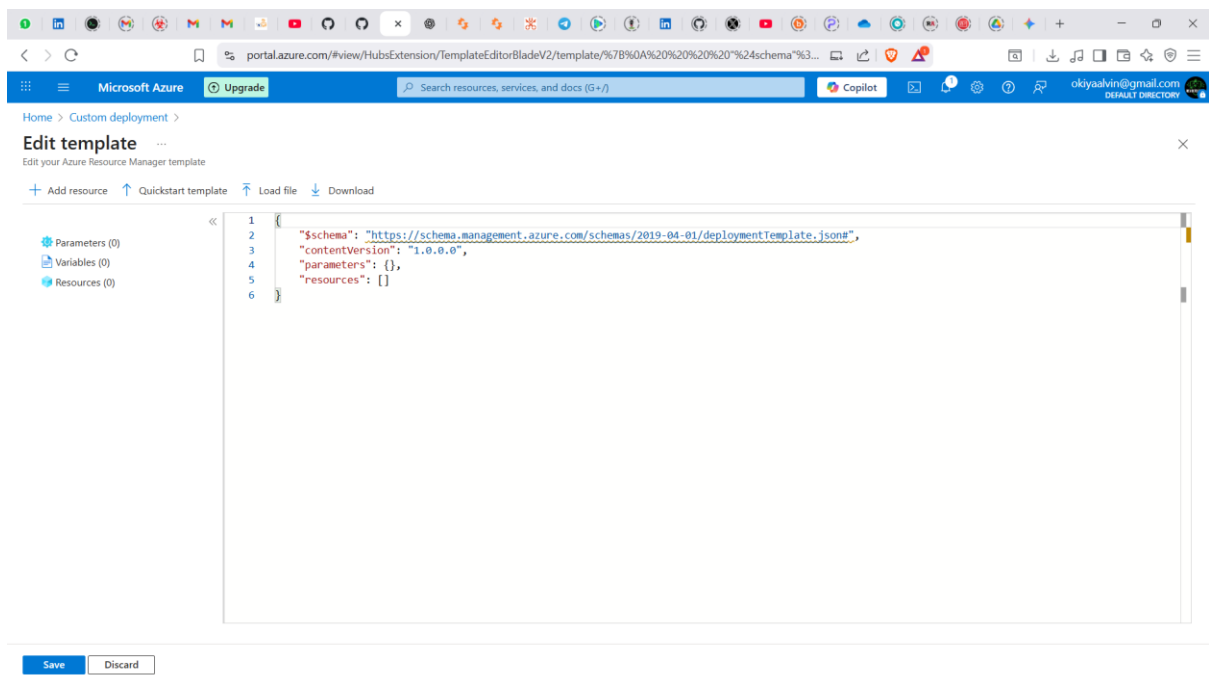
To accomplish this task I started by logging into my azure portal and the **Search resources, services, and docs** text box at the top of the Azure portal page, I typed **Deploy a custom template** then selected the first option:



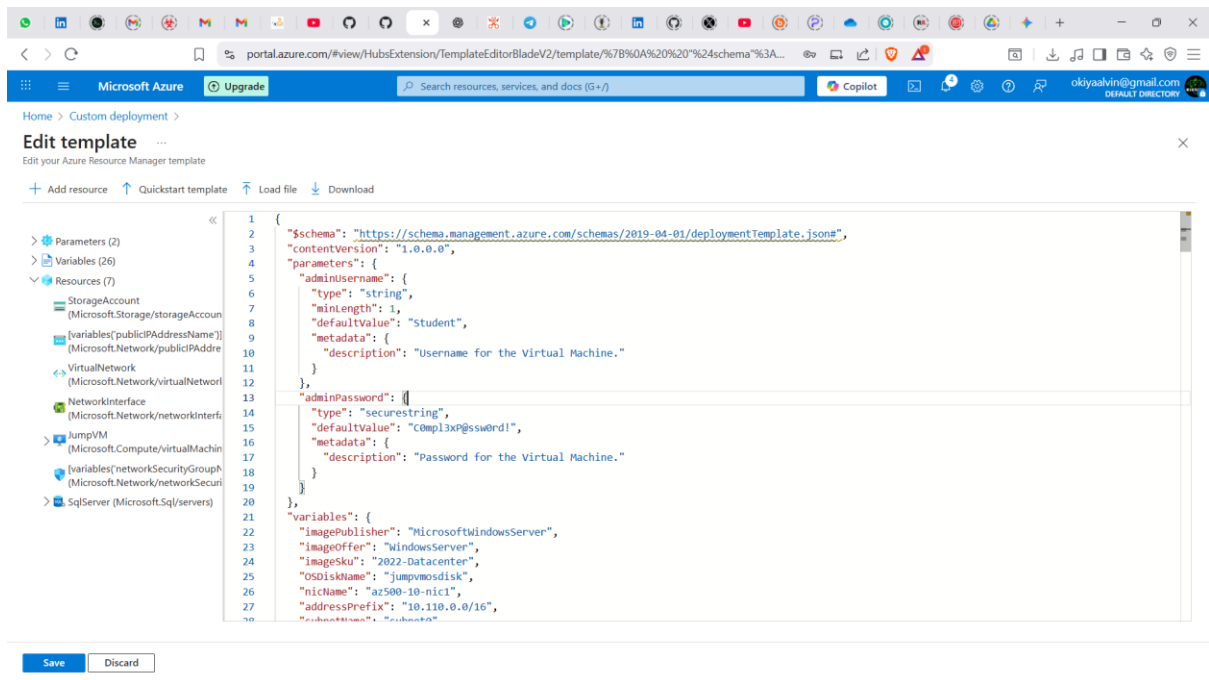
Then I clicked on the “Build your own template in the editor” section:



It took me to this page where I was able to click load file and loaded the `\Allfiles\Labs\10\az-500-10_azuredeploy.json` file.



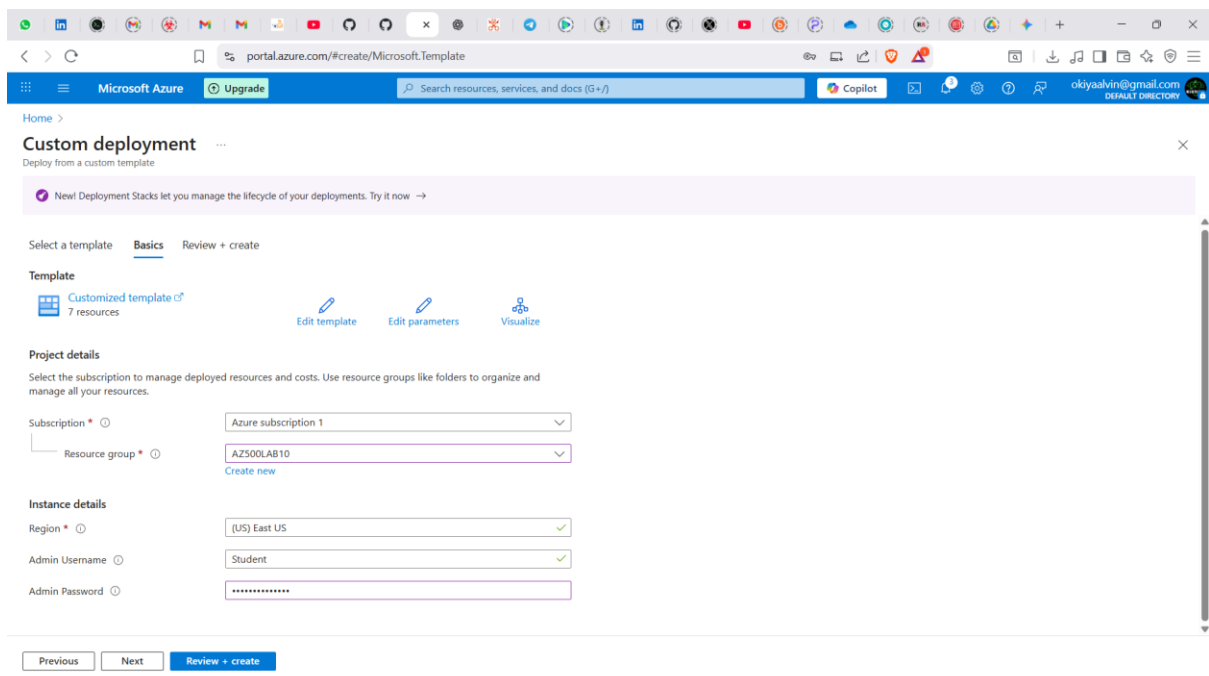
I uploaded it and clicked on save:



I took note of the username and password as shown on the screenshot:

Username: Student

Password: C0mpl3xP@ssw0rd!



I created a new resource group and password as shown in the above screenshot, then clicked "Review + create" then "create"

Microsoft.Template-20250717125528 | Overview

Deployment is in progress

Deployment name: Microsoft.Template-20250717125528
Subscription: Azure subscription 1
Resource group: AZ500LAB10

Start time: 7/17/2025, 12:48:01 PM
Correlation ID: f7b12286-35cc-44db-b6e3-6823bda365d2

Resource	Type	Status	Operation details
az500-10-vm1/VMConfig	Microsoft.Compute/virtualMach	Created	Operation details
sqlservermas6nsrnkoko/medical	Microsoft.Sql/servers/databases	Created	Operation details
sqlservermas6nsrnkoko/AllowAllMicrosoft	Microsoft.Sql/servers/firewallrul	OK	Operation details
az500-10-vm1	Virtual machine	OK	Operation details
vhdstoragefw62pyzxa7ug	Storage account	OK	Operation details
az500-10-nic1	Microsoft.Network/networkinter	OK	Operation details
az500-10-vnet1	Virtual network	OK	Operation details
az500-10-nsg1	Network security group	OK	Operation details
az500-10-pip1	Public IP address	OK	Operation details
vhdstoragefw62pyzxa7ug	Storage account	OK	Operation details
sqlservermas6nsrnkoko	Microsoft.Sql/servers	Created	Operation details

Since it was taking time to deploy, I proceeded to the next Exercise.

Exercise 2: Configure the Key Vault Resource with a Key and a Secret

Task 1: Create and configure an Azure Key Vault.

In this task, I was to create an Azure Key Vault resource and also configure the Azure Key Vault permissions. I started by opening the Cloud Shell > **PowerShell** > **Create storage**.

Microsoft.Template-20250717125528 | Overview

Deployment is in progress

Deployment name: Microsoft.Template-20250717125528
Subscription: Azure subscription 1
Resource group: AZ500LAB10

Start time: 7/17/2025, 12:44:01 PM
Correlation ID: f7b12286-35cc-44db-b6e3-6823bda365d2

Resource	Type	Status	Operation details
az500-10-vm1/VMConfig	Microsoft.Compute/virtualMach	Created	Operation details
sqlservermas6nsrnkoko/medical	Microsoft.Sql/servers/databases	Created	Operation details

Welcome to Azure Cloud Shell

Type "az" to use Azure CLI
Type "help" to learn about Cloud Shell

Your Cloud Shell session will be ephemeral so no files or system changes will persist beyond your current session.

MDT: Azure Cloud Shell now includes Predictive IntelliSense! Learn more: <https://aka.ms/CloudShell/IntelliSense>

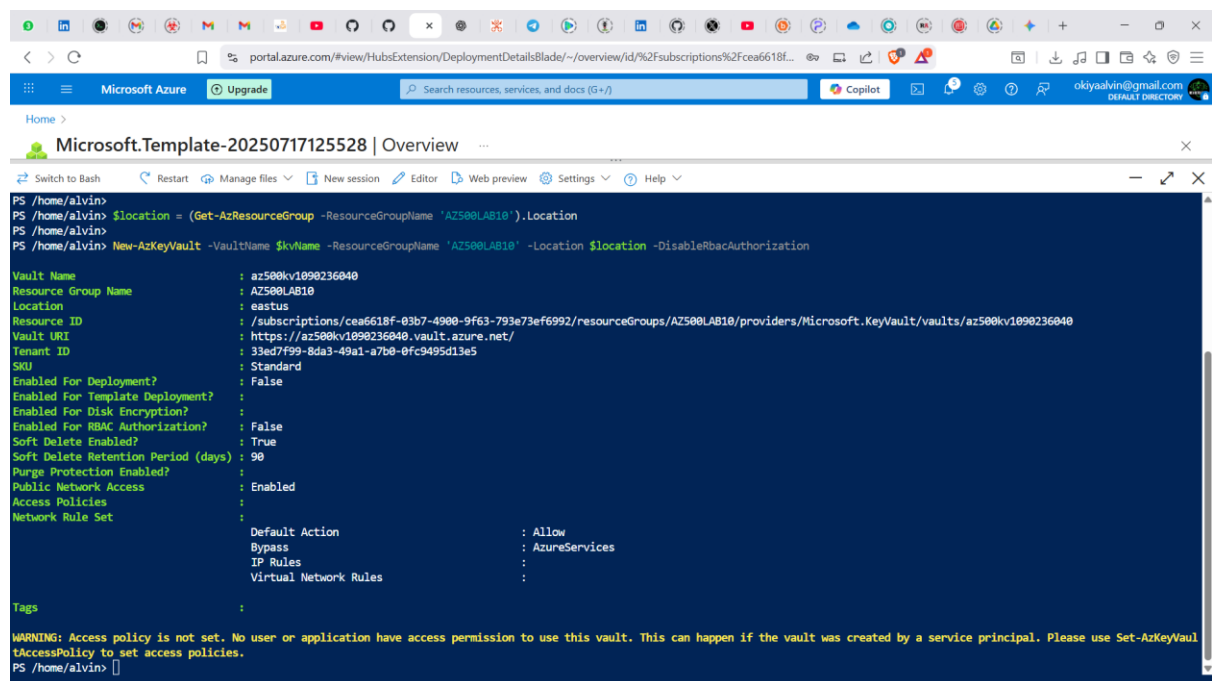
VERBOSE: Authenticating to Azure ...
VERBOSE: Building your Azure drive ...
PS /home/alvin>

On the powershell, I ran the following command to create an Azure Key Vault in the resource group **AZ500LAB10**. (If you chose another name for this lab's Resource Group out of Task 1, use that name for this task as well). The Key Vault name must be unique. Remember the name you have chosen. You will need it throughout this lab:

```
$kvName = 'az500kv' + $(Get-Random)
```

```
$location = (Get-AzResourceGroup -ResourceGroupName 'AZ500LAB10').Location
```

```
New-AzKeyVault -VaultName $kvName -ResourceGroupName 'AZ500LAB10' -Location  
$location -DisableRbacAuthorization
```



```
PS /home/alvin> $location = (Get-AzResourceGroup -ResourceGroupName 'AZ500LAB10').Location
PS /home/alvin> New-AzKeyVault -VaultName $kvName -ResourceGroupName 'AZ500LAB10' -Location $location -DisableRbacAuthorization

Vault Name                               : az500kv1090236040
Resource Group Name                     : AZ500LAB10
Location                               : eastus
Resource ID                             : /subscriptions/cea6618f-89b7-4980-9f63-793e73ef6992/resourceGroups/AZ500LAB10/providers/Microsoft.KeyVault/vaults/az500kv1090236040
Vault URI                               : https://az500kv1090236040.vault.azure.net/
Tenant ID                               : 33ed7f99-8da3-49a1-a7b0-0fc9495d13e5
SKU                                     : Standard
Enabled For Deployment?                 : False
Enabled For Template Deployment?        :
Enabled For Disk Encryption?            :
Enabled For RBAC Authorization?         : False
Soft Delete Enabled?                   : True
Soft Delete Retention Period (days)    : 90
Purge Protection Enabled?               :
Public Network Access                   : Enabled
Access Policies                         :
Network Rule Set                        :
    Default Action                       : Allow
    Bypass                               : AzureServices
    IP Rules                             :
    Virtual Network Rules                :

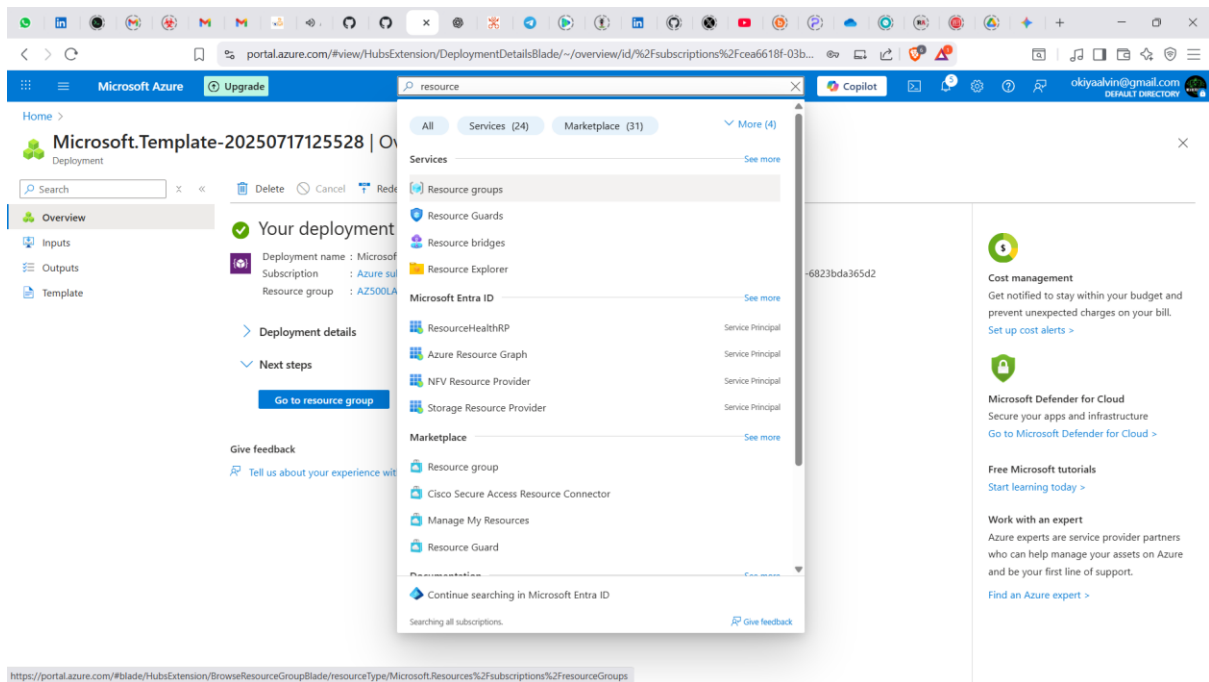
Tags                                    :

WARNING: Access policy is not set. No user or application have access permission to use this vault. This can happen if the vault was created by a service principal. Please use Set-AzKeyVaultAccessPolicy to set access policies.
PS /home/alvin>
```

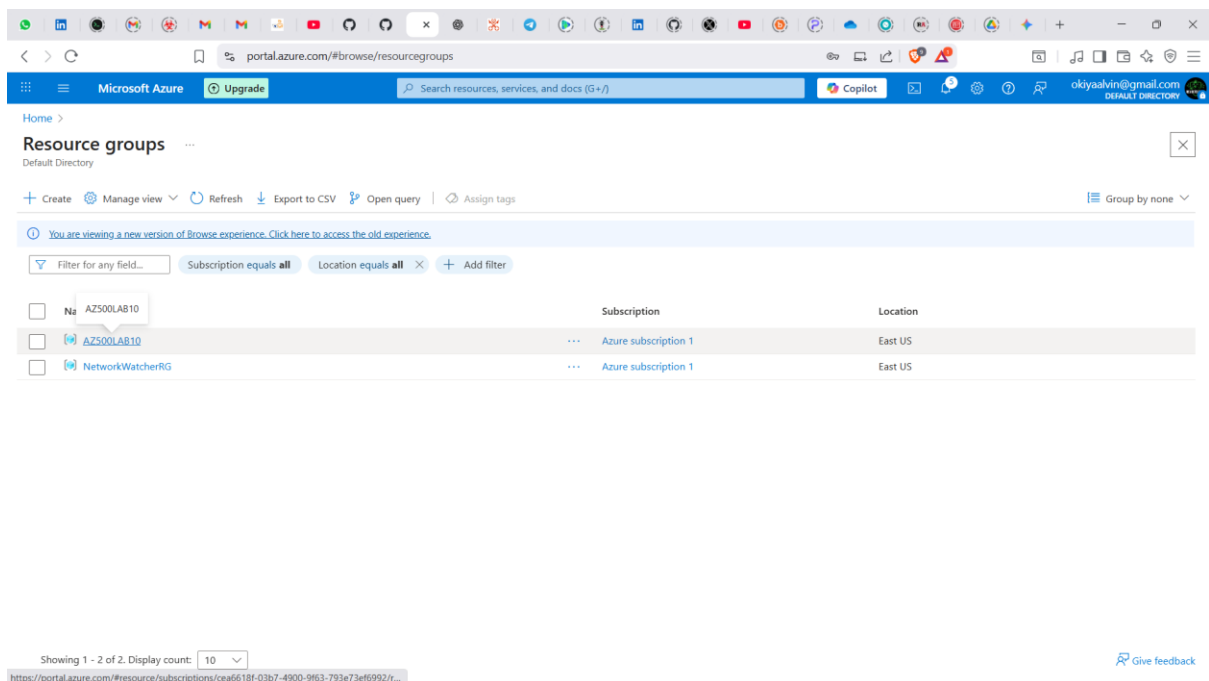
I then noted the Vault name and Vault URI.

Next step was to confirm the deployment of the Key vault:

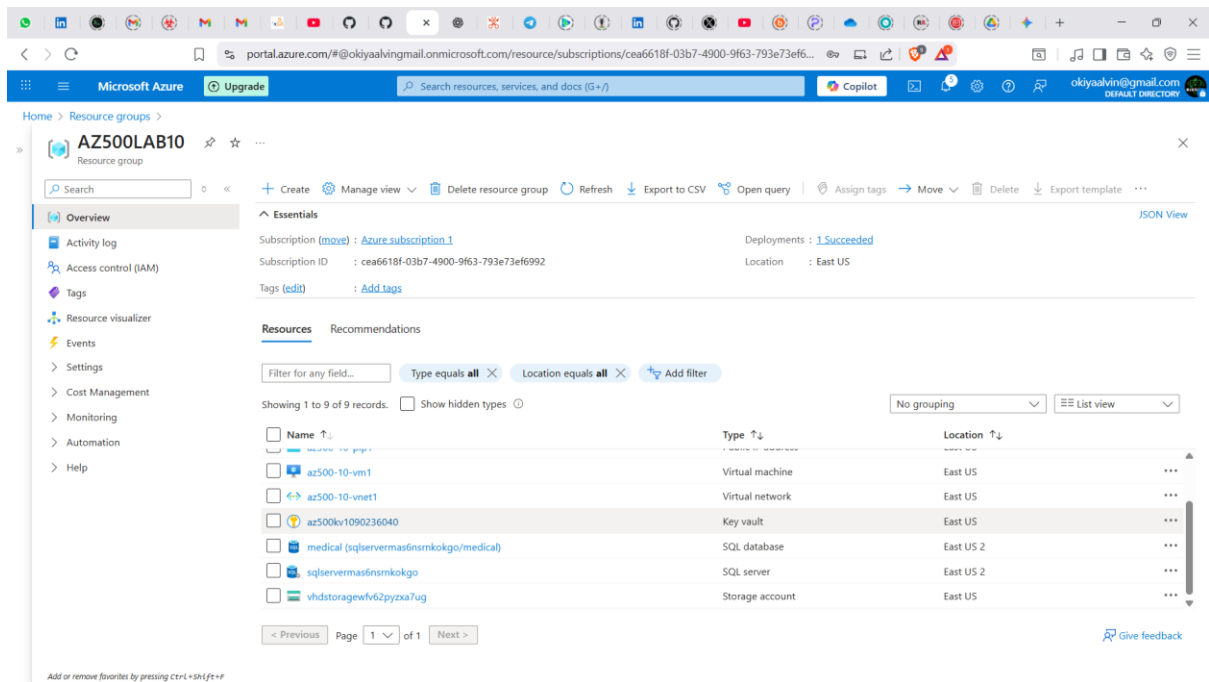
I went to the resourcegroup section of the azure portal:



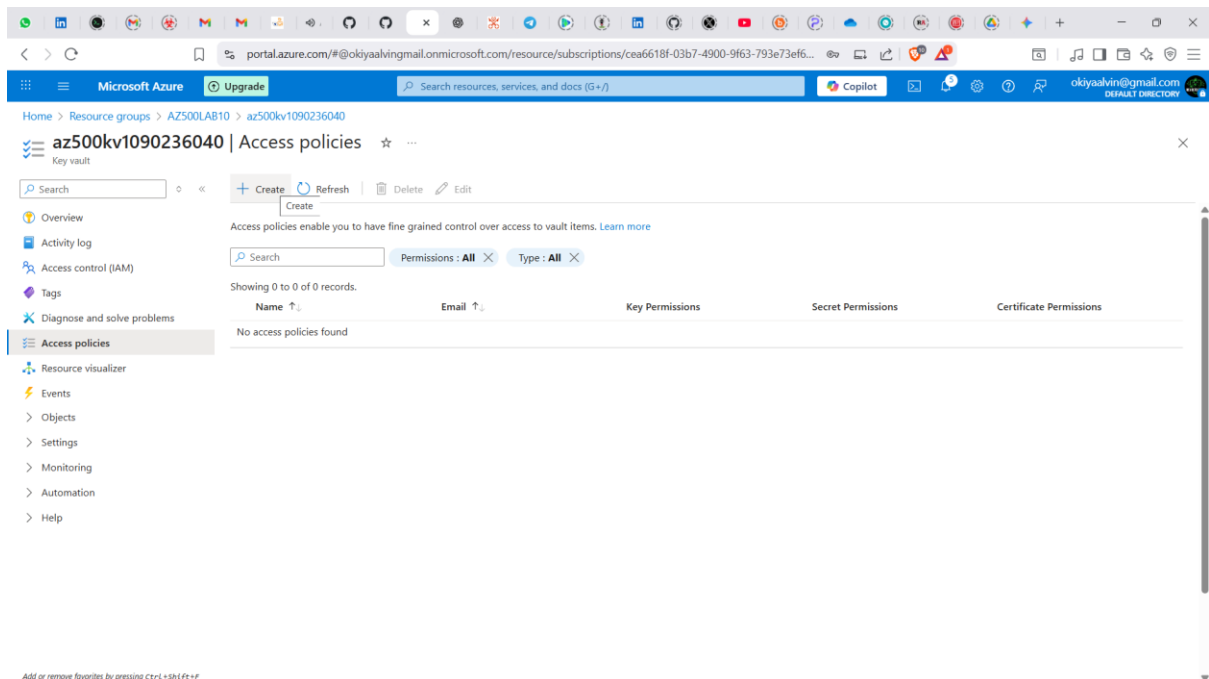
In the list of resource group, I clicked the **AZ500LAB10**



Next I located the keyvault, on the screenshot it can be seen as the one with a key icon:



I clicked on it, and in the **Overview** section, I clicked **Access Policies > + Create**

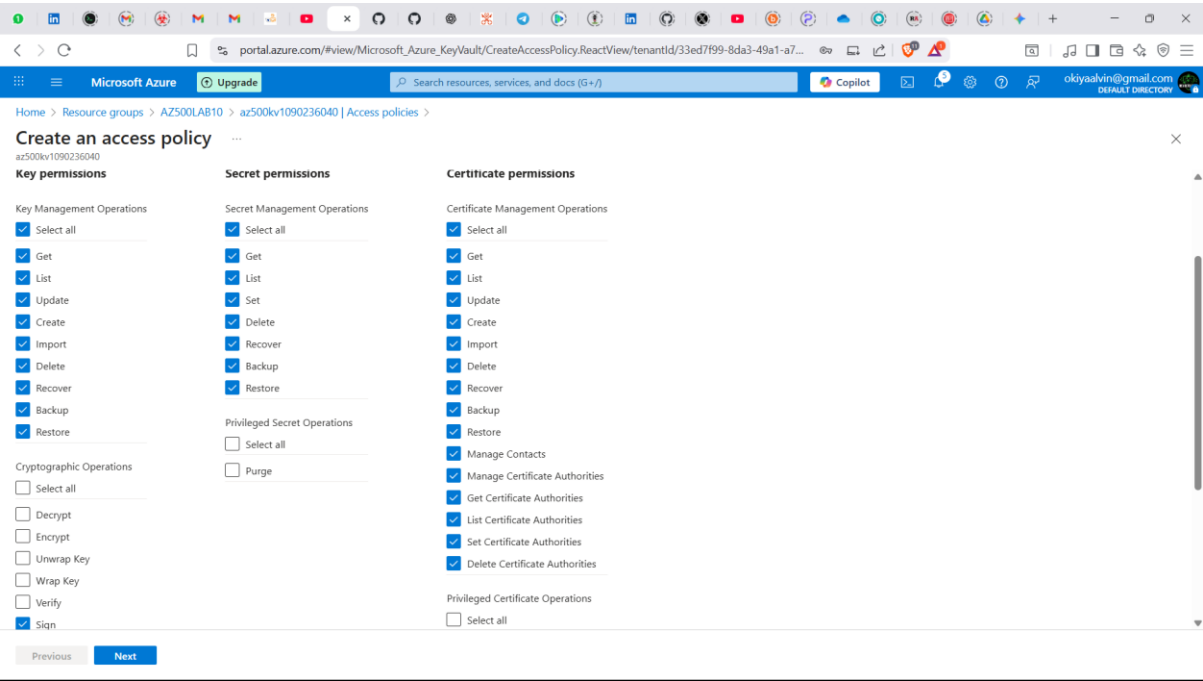


I then specified the access policies according the instructions below:

Setting	Value
Configure from template (optional)	Key, Secret, & Certificate Management
Key permissions	click Select all resulting in total of 9 selected permissions
Key permissions/Cryptographic Operations	click Sign resulting in total of 1 selected permissions
Secret permissions	click Select all resulting in total of 7 selected permissions
Certification permissions	click Select all resulting in total of 15 selected permissions
Select principal	click None selected , on the Principal blade, select your user account, and click Next
Application (optional)	click Next
Review + create	click Create

Note: The previous Review + create operation returns to the Access policies page that lists Application, Email, Key Permissions, Secret Permissions, and Certificate Permissions.

I did that as shown:



I then clicked on principal section to ensure the settings are in accordance with the instructions, where I was to next select my account and clicked Next:

portal.azure.com/#view/Microsoft_Azure_KeyVault/CreateAccessPolicy.ReactView/tenantId/33ed7f99-8da3-49a1-a7...

Microsoft Azure Upgrade Search resources, services, and docs (G+)

Capilot

okiyaalvin@gmail.com DEFAULT DIRECTORY

Home > Resource groups > AZ500LAB10 > az500kv1090236040 | Access policies >

Create an access policy ...

az500kv1090236040

Permissions Principal Application (optional) Review + create

Only 1 principal can be assigned per access policy.
Use the new embedded experience to select a principal. The previous popup experience can be accessed here. [Select a principal](#)

Search by object ID, name, or email address

aciapi
c5b17a4f-cc6f-4649-9480-684280a2af3a

Alvin Okiya
okiyaalvin@gmail.com

Azure Graph
d0cbdd02a-d7c4-42fb-8c27-b07e5118b848

Selected item

Alvin Okiya
okiyaalvin@gmail.com

Previous

Next

And Next again to review and create, then click create:

portal.azure.com/#view/Microsoft_Azure_KeyVault/CreateAccessPolicy.ReactView/tenantId/33ed7f99-8da3-49a1-a7...

Microsoft Azure Upgrade Search resources, services, and docs (G+)

Capilot

okiyaalvin@gmail.com DEFAULT DIRECTORY

Home > Resource groups > AZ500LAB10 > az500kv1090236040 | Access policies >

Create an access policy ...

az500kv1090236040

Permissions Principal Application (optional) Review + create

Key Permissions

Key Management Operations	All selected
Cryptographic Operations	Sign
Privileged Key Operations	None selected
Rotation Policy Operations	All selected

Secret Permissions

Secret Management Operations	All selected
Privileged Secret Operations	None selected

Certificate Permissions

Certificate Management Operations	All selected
Privileged Certificate Operations	None selected

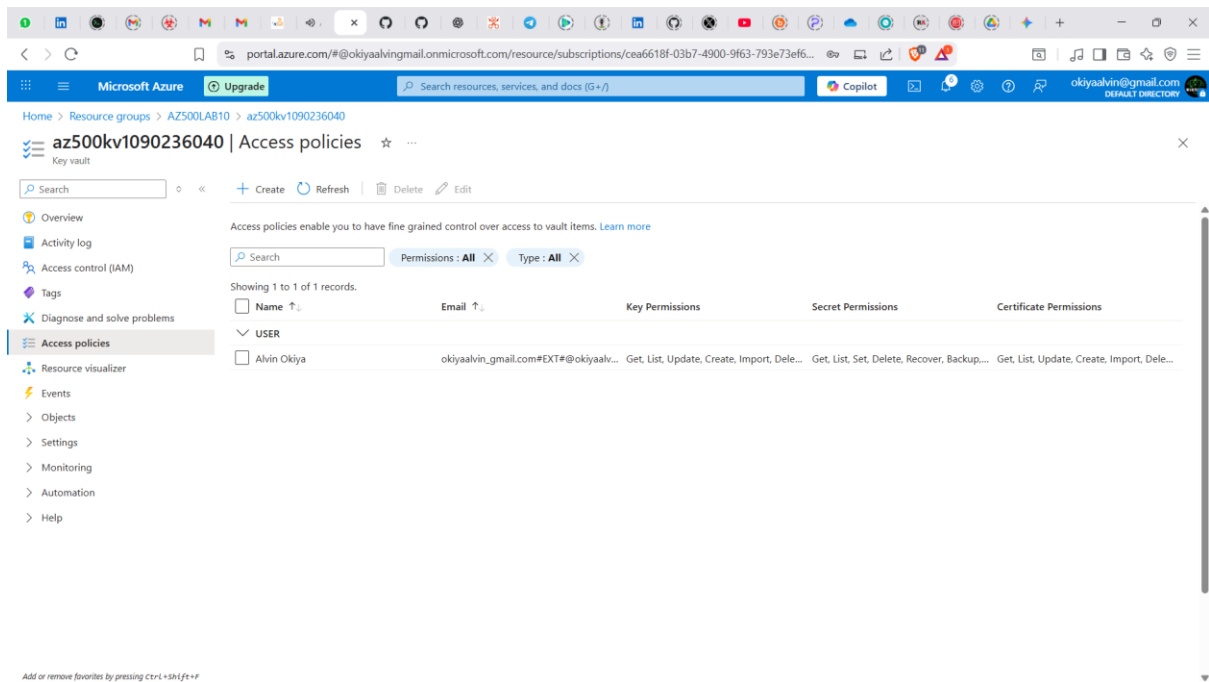
Principal

Principal name	Alvin Okiya
Object ID	6d4b2504-f970-4d04-a802-fe7463498cd3

Application

Previous

Create

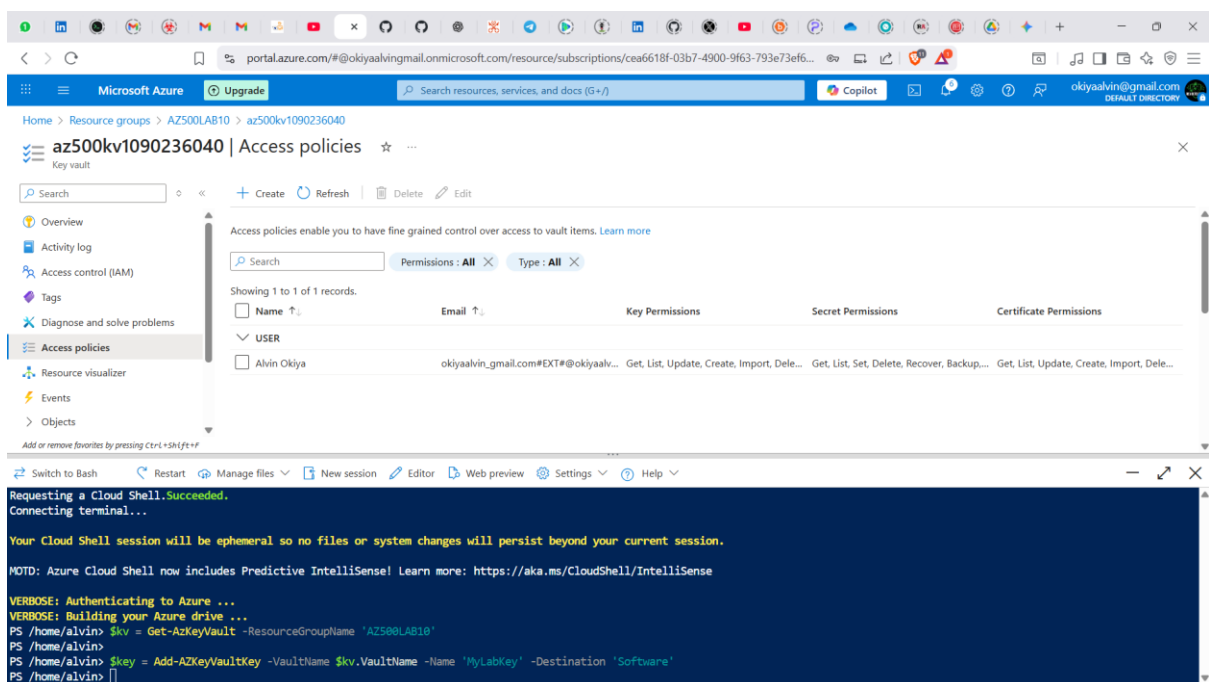


Task 2: Add a key to the Key Vault.

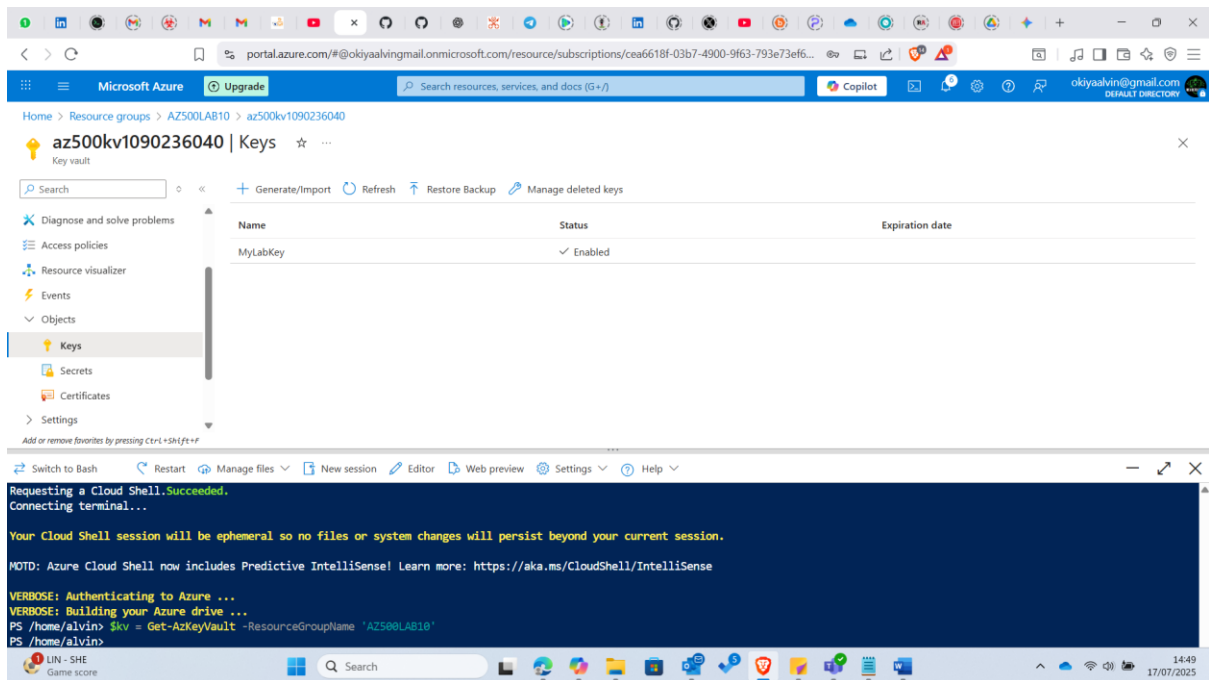
In this task, I was to add a key to the Key Vault and view information about the key. In the Azure portal, I opened a PowerShell session and used the following command to add a software-protected key to the Key Vault:

```
$kv = Get-AzKeyVault -ResourceGroupName 'AZ500LAB10'
```

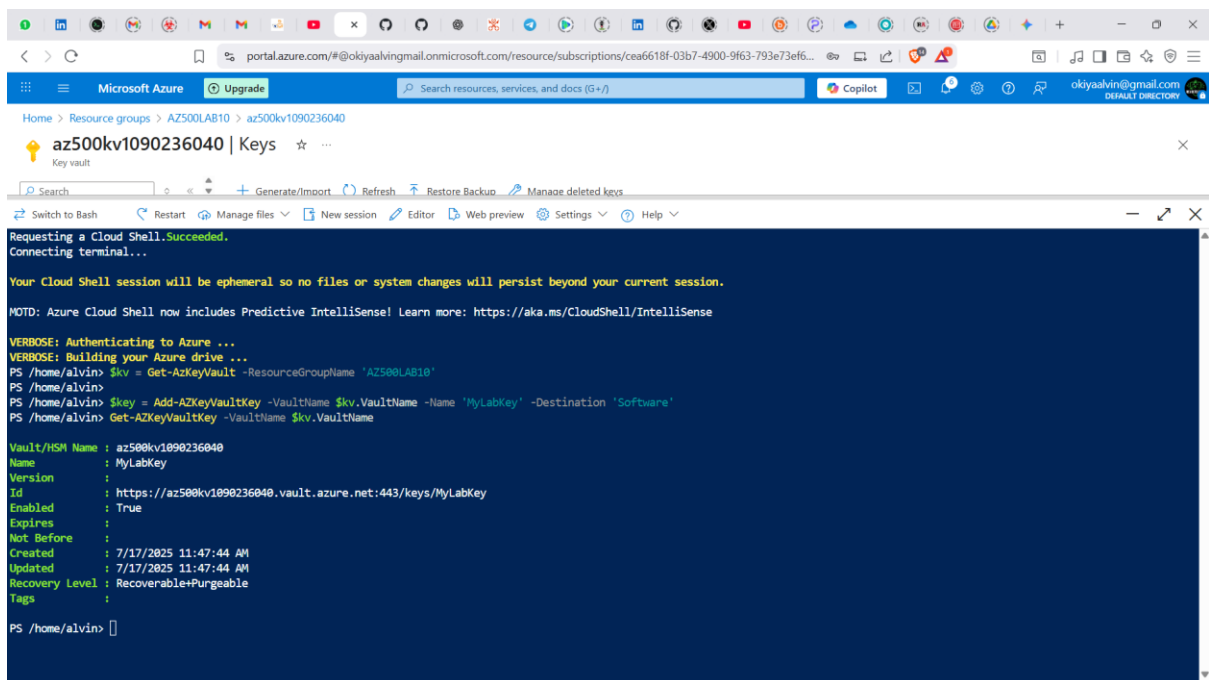
```
$key = Add-AZKeyVaultKey -VaultName $kv.VaultName -Name 'MyLabKey' -Destination 'Software'
```



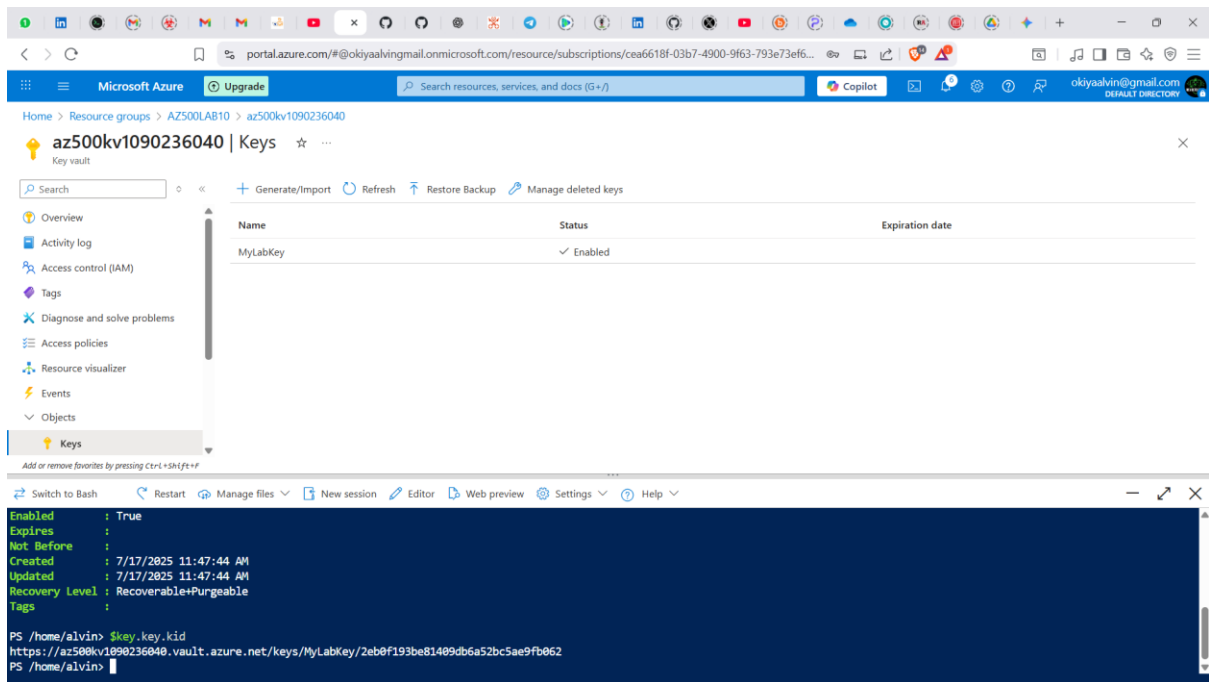
At the blade in the objects section when I clicked on Keys I was able to see the created key “My Lab Key”:



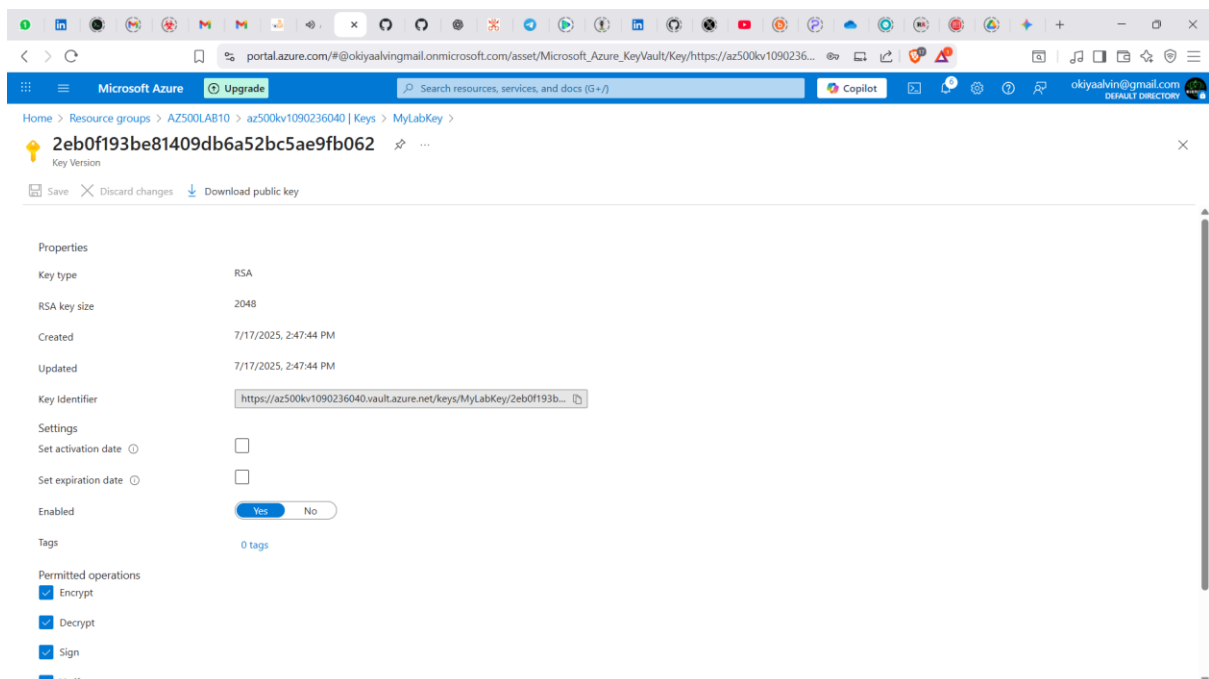
Verification using powershell command : “Get-AZKeyVaultKey -VaultName \$kv.VaultName “ also confirmed successful creation of the key:



I then used the command “ \$key.key.kid” on the powershell to know the key identifier:



I then went to the portal section on MyLabkeys under keys section of the vault



Analysing the Key its an RSA, with 2048 Bytes having date for creation.

Task 3: Add a secret to the Key Vault.

I switched back to the Cloud Shell pane.

In the PowerShell session within the Cloud Shell pane, I ran the following to create a variable with a secure string value:

```
$secretvalue = ConvertTo-SecureString 'Pa55w.rd1234' -AsPlainText -Force
```

I then ran the following to add the secret to the vault:

```
$secret = Set-AZKeyVaultSecret -VaultName $kv.VaultName -Name 'SQLPassword' -  
SecretValue $secretvalue
```

I then ran the following to verify the secret was created

```
Get-AZKeyVaultSecret -VaultName $kv.VaultName
```

The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the Microsoft Azure logo, an 'Upgrade' button, a search bar, and a user profile. The main content area displays the 'Keys' section for the Key Vault 'az500kv1090236040'. A table lists the keys, with one key named 'MyLabKey' in an 'Enabled' status. Below the table, a terminal window is open, showing the following PowerShell commands and their output:

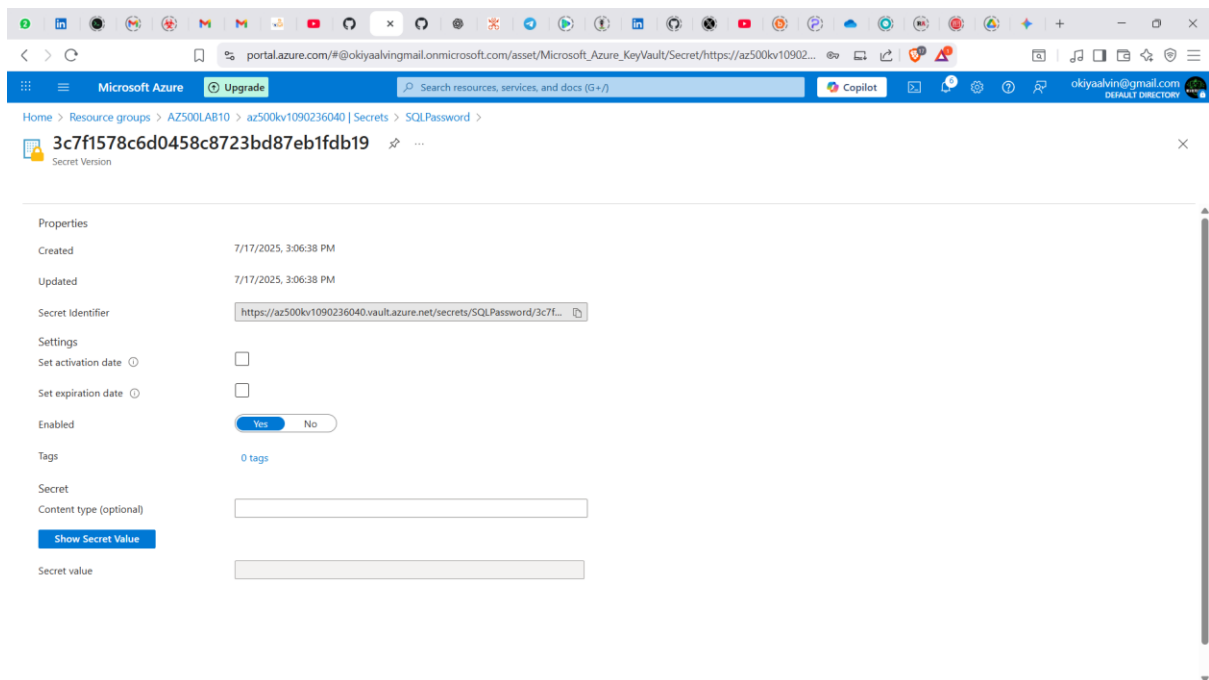
```
PS /home/alvin> $key.key.kid  
https://az500kv1090236040.vault.azure.net/keys/MyLabKey/2eb0f193be81409db6a52bc5ae9fb62  
PS /home/alvin> $secretvalue = ConvertTo-SecureString 'Pa55w.rd1234' -AsPlainText -Force  
PS /home/alvin> $secret = Set-AZKeyVaultSecret -VaultName $kv.VaultName -Name 'SQLPassword' -SecretValue $secretvalue  
PS /home/alvin> Get-AZKeyVaultSecret -VaultName $kv.VaultName  
  
Vault Name : az500kv1090236040  
Name       : SQLPassword  
Version    :  
Id         : https://az500kv1090236040.vault.azure.net:443/secrets/SQLPassword  
Enabled    : True  
Expires    :  
Not Before :  
Created    : 7/17/2025 12:06:38 PM  
Updated    : 7/17/2025 12:06:38 PM  
Content Type :  
Tags       :  
PS /home/alvin>
```

I then navigated back to the Key Vault blade, in **Objects** section, clicked on:

Secrets > SQLPassword

The screenshot shows the Microsoft Azure portal interface, specifically the 'Secrets' section for the Key Vault 'az500kv1090236040'. The left sidebar contains a navigation menu with options like Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Access policies, Resource visualizer, Events, Objects, Keys, Secrets, Certificates, Settings, Monitoring, Automation, and Help. The main content area displays a table of secrets, with one secret named 'SQLPassword' in an 'Enabled' status. The table has columns for Name, Type, Status, and Expiration date.

I then examined the information of the secret I created:

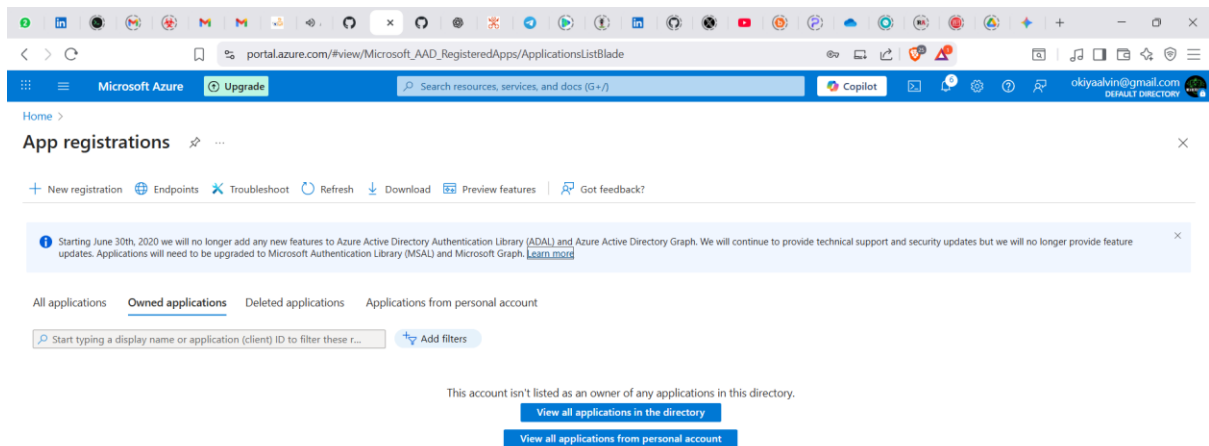


Exercise 3: Configure an Azure SQL Database and a Data-Driven Application

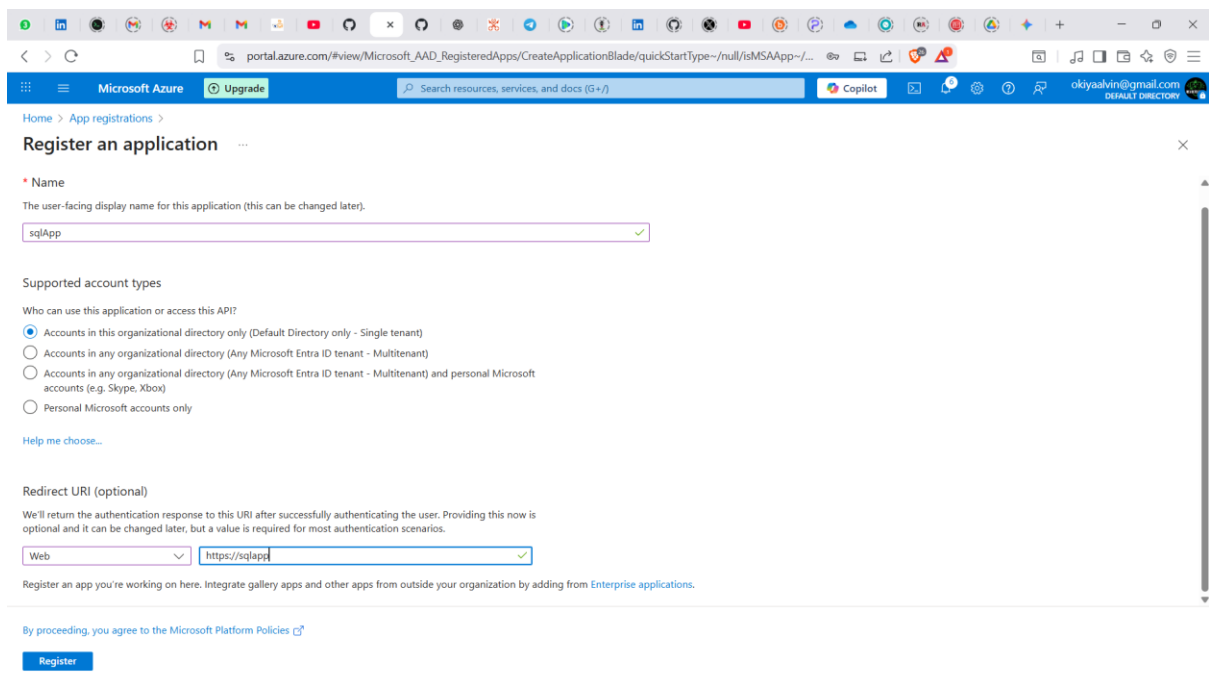
Task 1: Enable a client application to access the Azure SQL Database service.

In this task, I will enable a client application to access the Azure SQL Database service. This will be done by setting up the required authentication and acquiring the Application ID and Secret that I will need to authenticate my application.

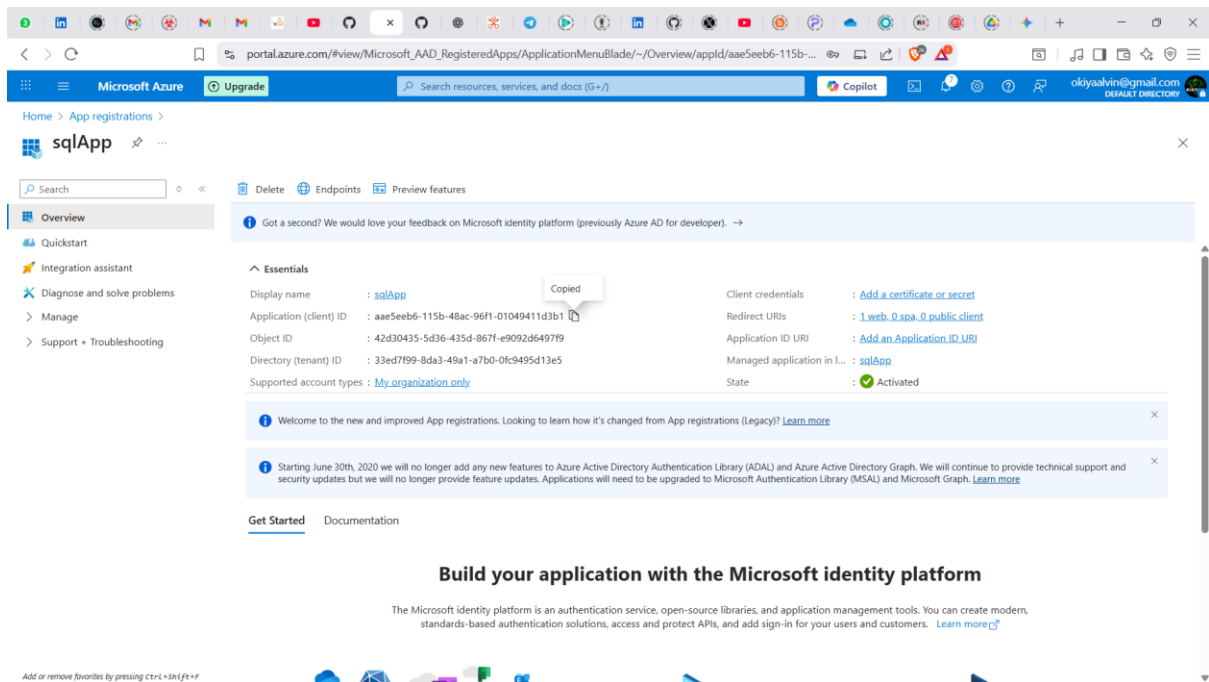
I started by using the search bar to search **App Registrations** and clicking on it, then clicking **+ New registration**



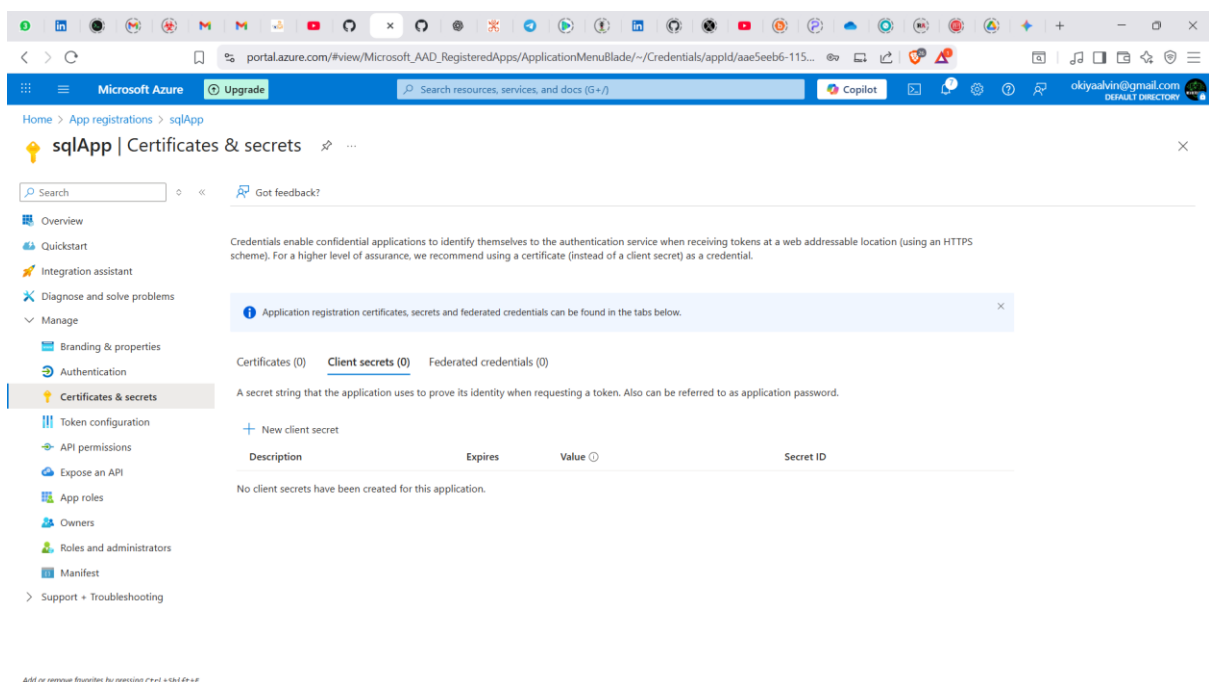
Then I applied the following settings then clicked on Register:



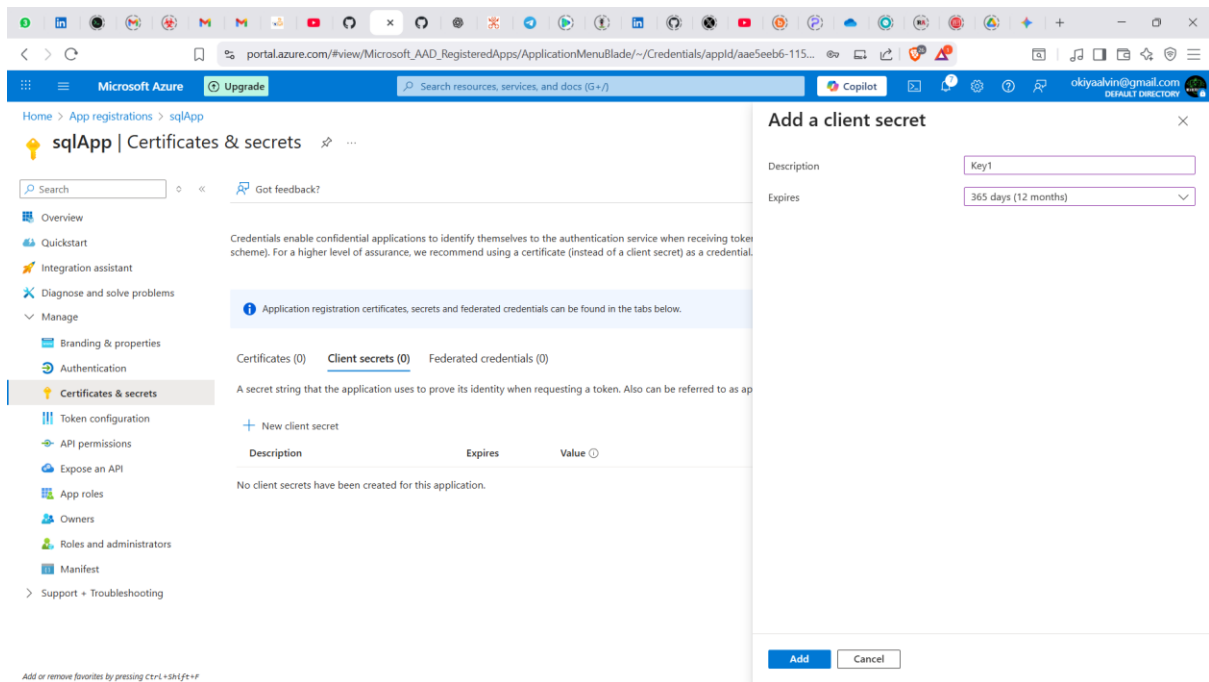
On the **sqlApp** blade, I identified the value of **Application (client) ID** :



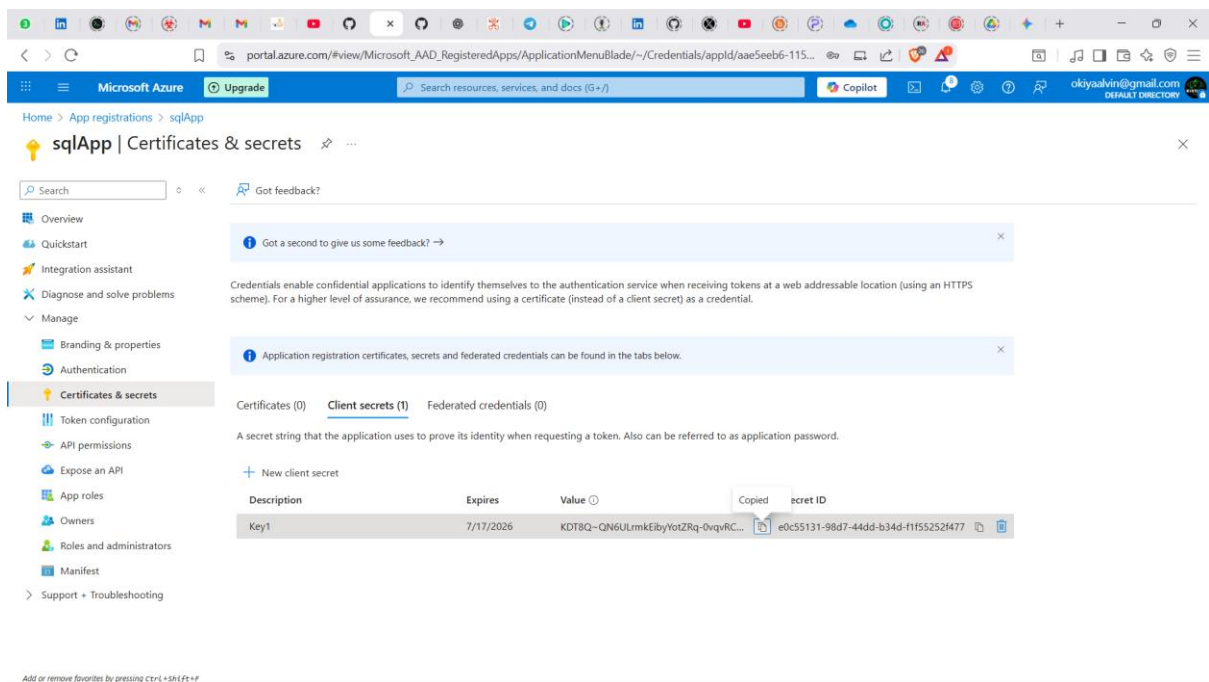
I went to the Management section on the SqlApp blade and On the **sqlApp | Certificates & secrets** blade / **Client Secrets** section, I clicked **+ New client secret**:



I specified the following settings and clicked on Add:



On the **sqlApp | Certificates & secrets** blade, I identified the value of **Key1** and recorded it so as to be used on the next steps:



Task 2: Create a policy allowing the application access to the Key Vault.

In this task, I will grant the newly registered app permissions to access secrets stored in the Key Vault.

-I started by opening the powershell and In the PowerShell session within the Cloud Shell pane, ran the following to create a variable storing the **Application (client) ID** recorded in the previous task (replace the <Azure_AD_Application_ID> placeholder with the value of the **Application (client) ID**):

```
$applicationId = '<Azure_AD_Application_ID>'
```

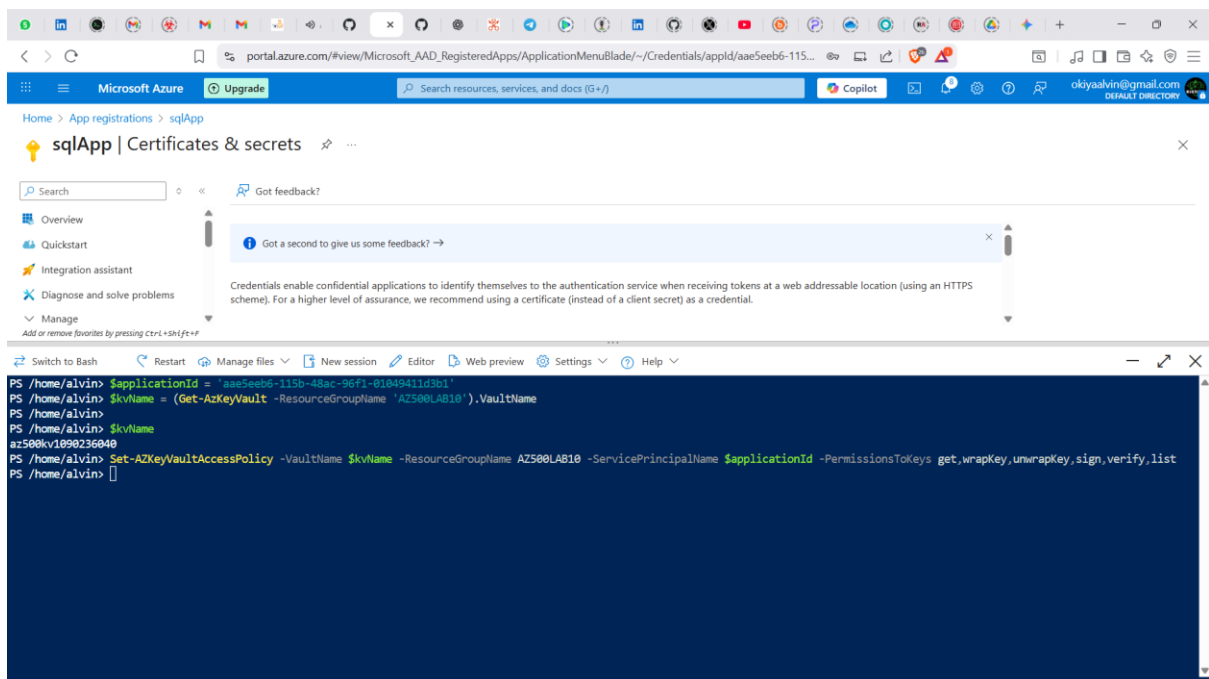
-In the PowerShell session within the Cloud Shell pane,I ran the following to create a variable storing the Key Vault name:

```
$kvName = (Get-AzKeyVault -ResourceGroupName 'AZ500LAB10').VaultName
```

```
$kvName
```

-I then ran the following to grant permissions on the Key Vault to the application registered in the previous task:

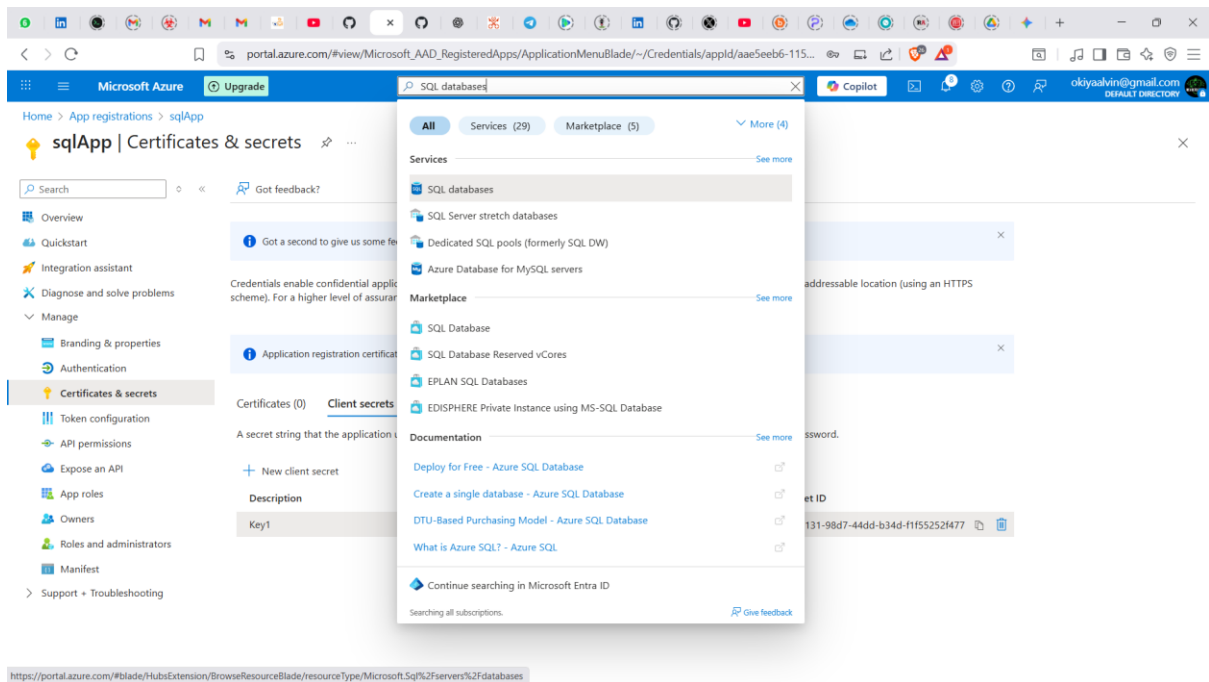
```
Set-AZKeyVaultAccessPolicy -VaultName $kvName -ResourceGroupName AZ500LAB10 -  
ServicePrincipalName $applicationId -PermissionsToKeys  
get,wrapKey,unwrapKey,sign,verify,list
```



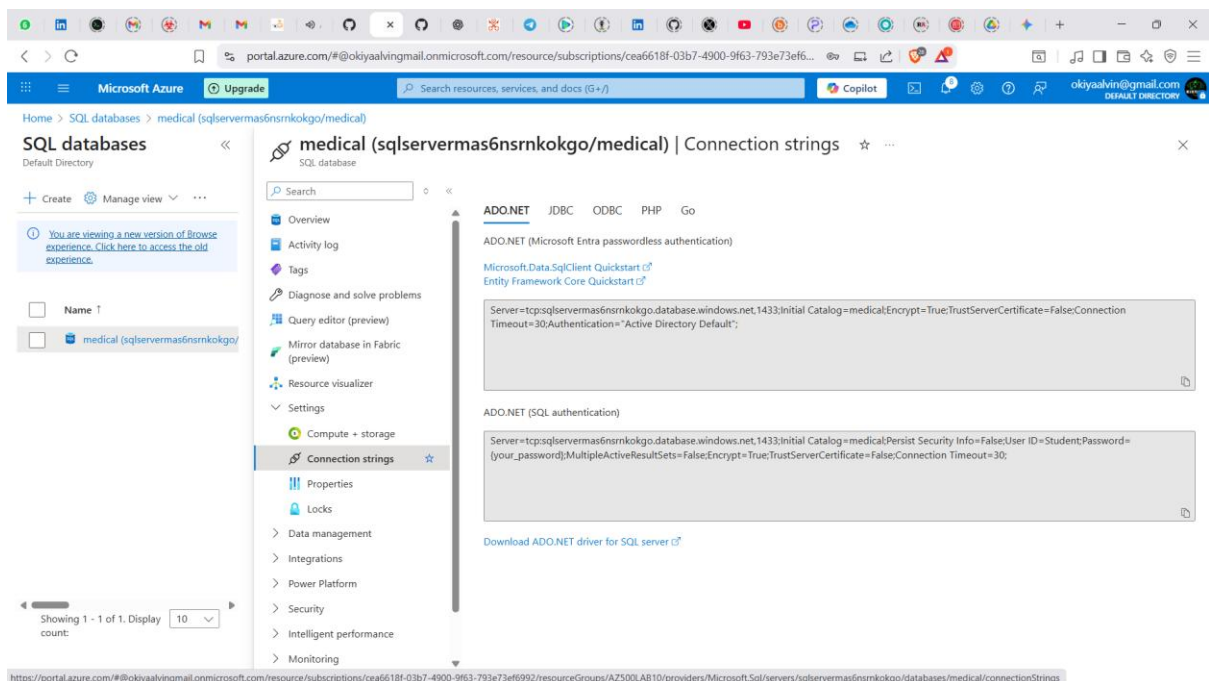
Task 3: Retrieve the Azure SQL Database ADO.NET Connection String.

The ARM-template deployment in Exercise 1 provisioned an Azure SQL Server instance and an Azure SQL database named **medical**. In this step we will update the empty database resource with a new table structure and select data columns for encryption

-In the search bar in the azure portal, I searched for **SQL databases** and selected it as shown:



Then clicked medical >Settings > connection strings>

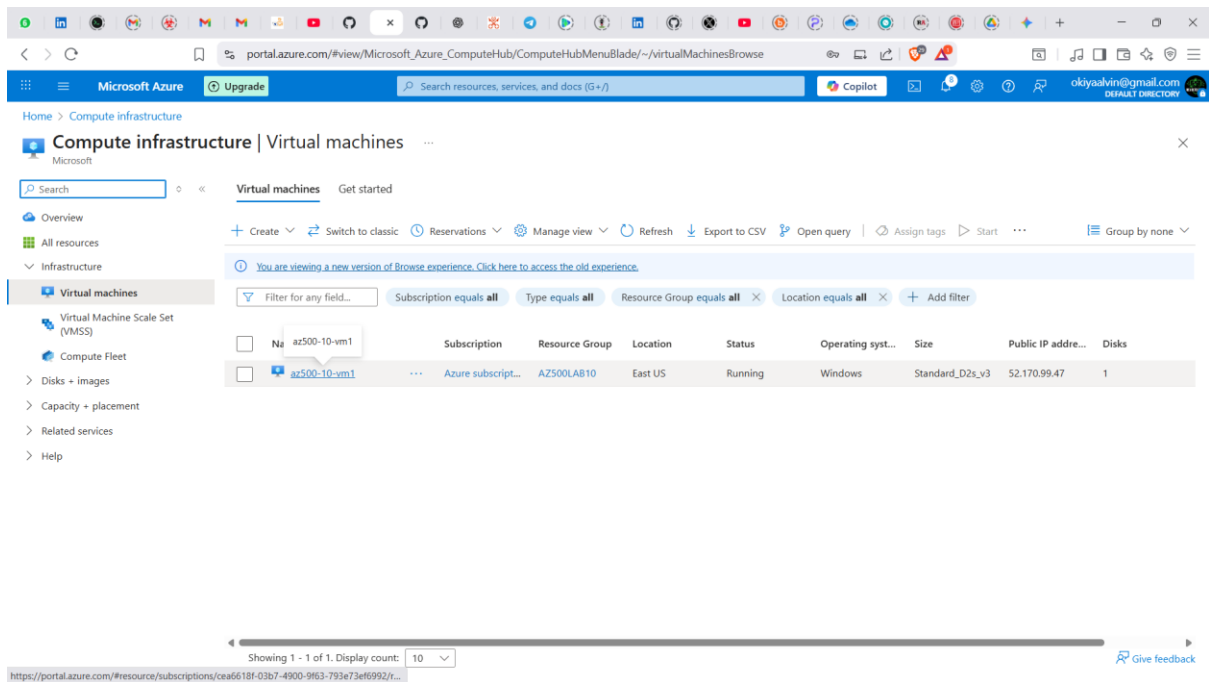


I then copied the ADO.NET (SQL authentication) string as it will be needed later.

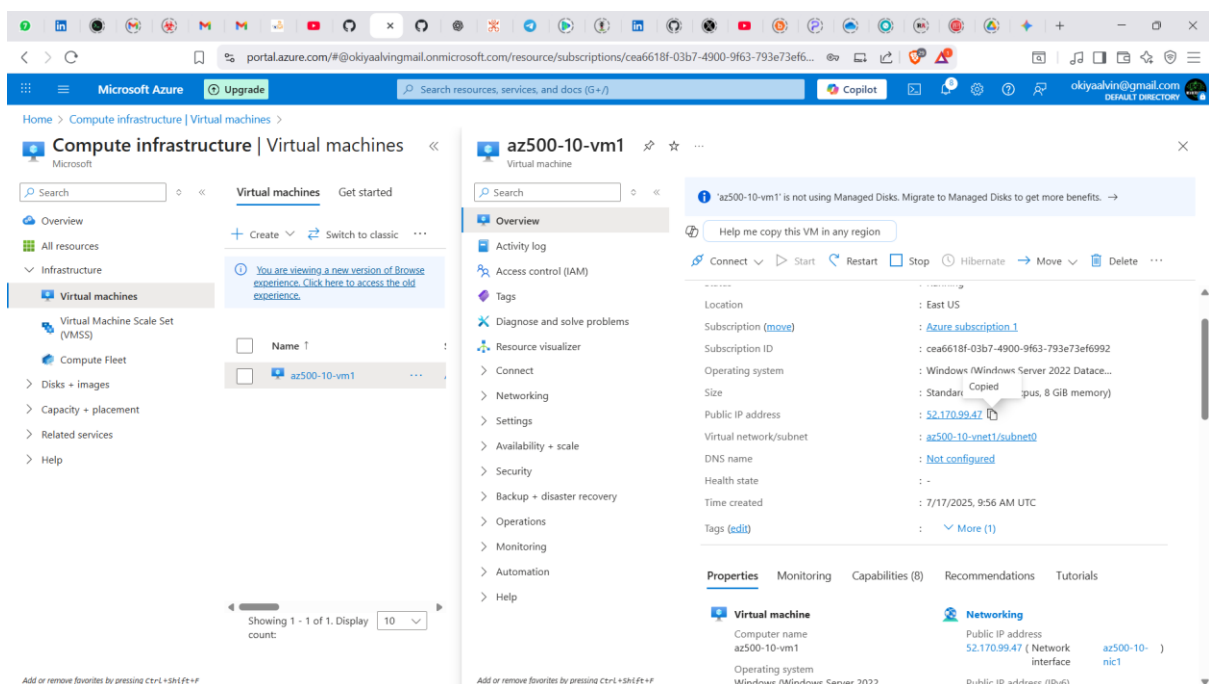
Task 4: Log on to the Azure VM running Visual Studio That was created on Exercise 1.

In this task, I was to log on to the Azure VM, which deployment I initiated in Exercise 1.

On the azure portal, at the search bar I searched for Virtual Machines then selected the VM that was created on Exercise 1 which was **az500-10-vm1**:



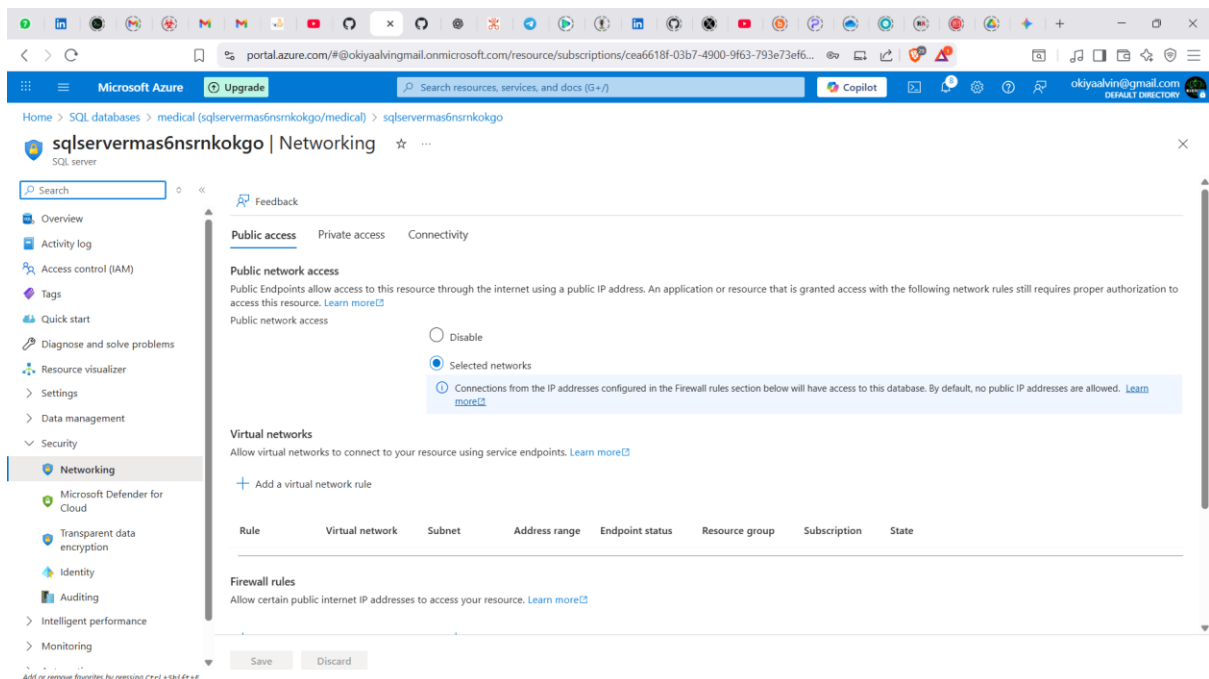
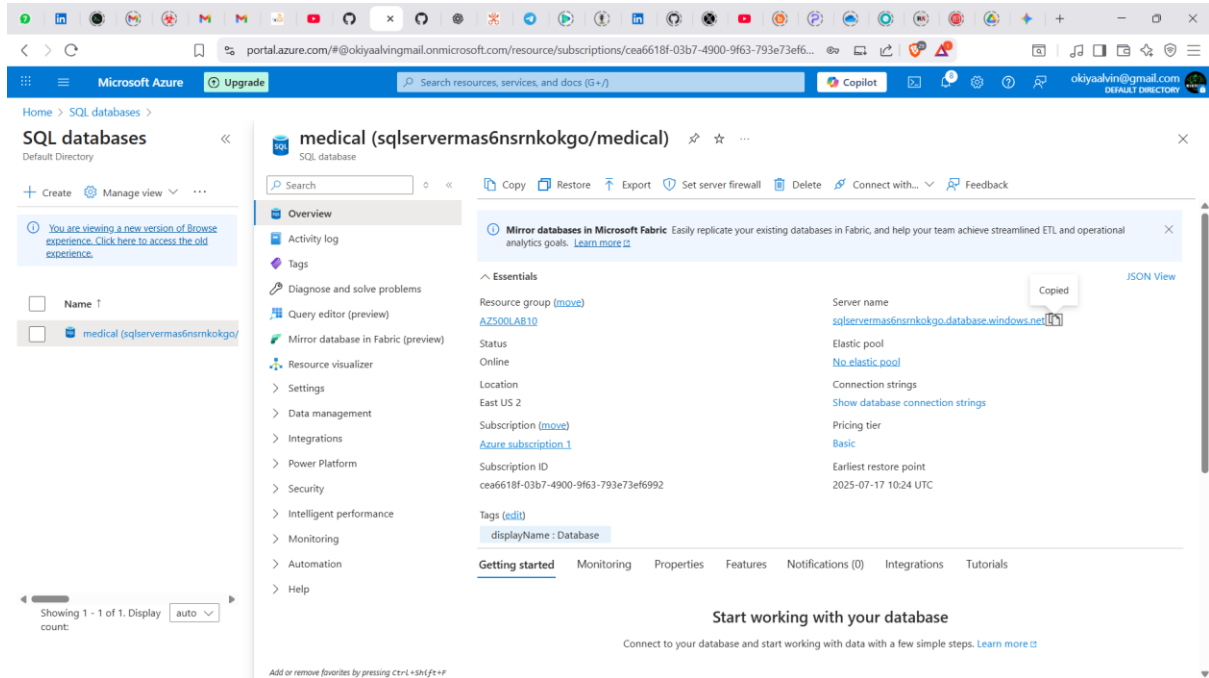
On the **az500-10-vm1** blade, on the **Essentials** pane, I took note of the **Public IP address** as I will use this later:



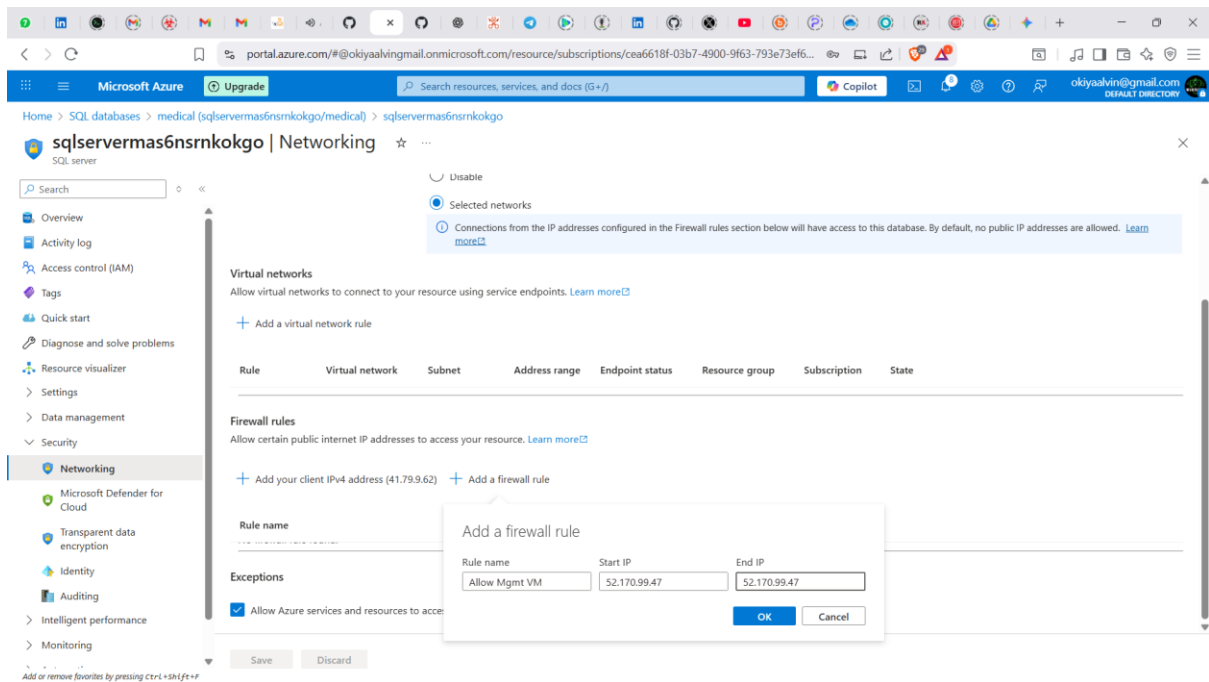
Task 5: Create a table in the SQL Database and select data columns for encryption

In this task, I was to connect to the SQL Database with SQL Server Management Studio and create a table. I then encrypted two data columns using an autogenerated key from the Azure Key Vault.

In the Azure portal, I navigated to the blade of the **medical** SQL database, in the **Essentials** section, identified the **Server name** (sqlservermas6nsrnkokgo.database.windows.net), and then, in the toolbar, clicked on **Set server firewall**



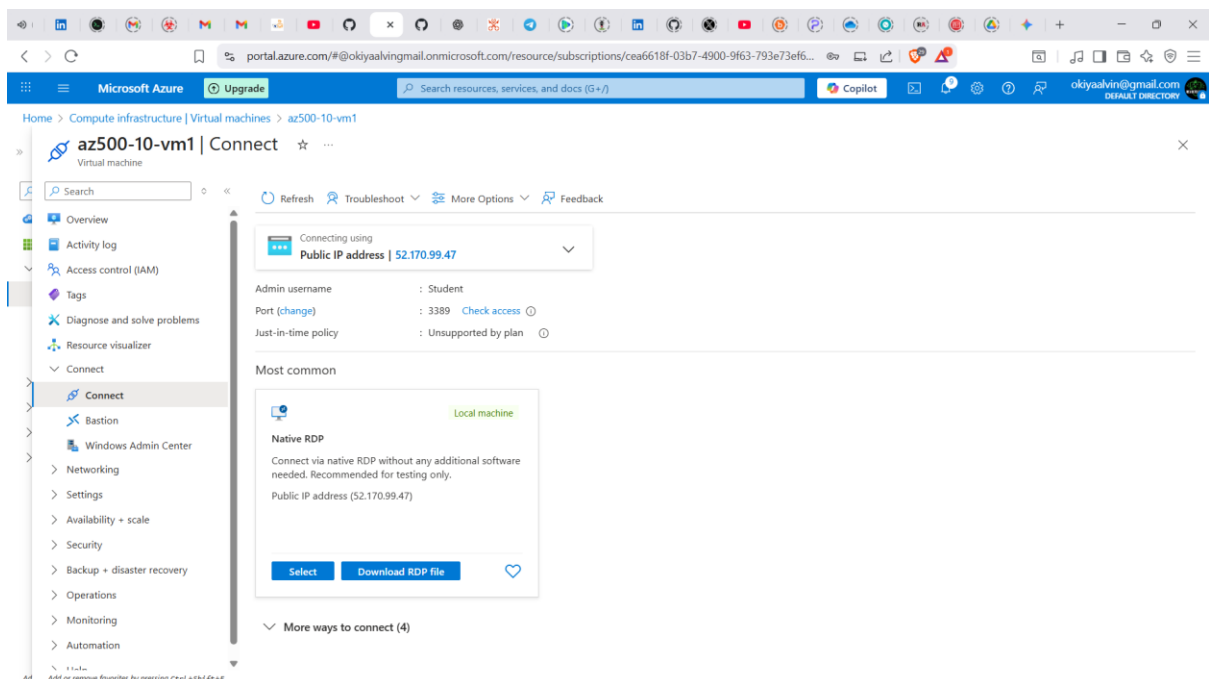
On the **Firewall settings** blade, I scrolled down to Rule Name, click **+ Add a firewall rule**, and specify the following settings:



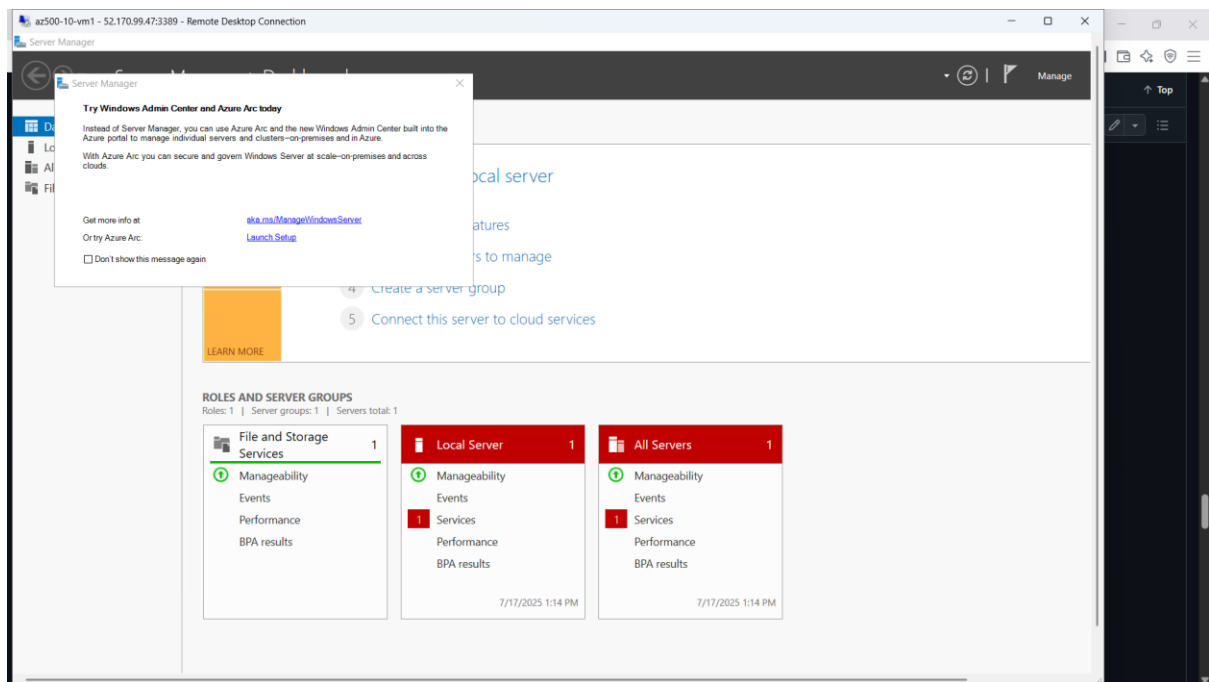
I clicked on save to save the settings.

Next was to download the RDP file

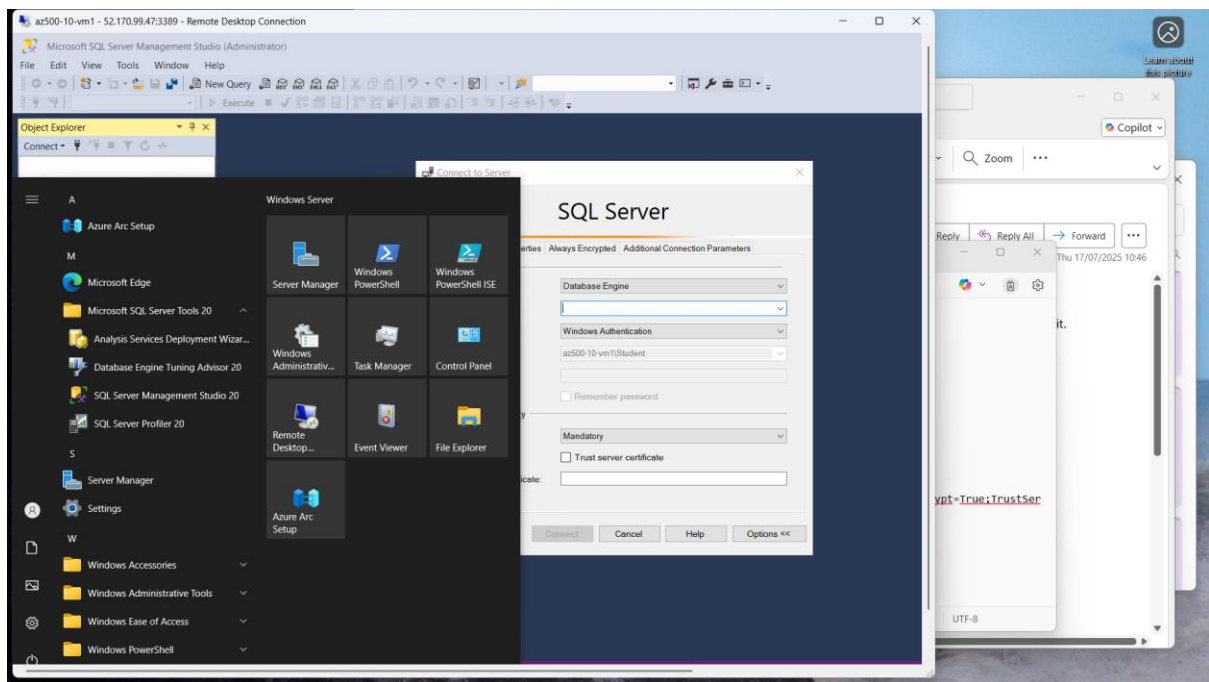
I navigated back to the **az500-10-vm1** > **Overview** > **Connect** and, in the drop down menu, clicked **Connect** then I downloaded the RDP file:



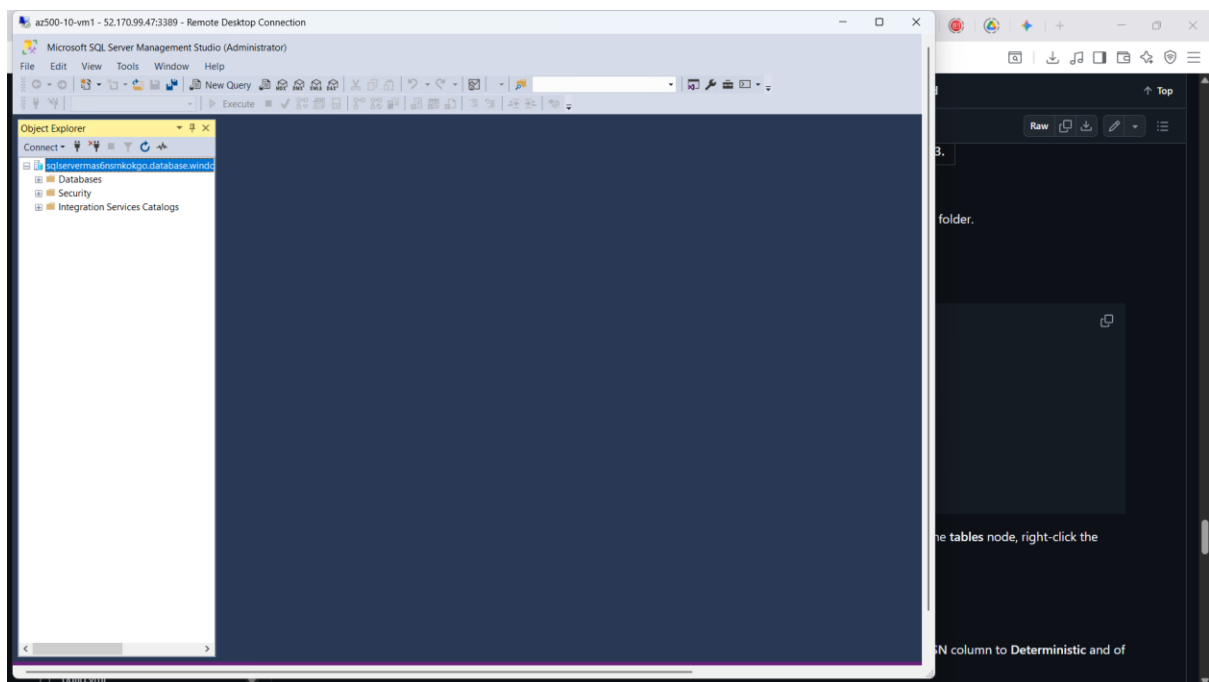
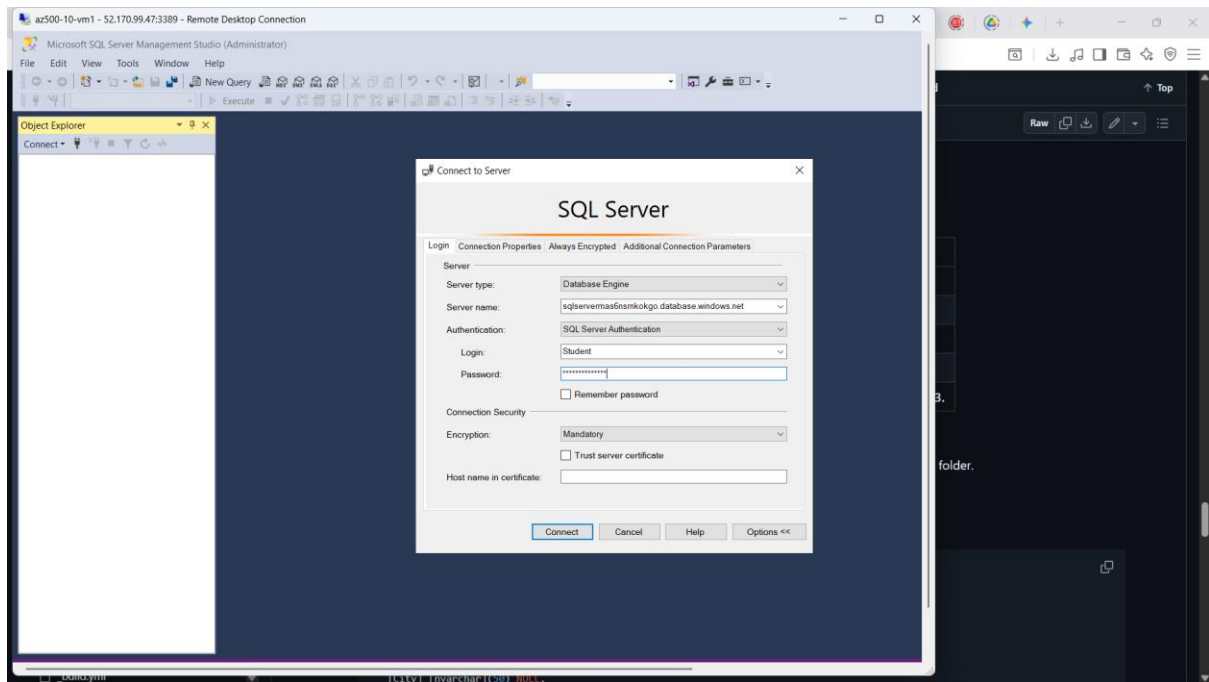
I then connected to the **az500-10-vm1** Azure VM via RDP using the credentials I created on Exercise 1:



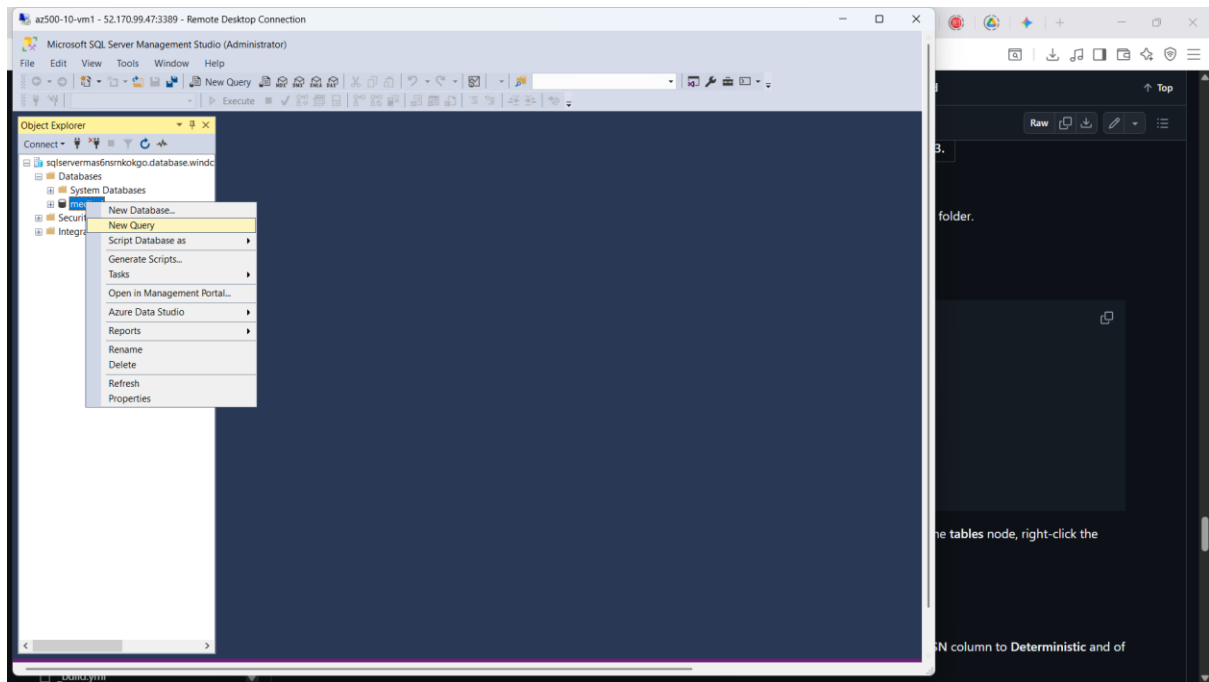
I then closed the server manager and proceeded with installing the [SQL Server Management Studio](#) on **az500-10-vm1**. Azure VM. I clicked on the windows I con as shown below and selected SQL Server Profiler 20.



I then filled the details as shown then clicked on connect:



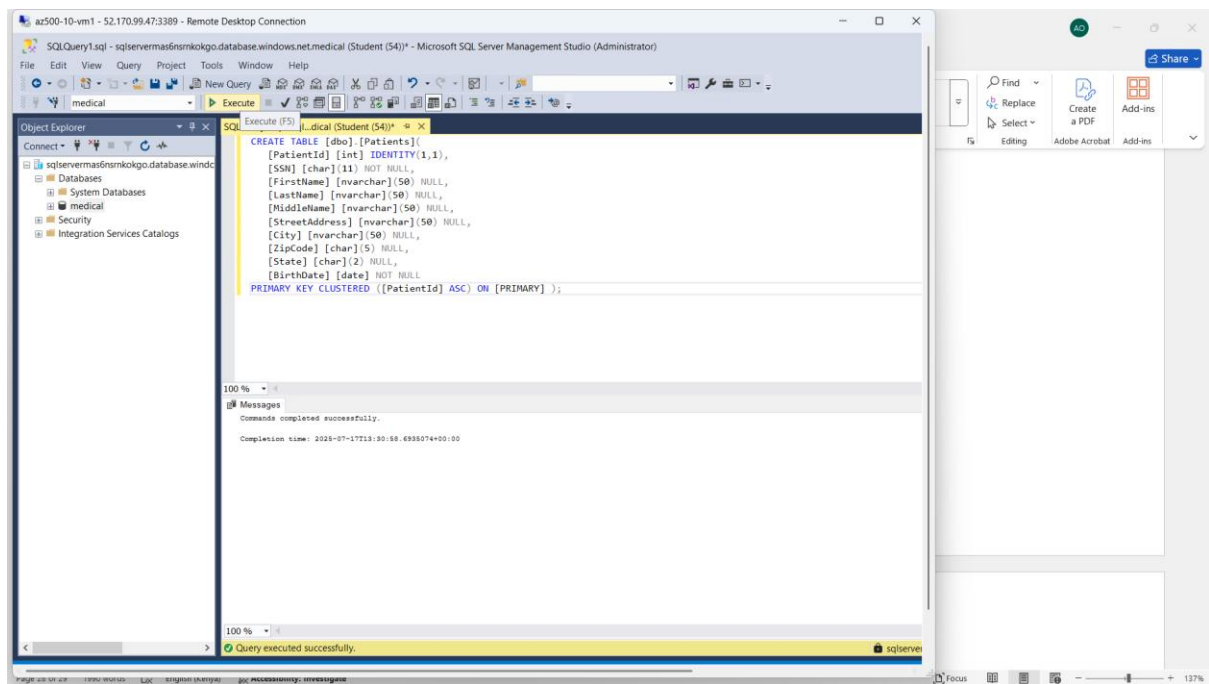
On the object explorer I expanded the Databases section and right clicked on the medical database and selected New query:



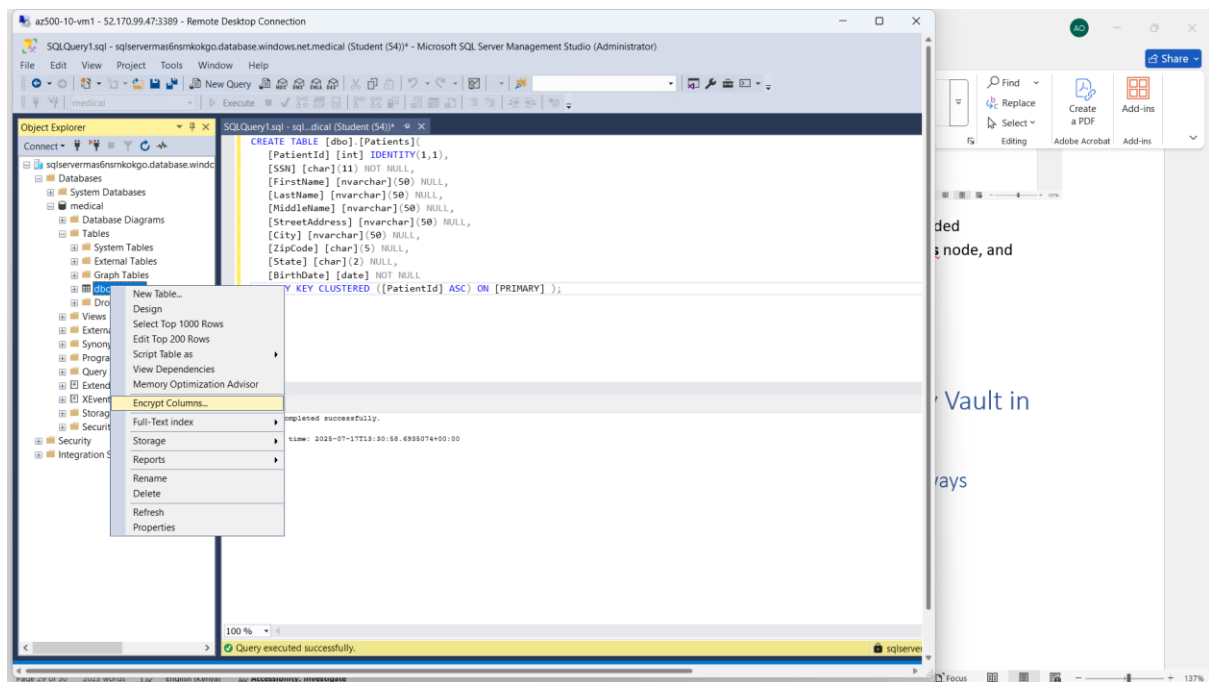
I then pasted the following code to create a patients table in the database:

```
CREATE TABLE [dbo].[Patients](  
    [PatientId] [int] IDENTITY(1,1),  
    [SSN] [char](11) NOT NULL,  
    [FirstName] [nvarchar](50) NULL,  
    [LastName] [nvarchar](50) NULL,  
    [MiddleName] [nvarchar](50) NULL,  
    [StreetAddress] [nvarchar](50) NULL,  
    [City] [nvarchar](50) NULL,  
    [ZipCode] [char](5) NULL,  
    [State] [char](2) NULL,  
    [BirthDate] [date] NOT NULL  
PRIMARY KEY CLUSTERED ([PatientId] ASC) ON [PRIMARY] );
```

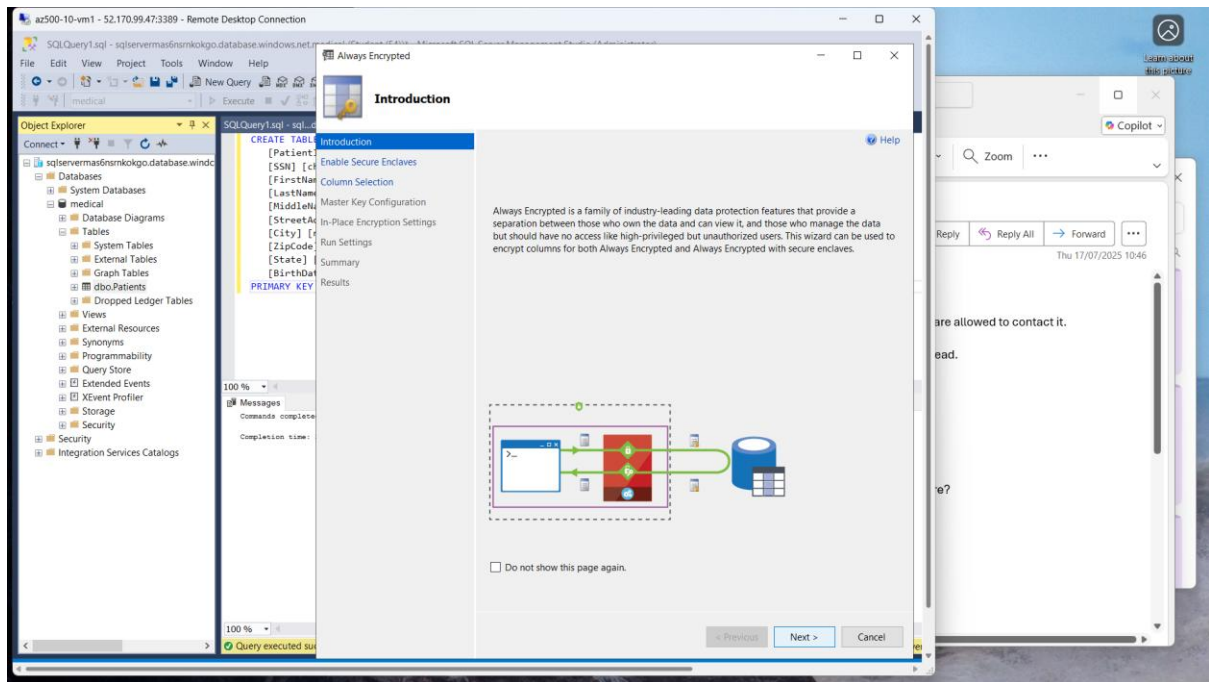
-I then clicked on execute:



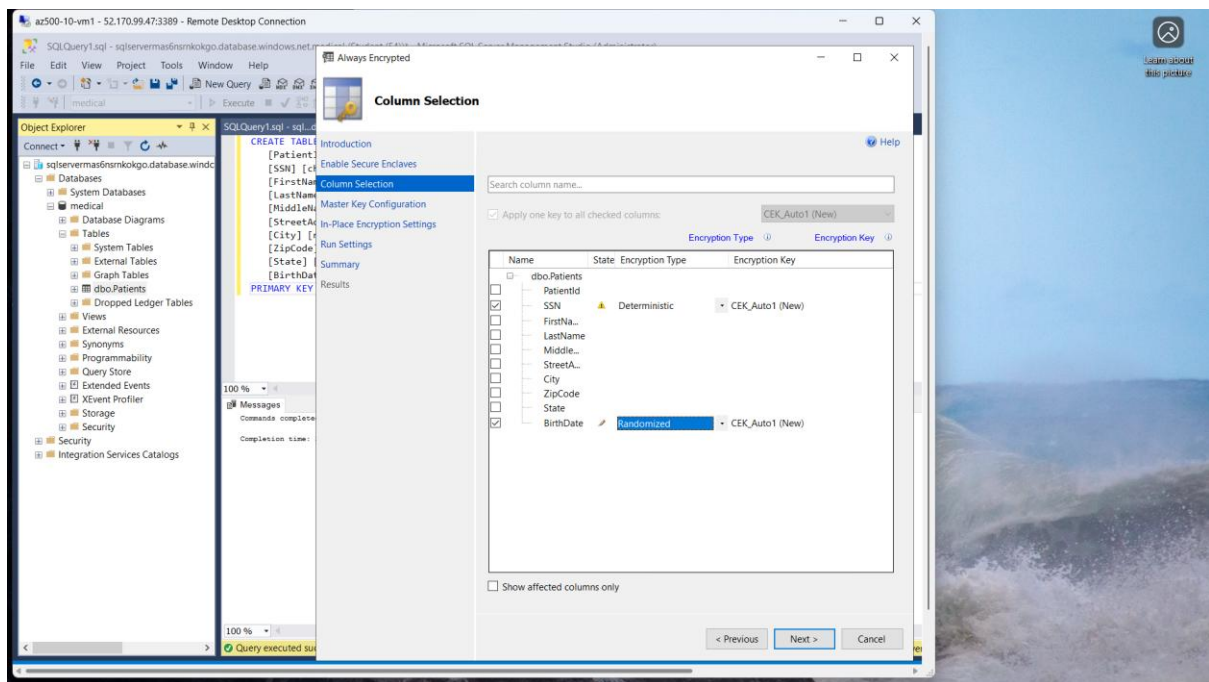
After the table is created successfully, in the **Object Explorer** pane, I expanded the **medical** database node, the **tables** node, right-clicked the **dbo.Patients** node, and clicked **Encrypt Columns**



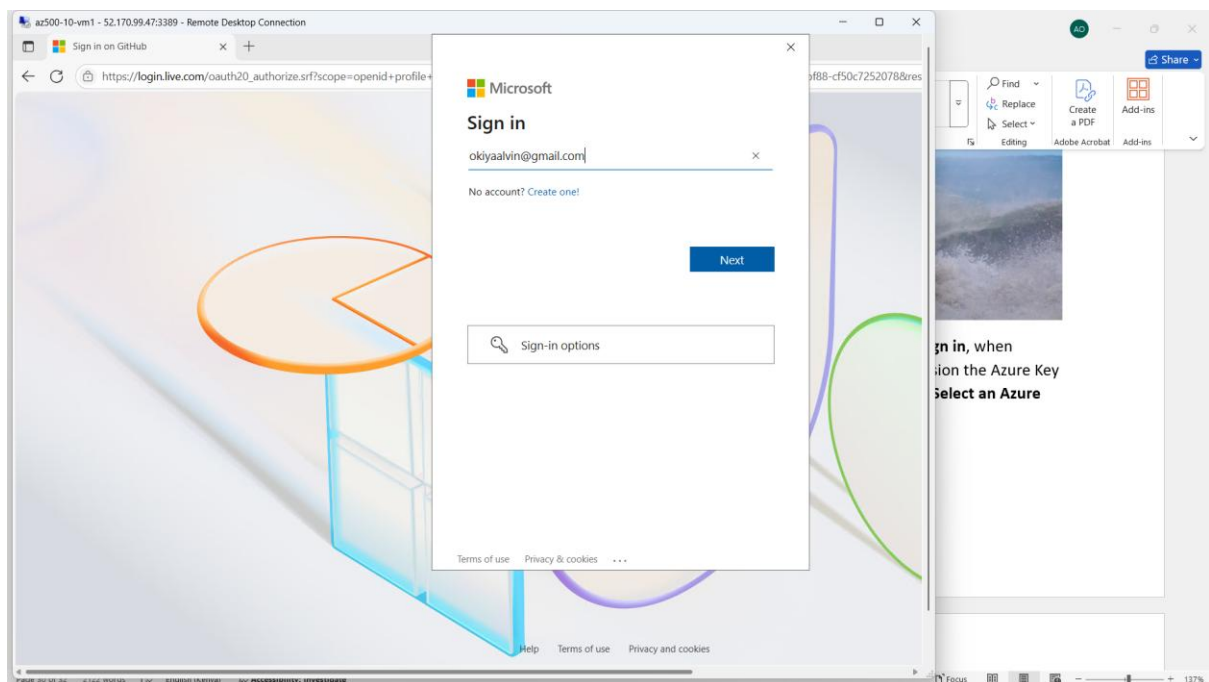
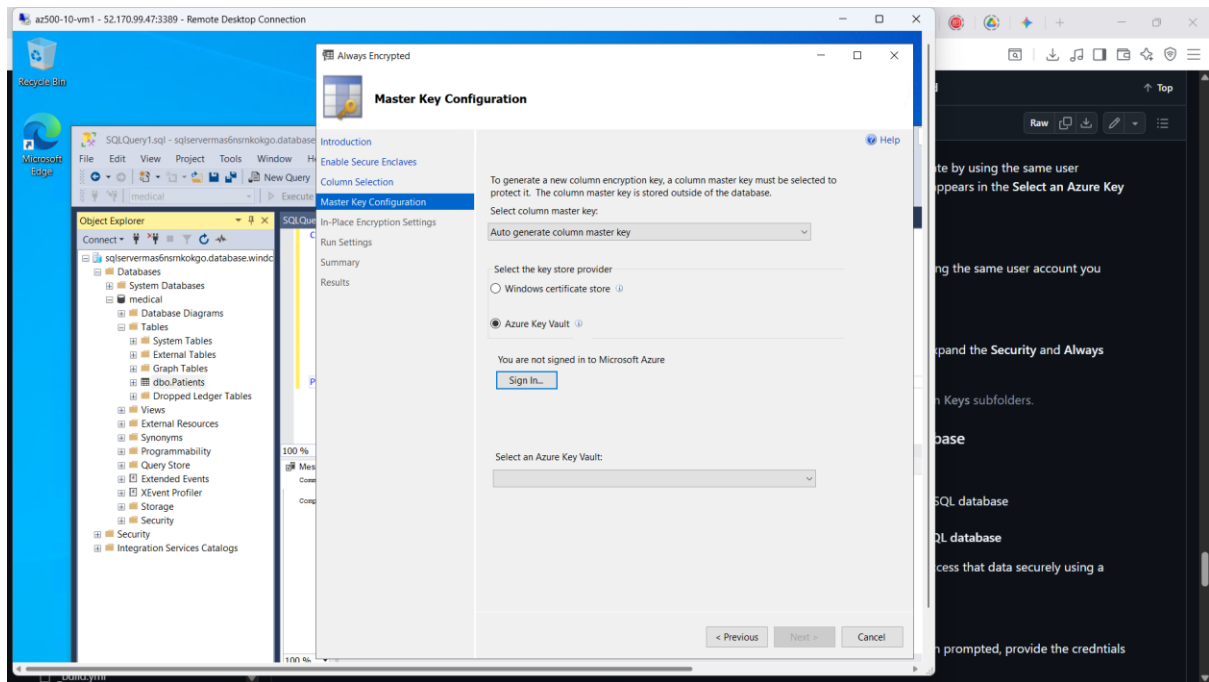
In the always encrypt wizard on the introduction page I clicked Next:



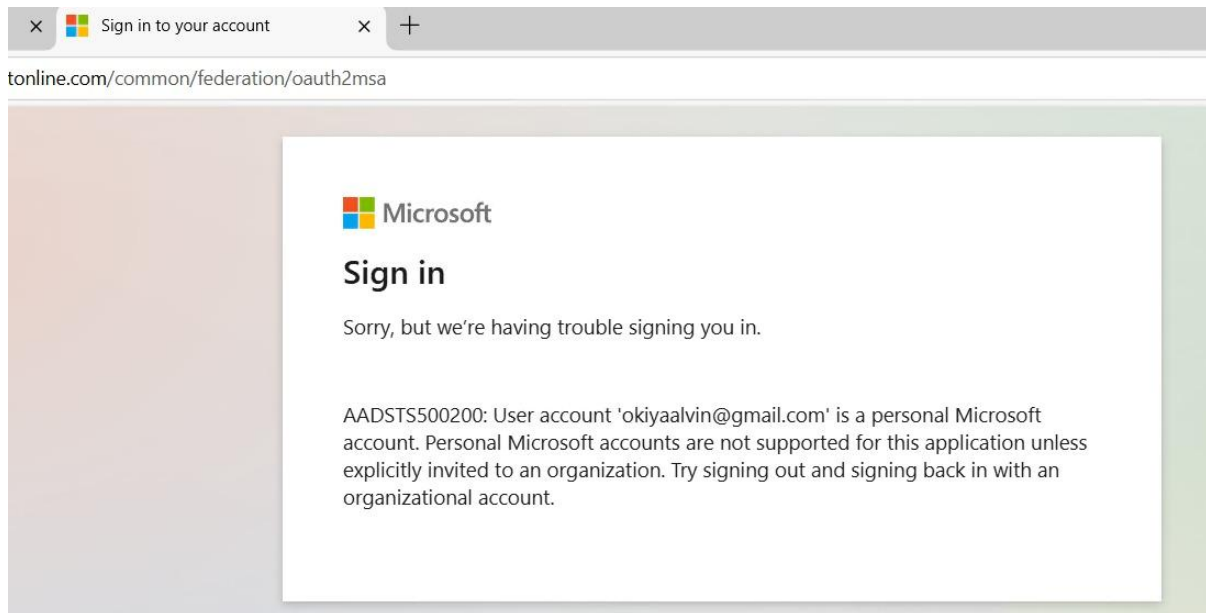
On the **Column Selection** page, I selected the **SSN** and **Birthdate** columns, set the **Encryption Type** of the **SSN** column to **Deterministic** and of the **Birthdate** column to **Randomized**, and clicked **Next**.



On the **Master Key Configuration** page, I selected **Azure Key Vault**, click **Sign in**, when prompted, authenticate by using the same user account used to provision the Azure Key Vault instance earlier in this lab, ensure that that Key Vault appears in the **Select an Azure Key Vault** drop down list, and click **Next**.



At this stage I started experiencing an error:



To resolve it: I followed the following option:

Use / Create an Organizational (Entra ID) User

1. In the Azure portal, go to **Microsoft Entra ID**.
2. Create a **new cloud user** (e.g., student@<yourtenant>.onmicrosoft.com). In my case i used student@okiyaalvingmail.onmicrosoft.com
3. Assign that user **Access Policy permissions** in your **Key Vault** (Key + Secret crypto ops as the lab described):

Step 1: Open Key Vault in Azure Portal

1. Sign in to [Azure Portal](#) with your current account (the one that deployed resources).
2. In the search bar, type **Key Vaults** and select your Key Vault (e.g., az500kv...).

Step 2: Navigate to Access Policies

1. Inside the Key Vault, go to **Access policies** (under Settings).
2. Click **+ Create** (or **Add Access Policy**).

Step 3: Configure Access Policy

- **Configure from template:** Choose **Key, Secret & Certificate Management**.
- Under **Key Permissions**, ensure:
 - Select all (or at least **Get, Wrap Key, Unwrap Key, Sign, Verify, List**).
- Under **Secret Permissions**:

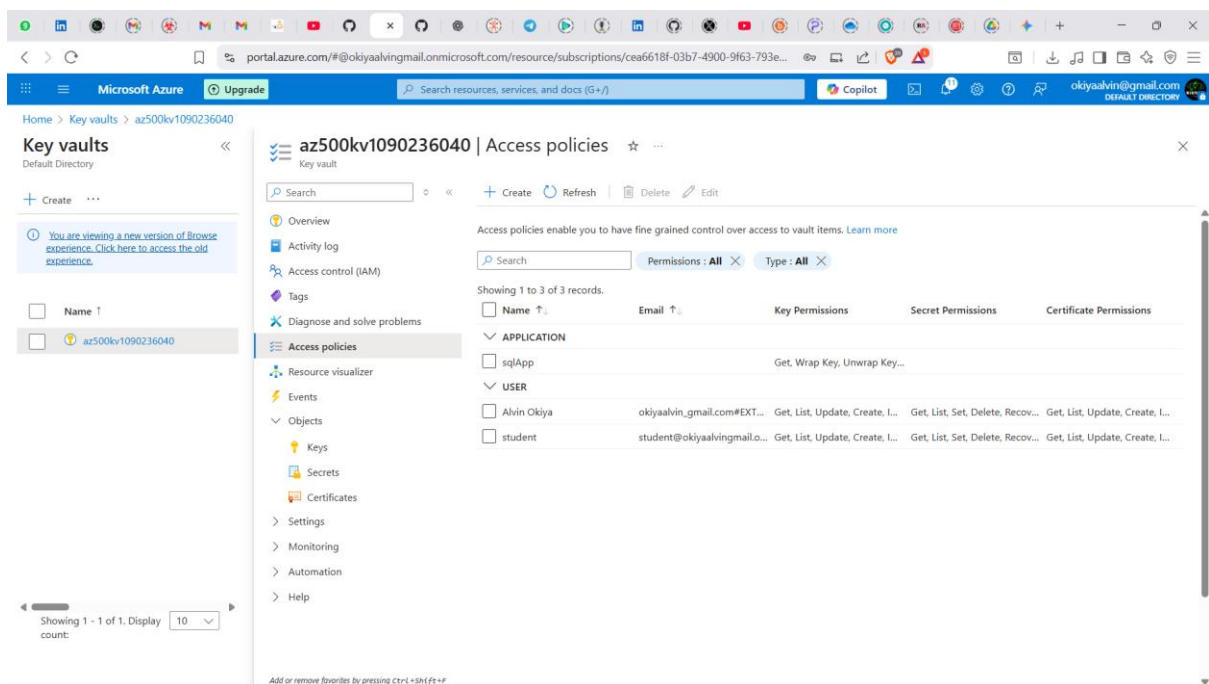
- Select all (or at least **Get, List**).
- (Certificates can remain as default unless required.)

Step 4: Select Principal (the New User)

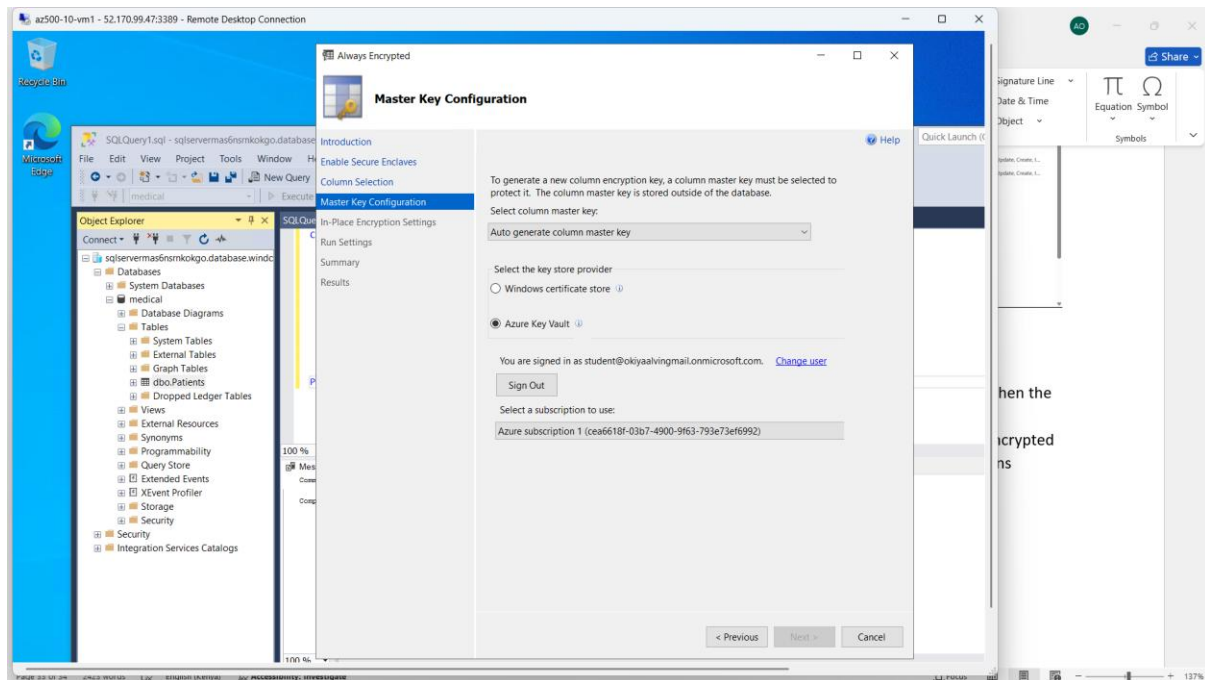
1. In the same panel, click **Select principal**.
2. Search for the new user you created (e.g., student@<yourtenant>.onmicrosoft.com).
3. Click **Select**.

Step 5: Save and Apply

- Click **Next** → Review the policy → Click **Create** (or **Add**).
- Confirm the user now appears in the Access Policies list



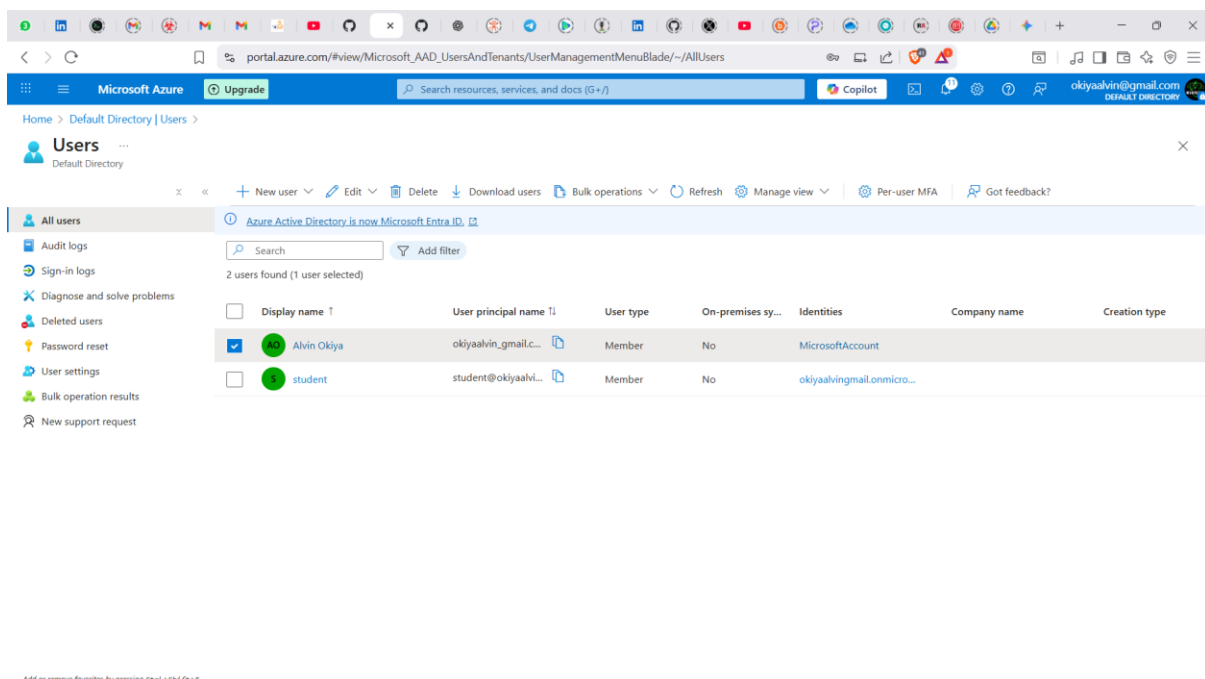
4. Give that user at least **Contributor** (or sufficient) RBAC role on the subscription/resource group if needed to see resources.
 5. Sign out in SSMS / the lab VM browser, then sign in with the **new org user** when the wizard prompts.
- This aligns with typical setup guidance for using Azure Key Vault in Always Encrypted scenarios, which require proper Azure AD identities and Key Vault permissions



However, this did not work since the “Next” Key as shown on the screenshot is not enabled and theres no option to select a Key Vault.

Due to this I decided to go back to my azure portal and on the search bar, I searched for Microsoft Entra Id and on opening it, I navigated to the users section and copied the initial email address okiyaalvin@gmail.com but as it appears on entra ID:

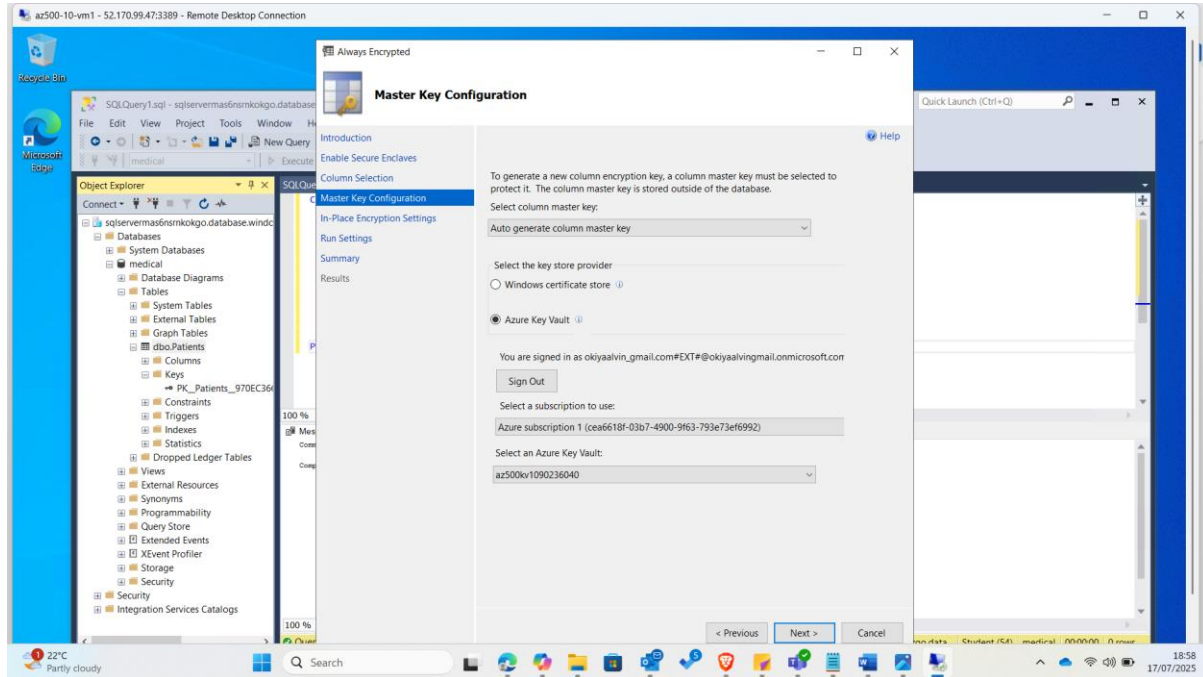
okiyaalvin_gmail.com#EXT#@okiyaalvingmail.onmicrosoft.com



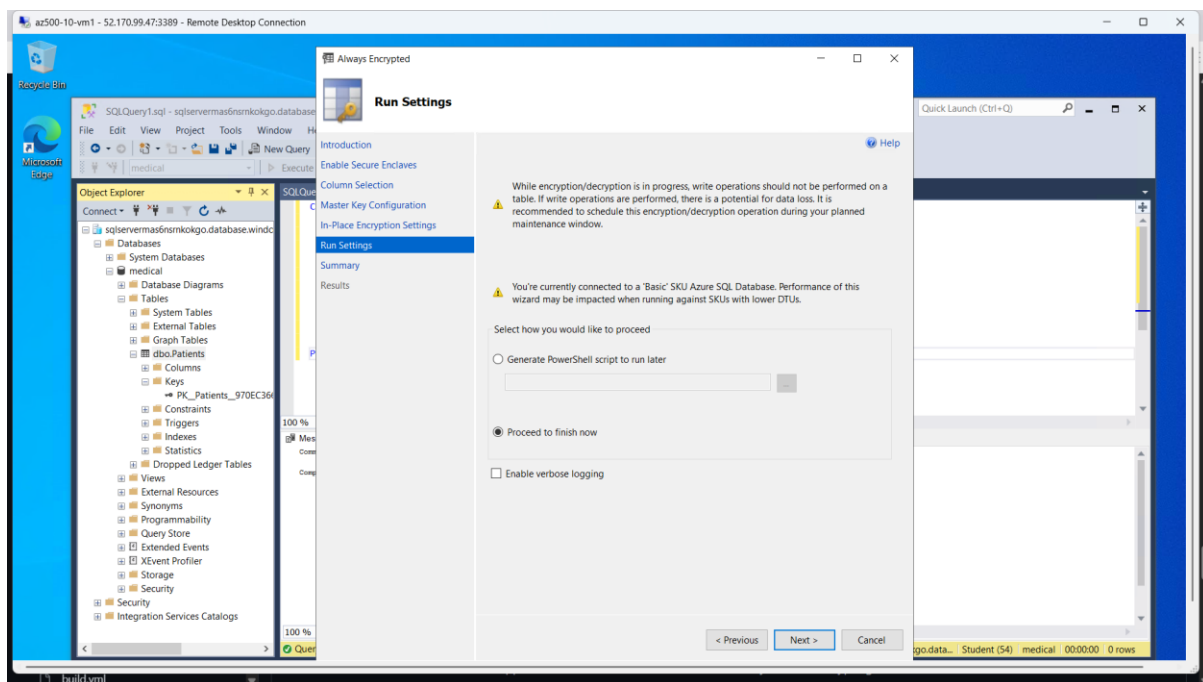
I then went to the RDP session and signed out of student@okiyaalvingmail.onmicrosoft.com and signed in using okiyaalvin_gmail.com#EXT#@okiyaalvingmail.onmicrosoft.com. This

time round it accepted without any errors. I signed out of the master configuration page, closed it and repeated the process from right clicking dbo.patients, selecting encrypt columns, then signing into the Master keyvault using okiyaalvin@gmail.com#EXT#@okiyaalvingmail.onmicrosoft.com

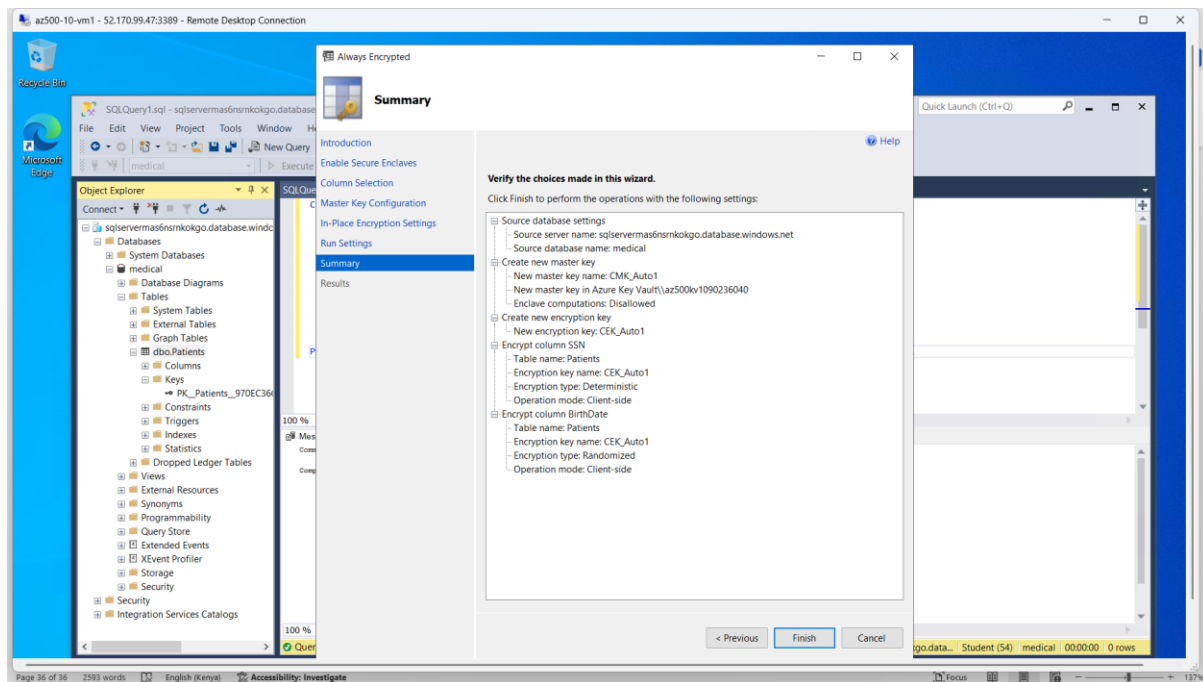
As you can see below, the error was now resolved and I was able to select the Key vault and proceed:



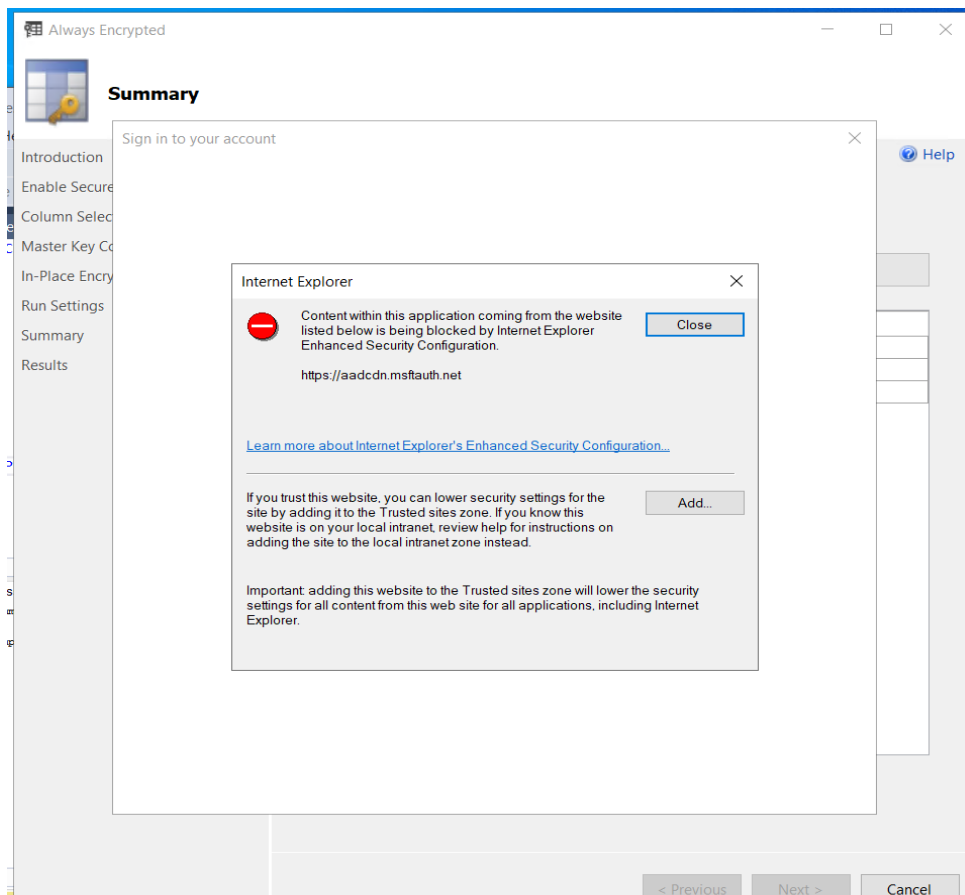
On the Run settings page I clicked Next:



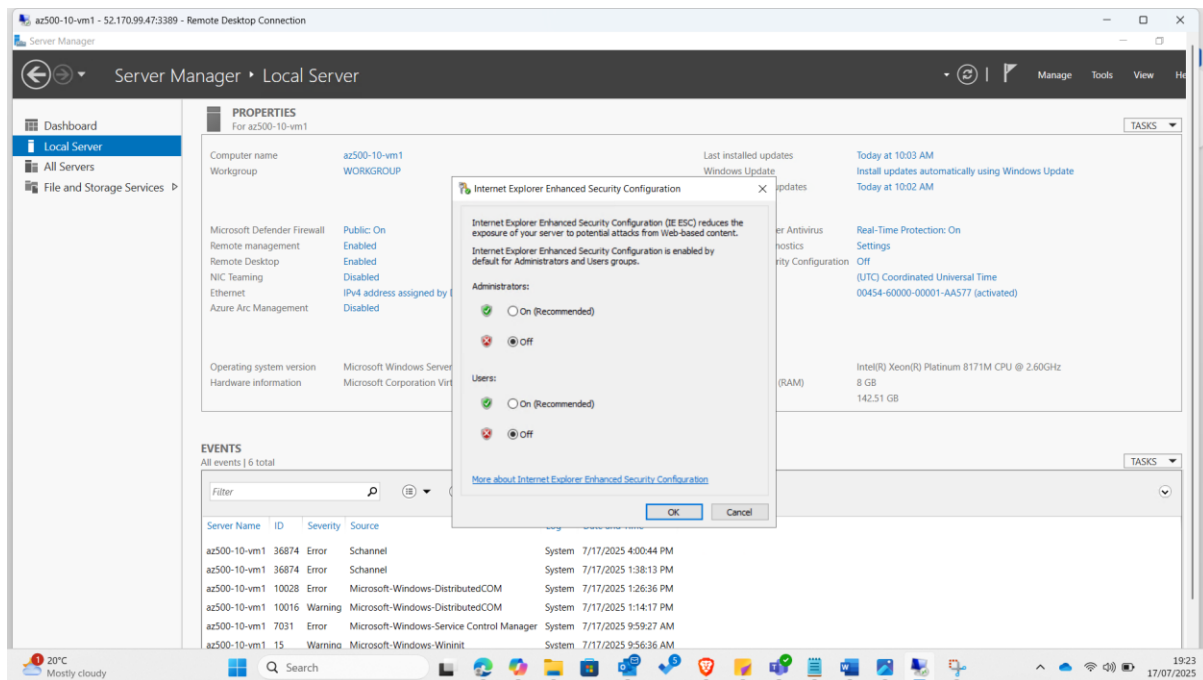
Then on the Summary page I clicked finish to proceed with the encryption:



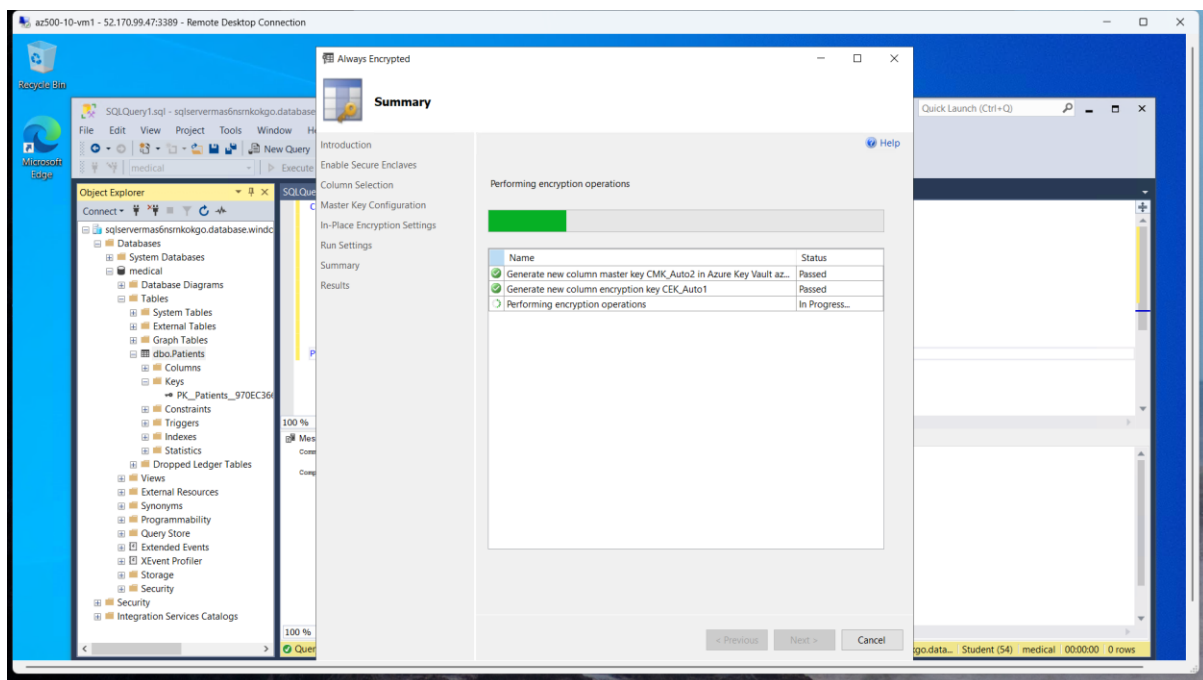
However, on clicking finish, the error below popped up:



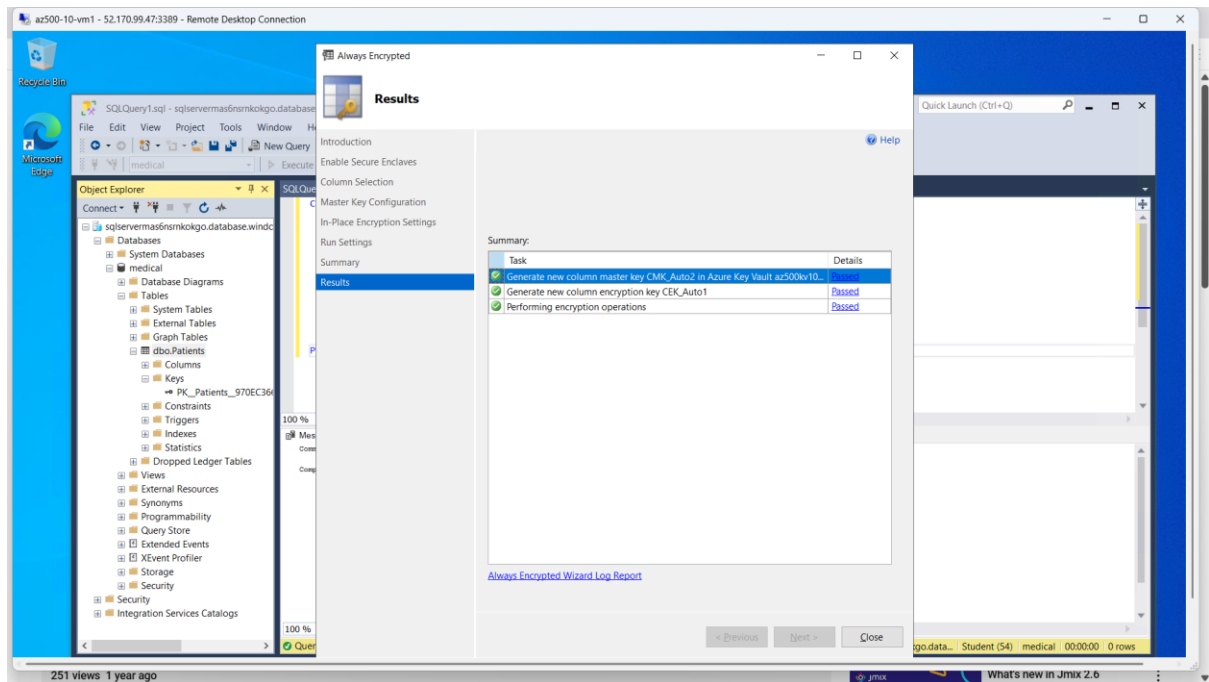
To solve this, I disabled the IE Enhanced security on the server manager:



This enabled me move to the next step, and prompted me to log in to my azure account, I used the email okiyaalvin@gmail.com#EXT#@okiyaalvingmail.onmicrosoft.com

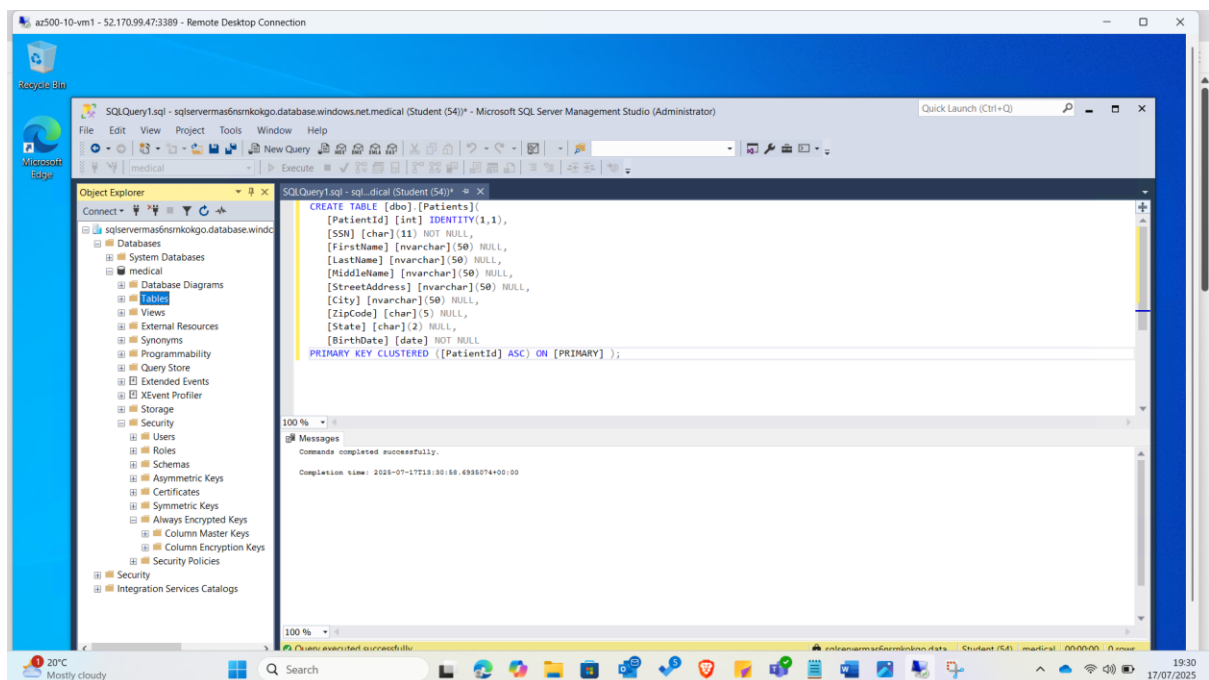


Once the encryption process completed, on the results page I clicked close:



On the objects explorer section, under the Medical > Security > Always Encrypted Keys section, I confirmed the success of this section by being able to view the subfolders:

Column Master Keys and Column Encryption Keys

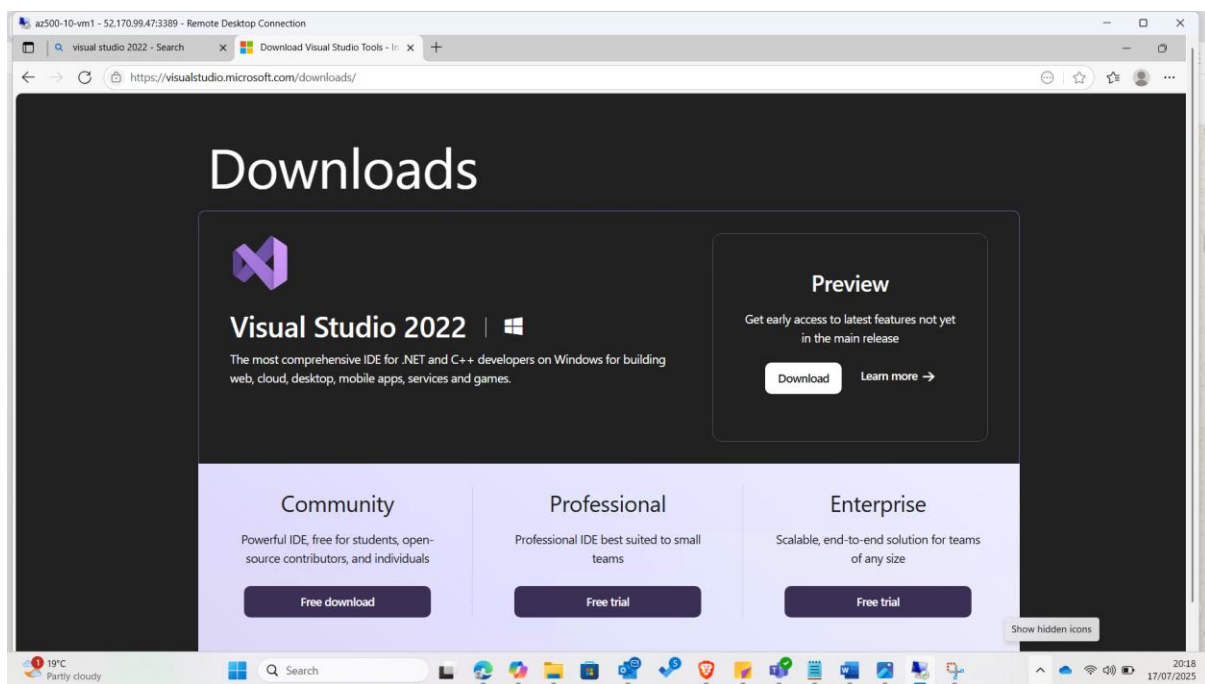


Exercise 4: Demonstrate the Use of Azure Key Vault in Encrypting the Azure SQL Database

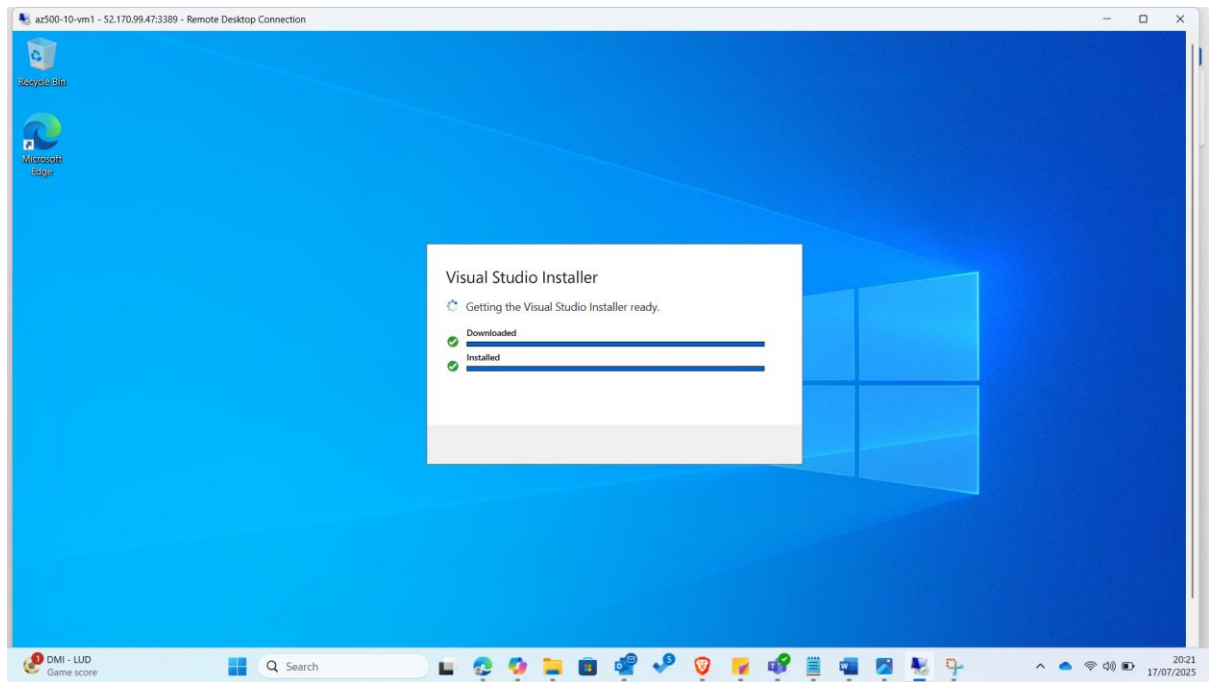
Task 1: Run a data-driven application to demonstrate Always Encrypted with Azure Key Vault

In this section, I created a Console application using Visual Studio to load data into the encrypted columns and then accessed that data securely using a connection string that accesses the key in the Key Vault. I followed the steps below:

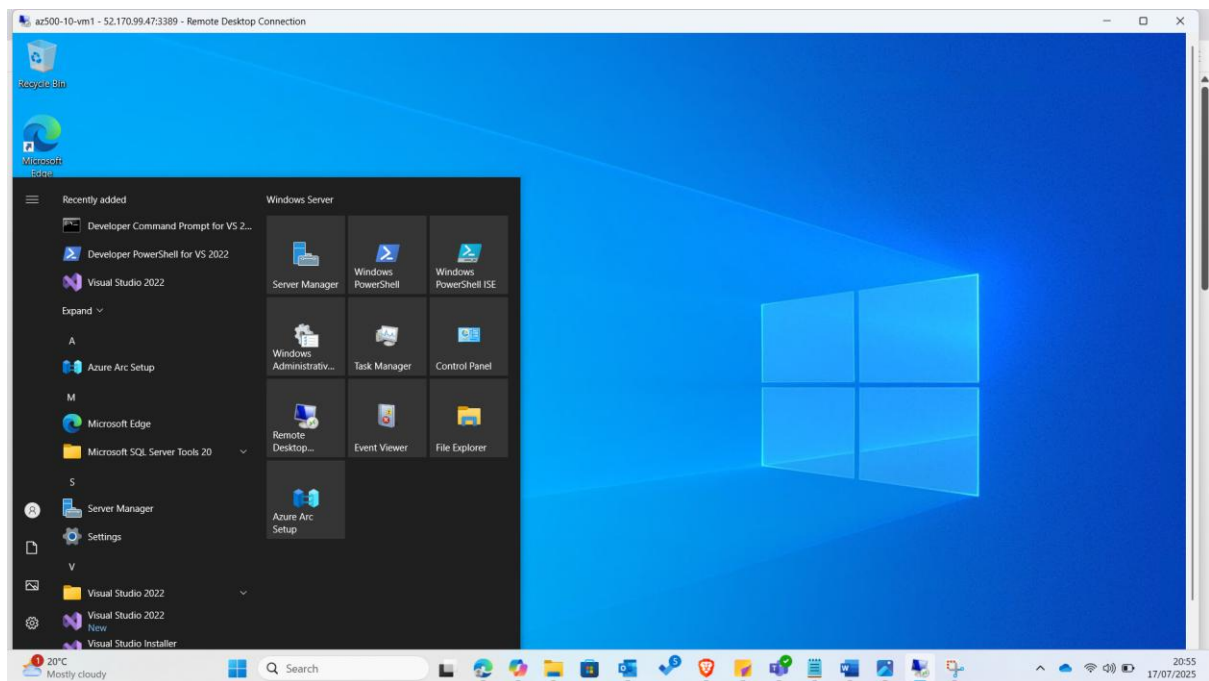
From the RDP session to the **az500-10-vm1**, I tried to launch **Visual Studio** from the **Start menu**. However, I did not find it so I opted to download it:



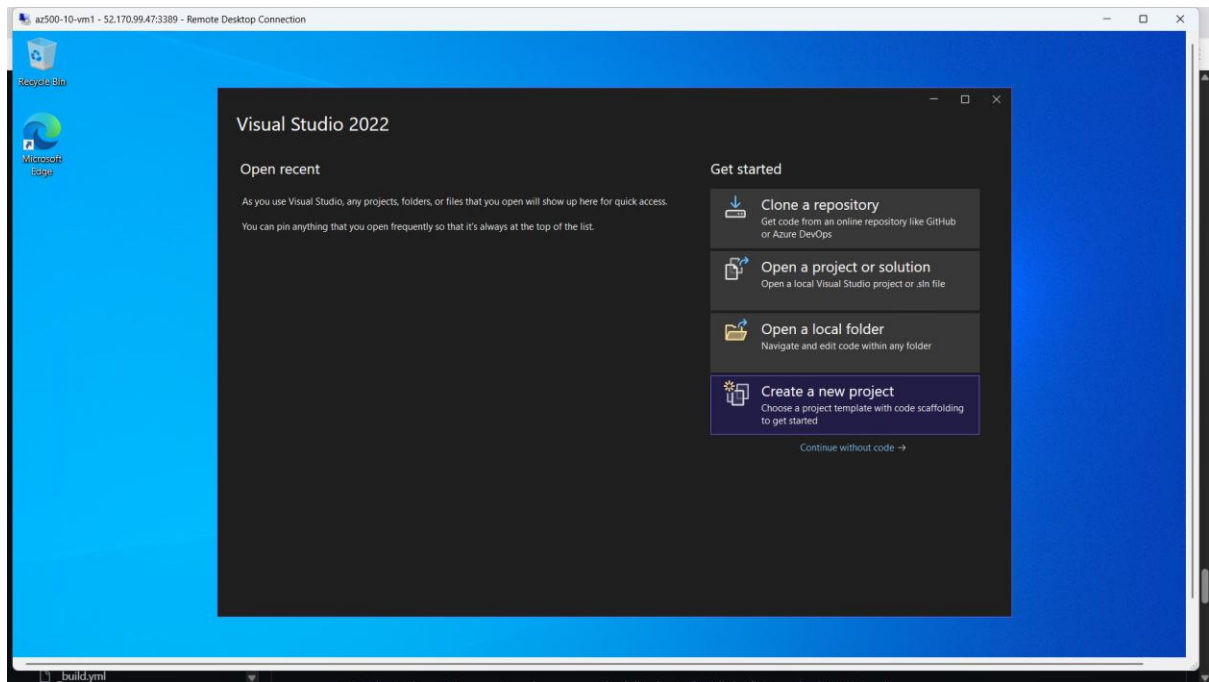
Then installed it:



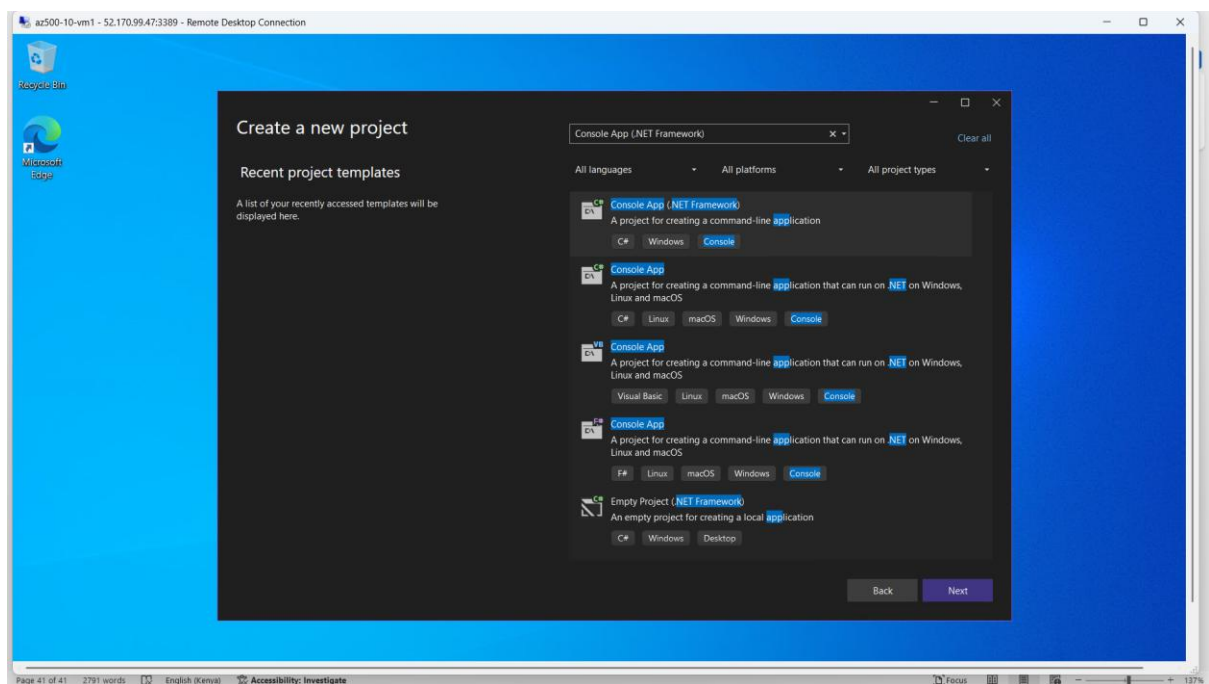
After installation was complete, I proceeded to launch it from the start menu:



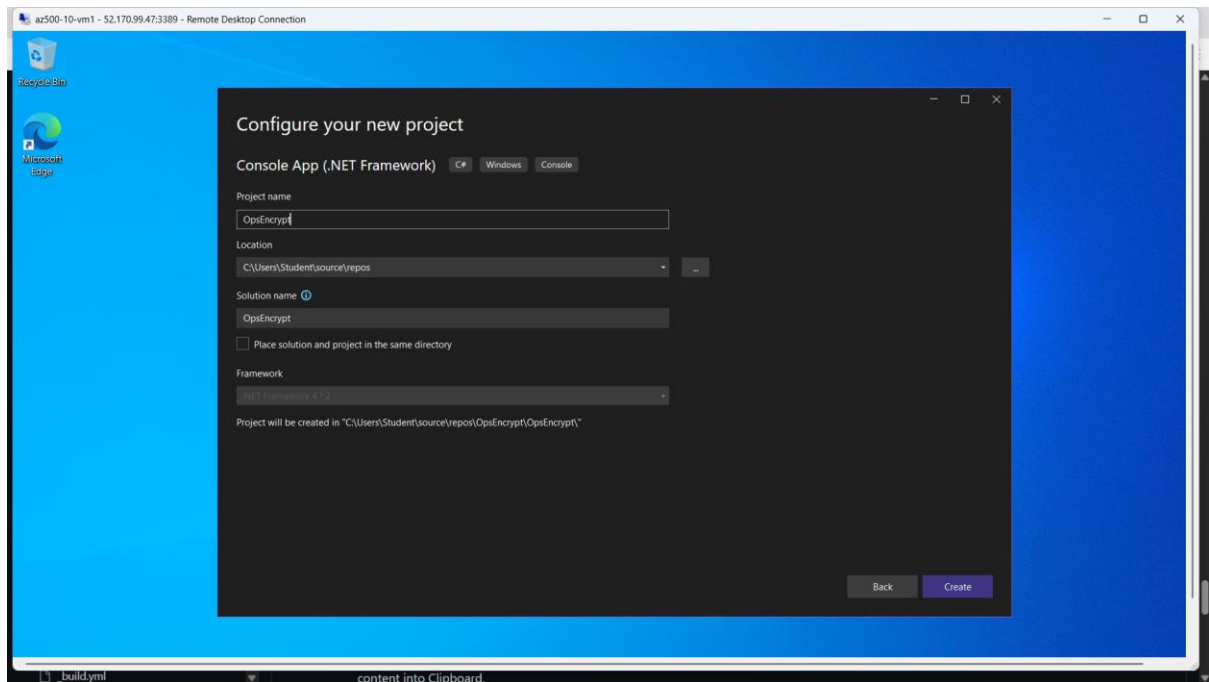
I then clicked on Create a New Project:



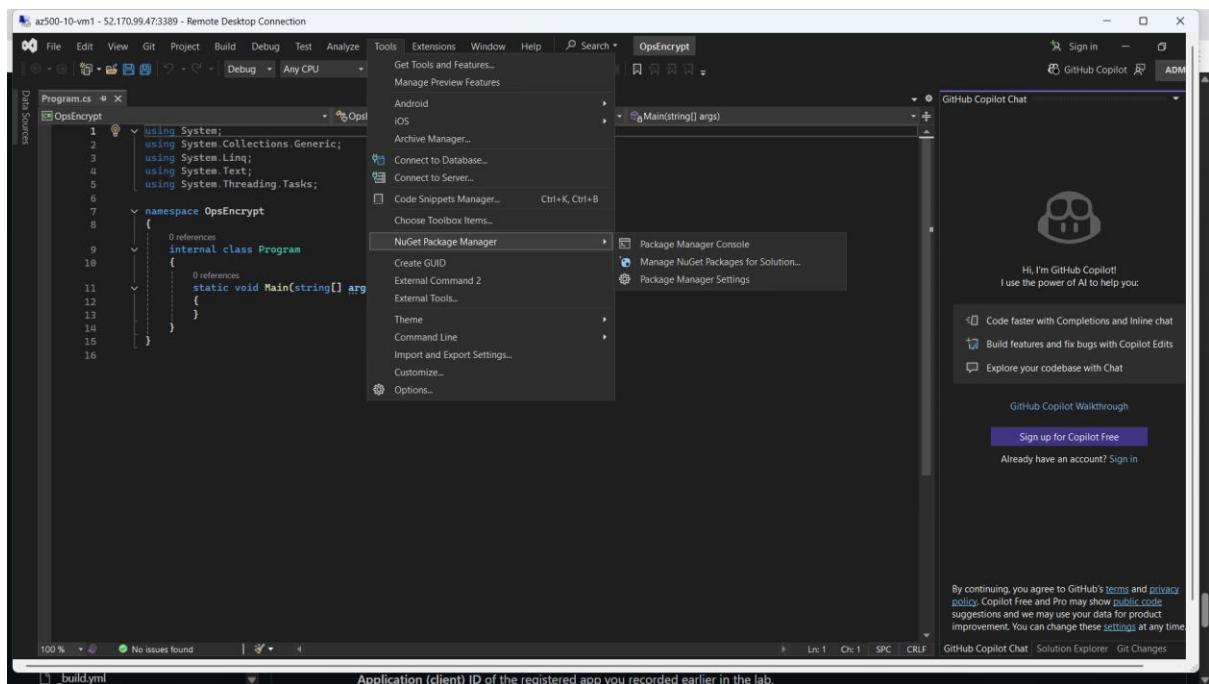
On the search bar I searched for **Console App (.NET Framework)** and selected the first option then clicked on Next:



And on the Specify your new projects page I specified the following settings then clicked Create:

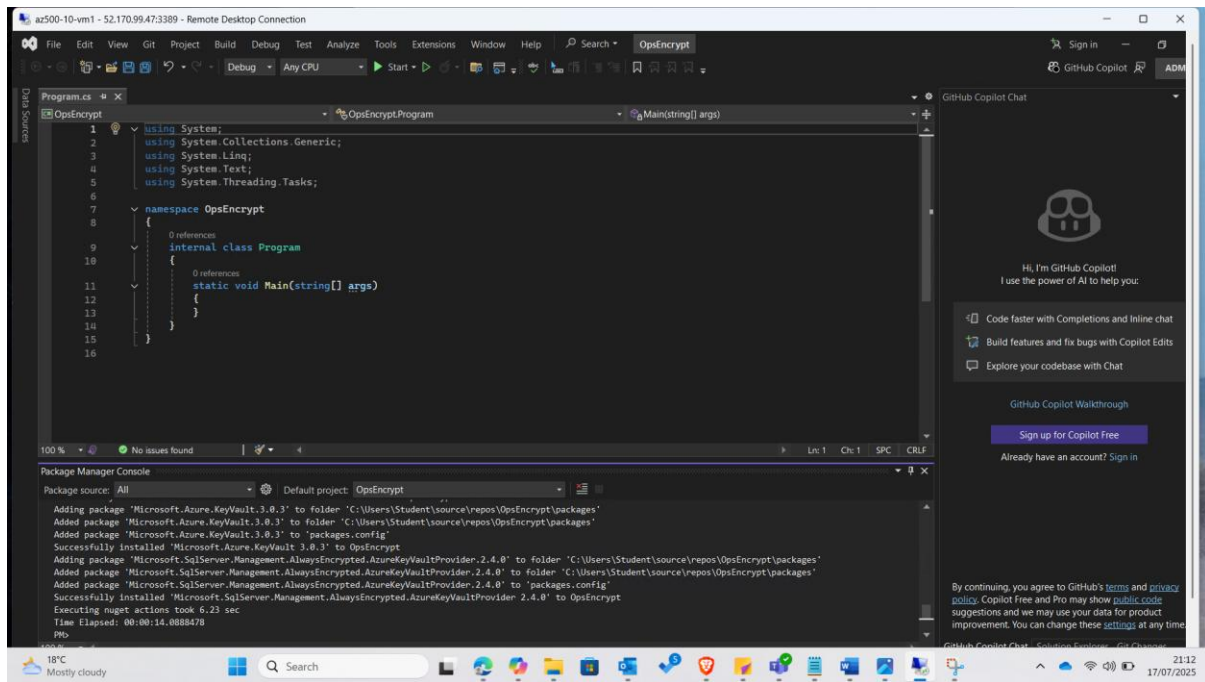


In the Visual Studio console, I clicked the **Tools** menu, in the drop down menu, I clicked **NuGet Package Manager**, and, in the cascading menu, I clicked **Package Manager Console**.



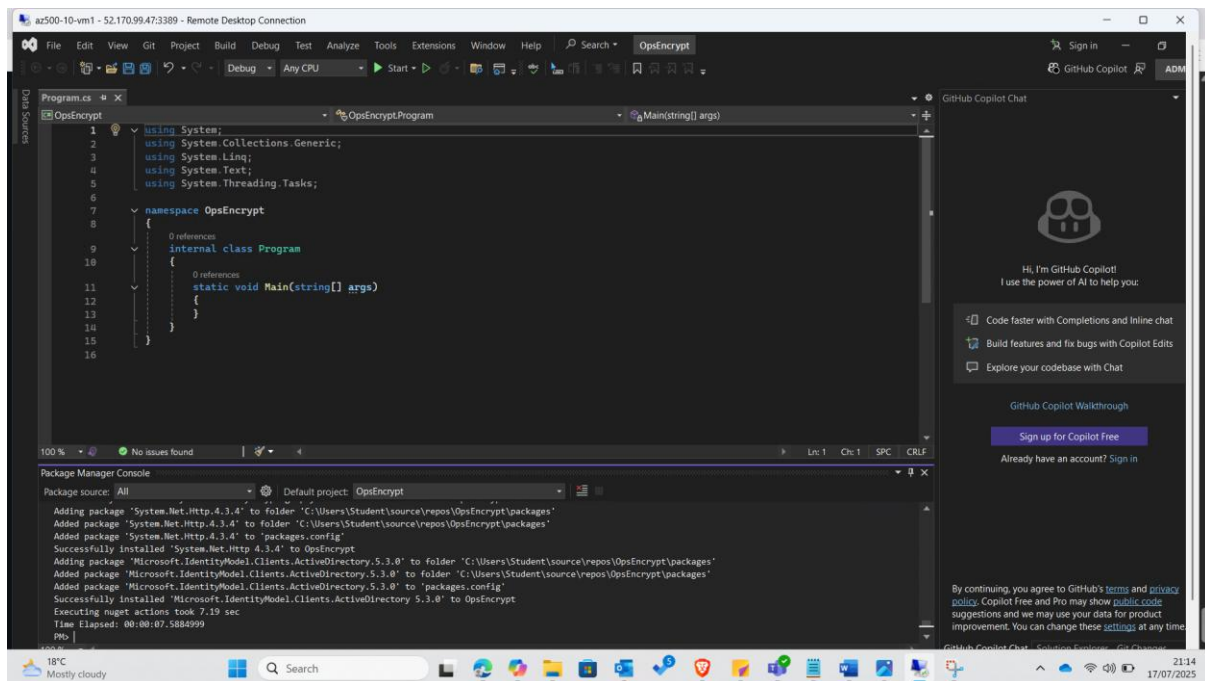
In the package manager console, I ran the following command to install the first required NuGet package

Install-Package Microsoft.SqlServer.Management.AlwaysEncrypted.AzureKeyVaultProvider



I then ran the following command to install the second required NuGet packages:

Install-Package Microsoft.IdentityModel.Clients.ActiveDirectory



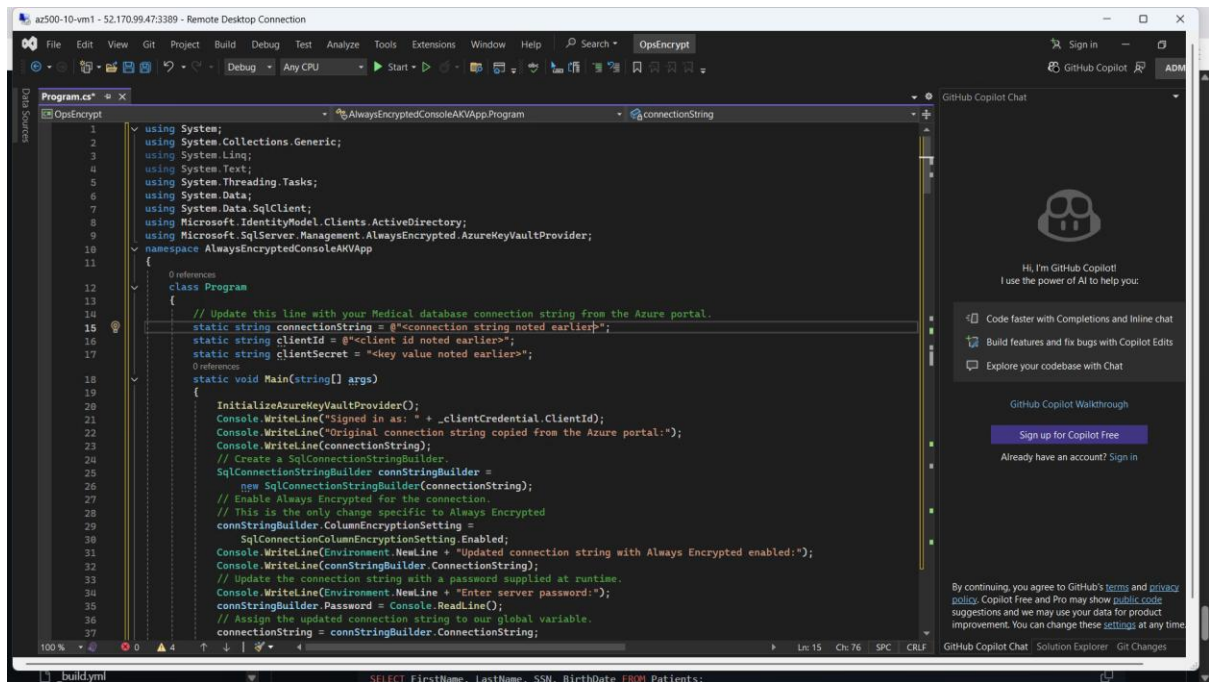
I then minimized the RDP session and on my browser, navigated to the Allfiles/Labs/10/program.cs file in the lab in github and copied it on my clipboard:


```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using System.Data;
7 using System.Data.SqlClient;
8 using Microsoft.IdentityModel.Clients.ActiveDirectory;
9 using Microsoft.SqlServer.Management.AlwaysEncrypted.AzureKeyVaultProvider;
10 namespace AlwaysEncryptedConsoleKVApp
11 {
12     class Program
13     {
14         // Update this line with your Medical database connection string from the Azure portal.
15         static string connectionString = @"<connection string noted earlier>";
16         static string clientId = @"<client id noted earlier>";
17         static string clientSecret = @"<key value noted earlier>";
18         static void Main(string[] args)
19         {
20             InitializeAzureKeyVaultProvider();
21             Console.WriteLine("Signed in as: " + _clientCredential.ClientId);
22             Console.WriteLine("Original connection string copied from the Azure portal:");
23             Console.WriteLine(connectionString);
24             // Create a SqlConnectionStringBuilder.
25             SqlConnectionStringBuilder connStringBuilder =
```

I then got back to the RDP session and pasted the clipboard content to replace the content on program.cs file:

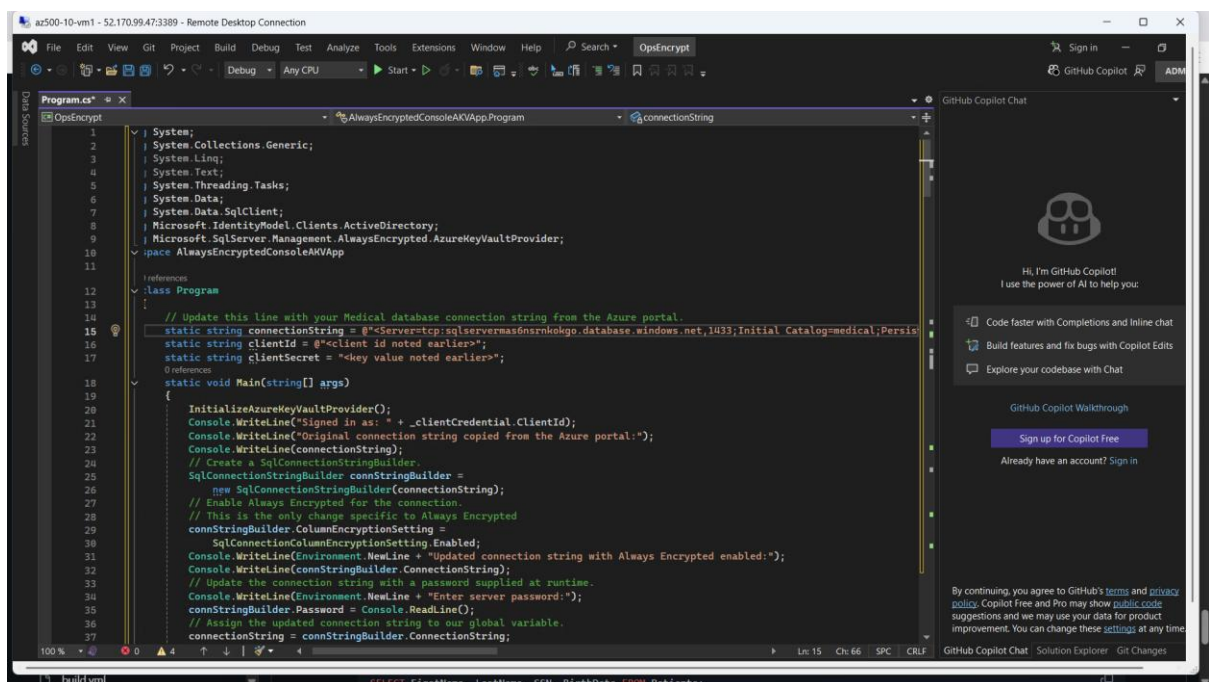
```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using System.Data;
7 using System.Data.SqlClient;
8 using Microsoft.IdentityModel.Clients.ActiveDirectory;
9 using Microsoft.SqlServer.Management.AlwaysEncrypted.AzureKeyVaultProvider;
10 namespace AlwaysEncryptedConsoleKVApp
11 {
12     class Program
13     {
14         // Update this line with your Medical database connection string from the Azure portal.
15         static string connectionString = "Server=tcp:az500-10.database.windows.net,1433;Initial Catalog=Medical;Persist Security Info=False;User ID=adomins;Password=StrongP@ssw0rd!;MultipleActiveResultSets=False;Encrypt=True;TrustServerCertificate=False;Connection Timeout=30;";
16         static string clientId = @"<client id noted earlier>";
17         static string clientSecret = @"<key value noted earlier>";
18         static void Main(string[] args)
19         {
20             InitializeAzureKeyVaultProvider();
21             Console.WriteLine("Signed in as: " + _clientCredential.ClientId);
22             Console.WriteLine("Original connection string copied from the Azure portal:");
23             Console.WriteLine(connectionString);
24             // Create a SqlConnectionStringBuilder.
25             SqlConnectionStringBuilder connStringBuilder =
```

I then scrolled to line 15 of the code in visual studio and replaced the <connection string noted earlier> placeholder with the Azure SQL database **ADO.NET** connection string I recorded earlier in the lab.



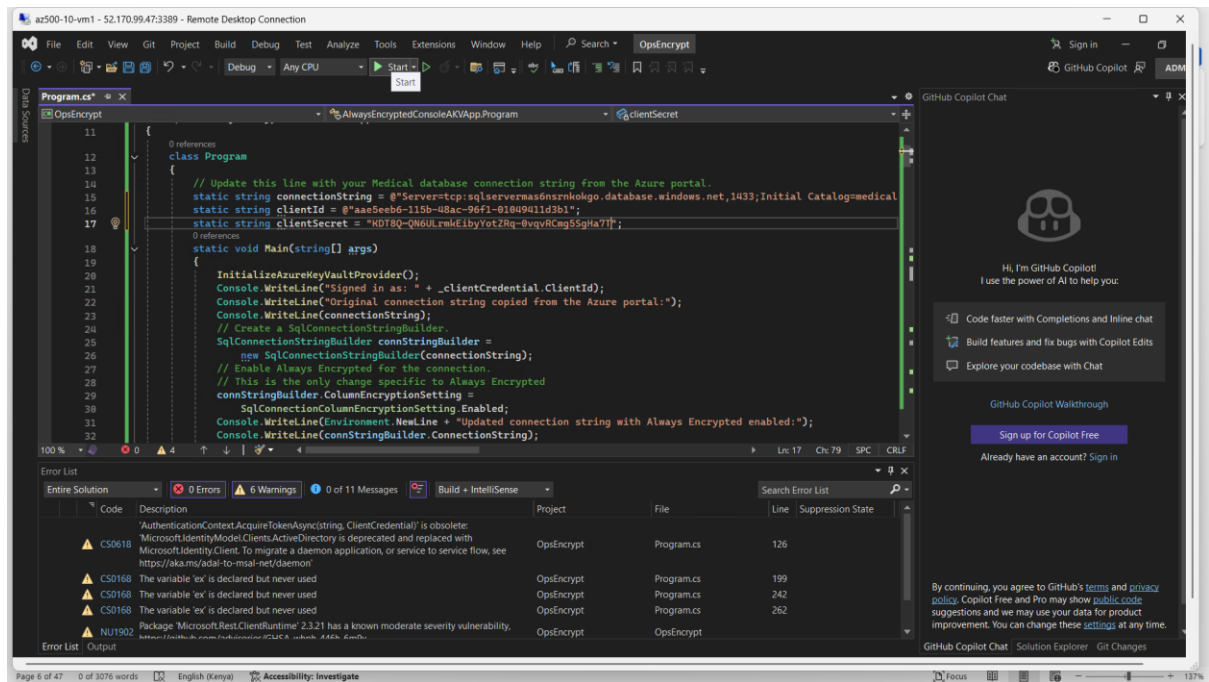
```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using System.Data;
7 using System.Data.SqlClient;
8 using Microsoft.IdentityModel.Clients.ActiveDirectory;
9 using Microsoft.SqlServer.Management.AlwaysEncrypted.AzureKeyVaultProvider;
10 namespace AlwaysEncryptedConsoleAKVApp
11 {
12     0 references
13     class Program
14     {
15         // Update this line with your Medical database connection string from the Azure portal.
16         static string connectionString = @"<connection string noted earlier>";
17         static string clientId = @"<client id noted earlier>";
18         static string clientSecret = "<key value noted earlier>";
19         0 references
20         static void Main(string[] args)
21         {
22             InitializeAzureKeyVaultProvider();
23             Console.WriteLine("Signed in as: " + _clientCredential.ClientId);
24             Console.WriteLine("Original connection string copied from the Azure portal:");
25             Console.WriteLine(connectionString);
26             // Create a SqlConnectionStringBuilder
27             SqlConnectionStringBuilder connStringBuilder =
28                 new SqlConnectionStringBuilder(connectionString);
29             // Enable Always Encrypted for the connection.
30             // This is the only change specific to Always Encrypted
31             connStringBuilder.ColumnEncryptionSetting =
32                 SqlConnectionColumnEncryptionSetting.Enabled;
33             Console.WriteLine(Environment.NewLine + "Updated connection string with Always Encrypted enabled:");
34             Console.WriteLine(connStringBuilder.ConnectionString);
35             // Update the connection string with a password supplied at runtime.
36             Console.WriteLine(Environment.NewLine + "Enter server password:");
37             connStringBuilder.Password = Console.ReadLine();
38             // Assign the updated connection string to our global variable.
39             connectionString = connStringBuilder.ConnectionString;
```

In the connection string, I replaced the {your_password} placeholder, with the password that I specified in the deployment template in Exercise 1.

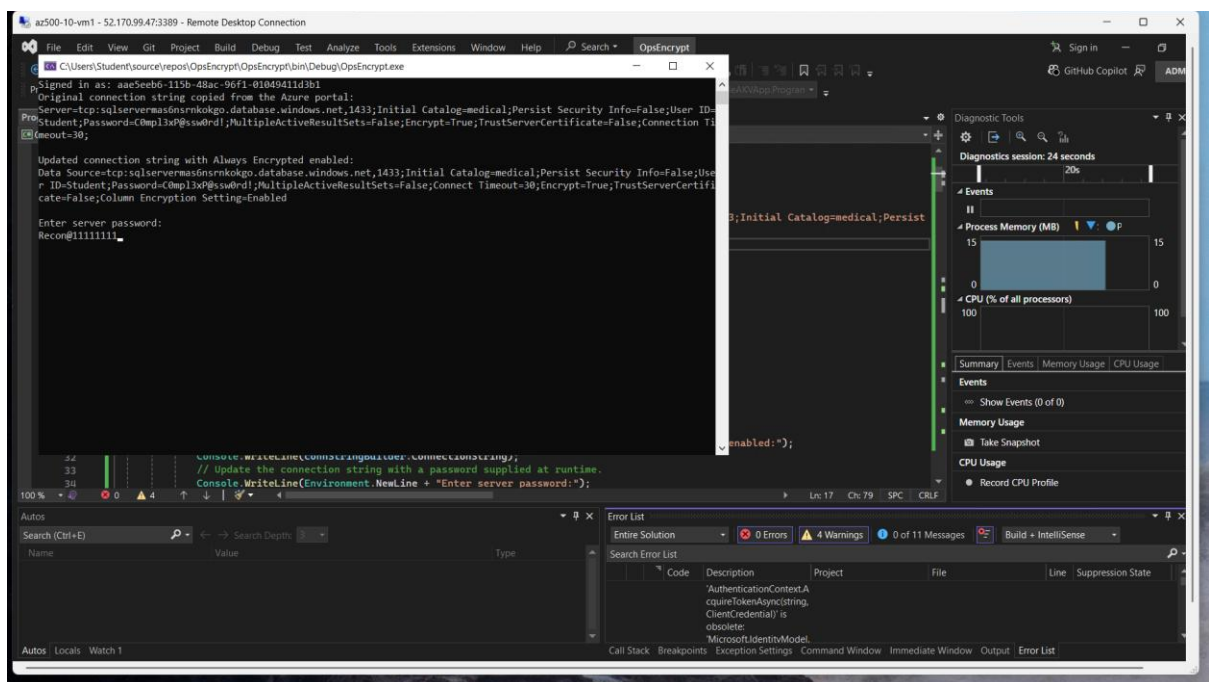


```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using System.Data;
7 using System.Data.SqlClient;
8 using Microsoft.IdentityModel.Clients.ActiveDirectory;
9 using Microsoft.SqlServer.Management.AlwaysEncrypted.AzureKeyVaultProvider;
10 namespace AlwaysEncryptedConsoleAKVApp
11 {
12     1 references
13     class Program
14     {
15         // Update this line with your Medical database connection string from the Azure portal.
16         static string connectionString = @"Server=tcpsqlservermas6nsrnhkqgo.database.windows.net,1433;Initial Catalog=medical;Persist
17         static string clientId = @"<client id noted earlier>";
18         static string clientSecret = "<key value noted earlier>";
19         0 references
20         static void Main(string[] args)
21         {
22             InitializeAzureKeyVaultProvider();
23             Console.WriteLine("Signed in as: " + _clientCredential.ClientId);
24             Console.WriteLine("Original connection string copied from the Azure portal:");
25             Console.WriteLine(connectionString);
26             // Create a SqlConnectionStringBuilder
27             SqlConnectionStringBuilder connStringBuilder =
28                 new SqlConnectionStringBuilder(connectionString);
29             // Enable Always Encrypted for the connection.
30             // This is the only change specific to Always Encrypted
31             connStringBuilder.ColumnEncryptionSetting =
32                 SqlConnectionColumnEncryptionSetting.Enabled;
33             Console.WriteLine(Environment.NewLine + "Updated connection string with Always Encrypted enabled:");
34             Console.WriteLine(connStringBuilder.ConnectionString);
35             // Update the connection string with a password supplied at runtime.
36             Console.WriteLine(Environment.NewLine + "Enter server password:");
37             connStringBuilder.Password = Console.ReadLine();
38             // Assign the updated connection string to our global variable.
39             connectionString = connStringBuilder.ConnectionString;
```

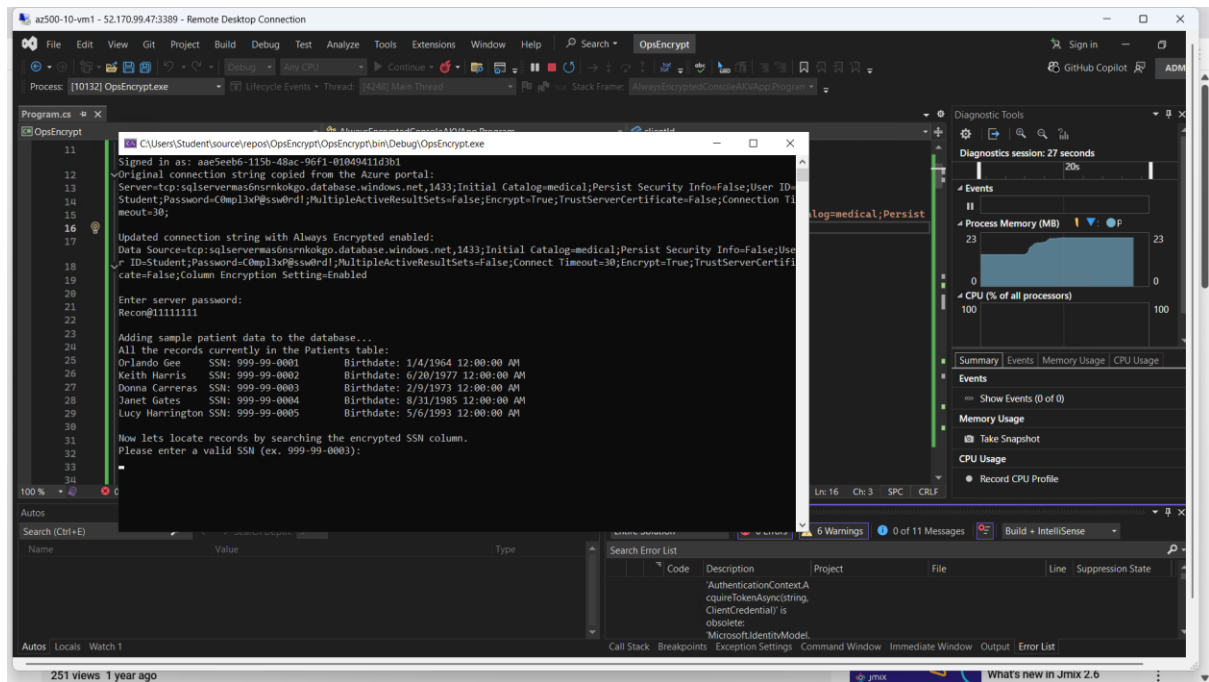
In the Visual Studio window, in the **Program.cs** pane, in line 16, I then replaced the <client id noted earlier> placeholder with the value of **Application (client) ID** of the registered app I recorded earlier in the lab



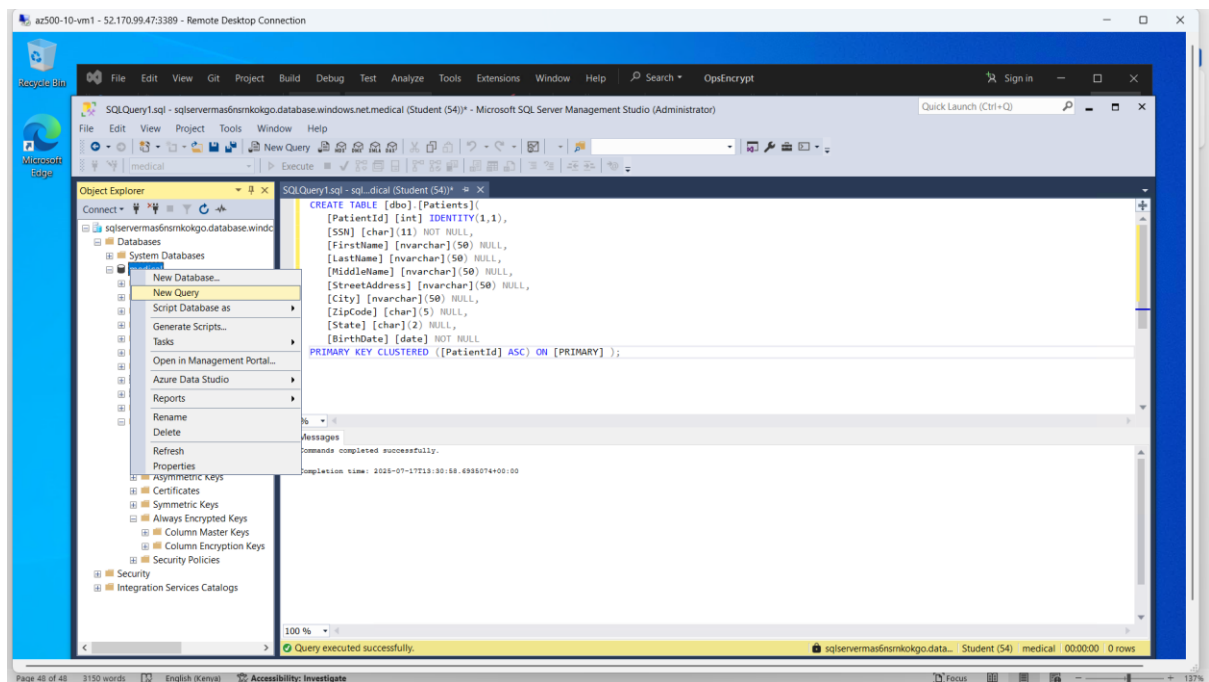
When prompted to enter the password I entered the password I used in the set up of exercise 1:



It started adding sample data below to the database:

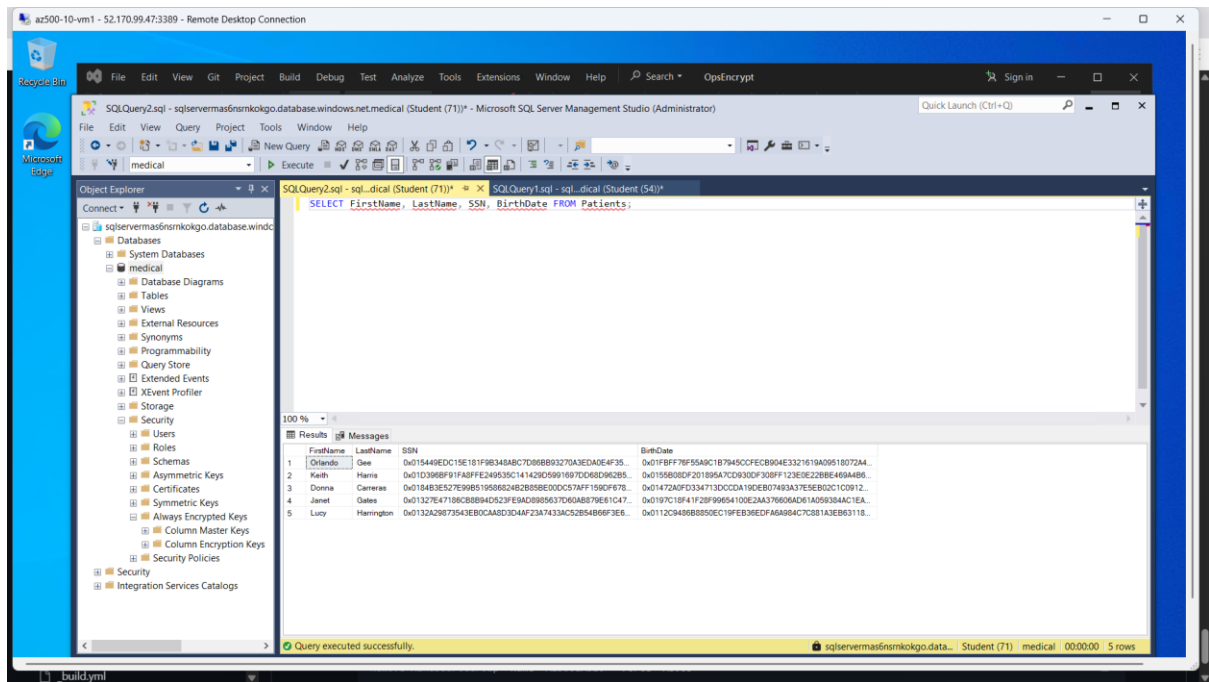


I left the app running and switched to the SQL management studio console, where under the Object Explorer pane I right clicked on the Medical database then clicked on New query:



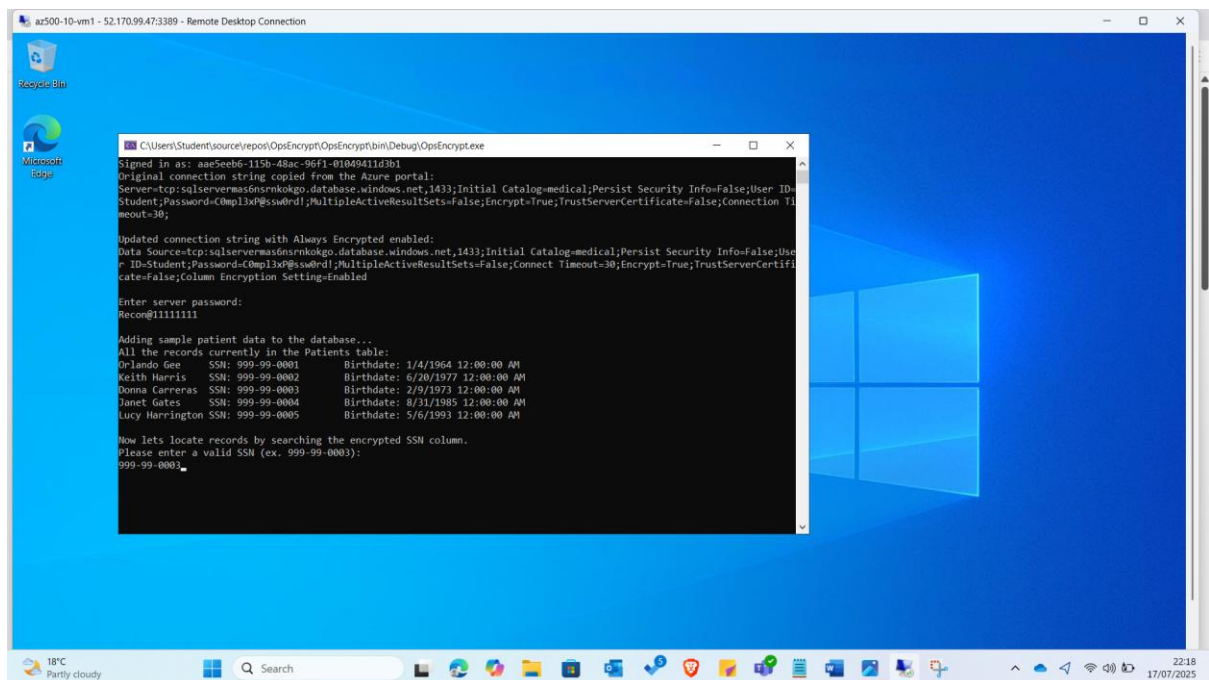
From the query window, I ran the following query to verify that the data that loaded into the database from the console app is encrypted:

SELECT FirstName, LastName, SSN, BirthDate FROM Patients;

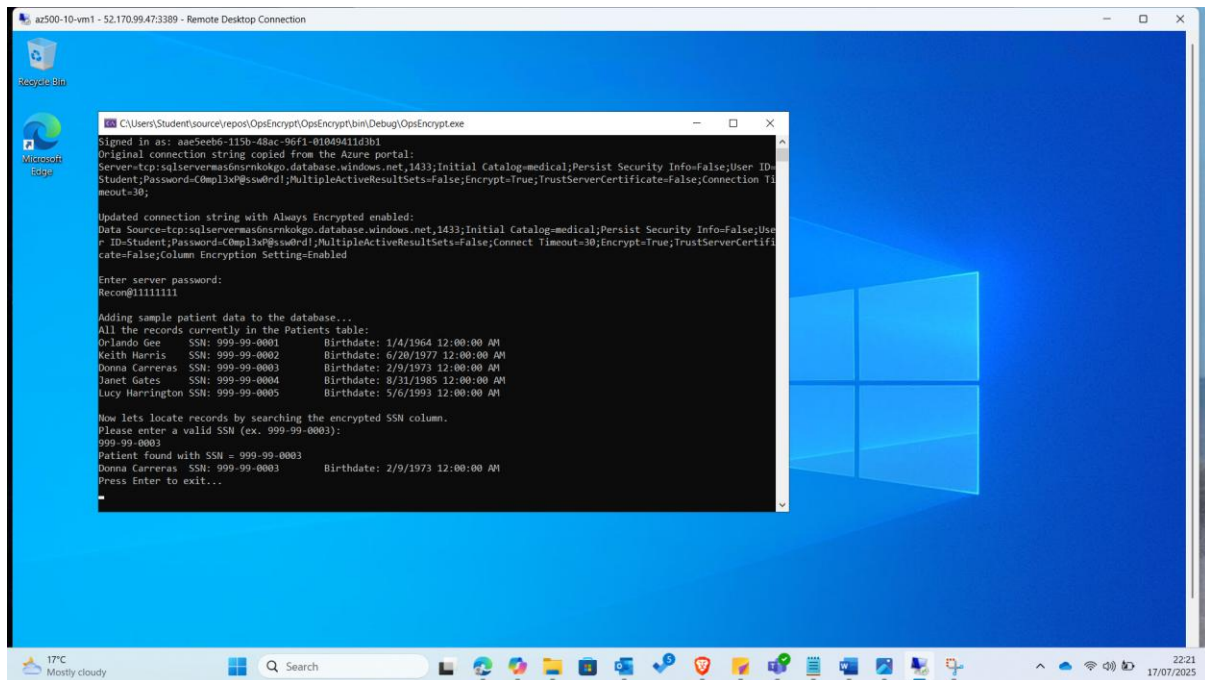


I then Switched back to the console application where I was prompted to enter a valid SSN. This will query the encrypted column for the data. At the Command Prompt, I typed the following and press the Enter key:

999-99-0003



On pressing enter, the data returned by the query was unencrypted as shown below:

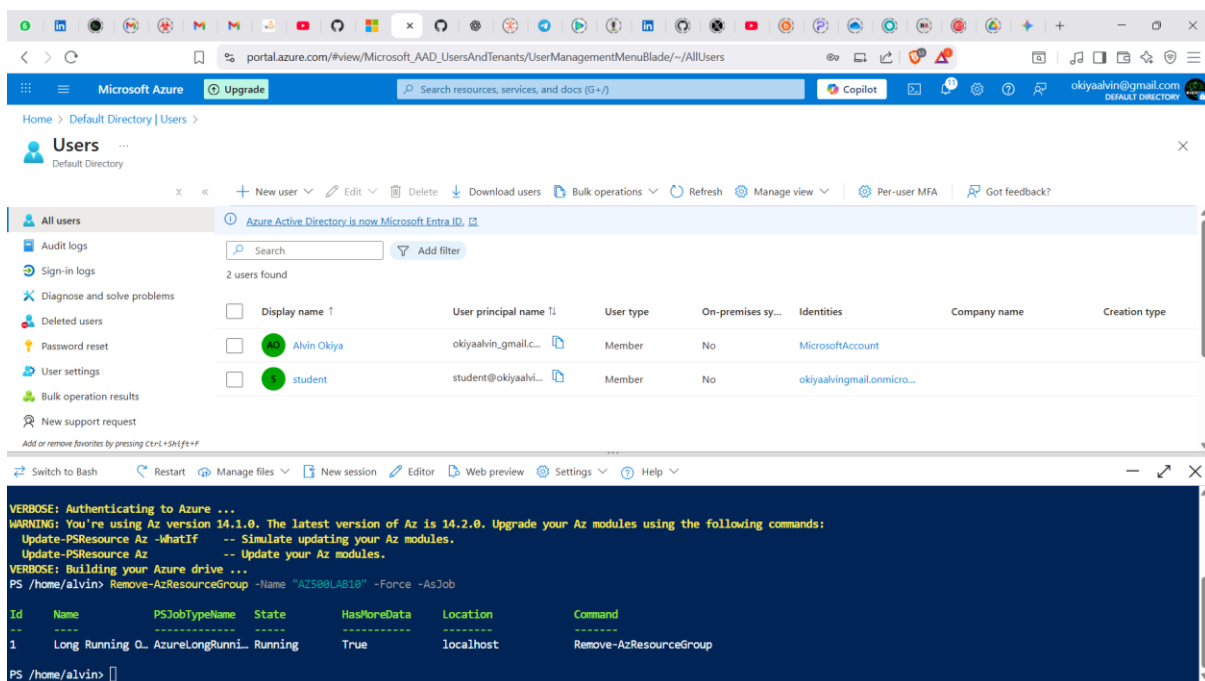


I then terminated the console app by pressing the enter key.

Clean up resources

To clean up the resources, I first stopped the RDP session and went back to the azure portal where I opened the powershell and run the following command, so as to delete the resource group I used in the lab:

Remove-AzResourceGroup -Name "AZ500LAB10" -Force -AsJob



Conclusion

By completing this lab, one will have demonstrated how Azure Key Vault can be integrated with Azure SQL Database Always Encrypted to safeguard sensitive data while supporting secure application access. Keys and secrets were provisioned and centrally managed in Key Vault; an application identity was registered in Microsoft Entra ID and granted the necessary Key Vault permissions; database columns were encrypted using a column master key stored in the vault; and a sample data-driven application was used to insert and retrieve encrypted data, validating end-to-end protection. This lab reinforces best practices for separating key management from data storage, reducing exposure of sensitive values, and strengthening an organization's security posture in the cloud. As always, deprovisioning unused lab resources helps prevent unnecessary cost and keeps the environment clean.